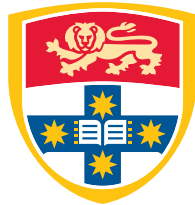

State Aware Risk Controller (STARC) for the Stopping Decision in Agentic Systems



THE UNIVERSITY OF
SYDNEY

School of Computer Science
The University of Sydney

Thesis

Author: Harry Kember

Academic Supervisor: Ying Zhou

A thesis submitted in fulfilment of the requirements of the degree of
Bachelor of Advanced Computing (Computational Data Science) and
Bachelor of Science (Mathematics)

May 2026

Abstract

Large Language Models (LLMs) are increasingly deployed as agentic Question Answering (QA) systems that clarify, gather evidence, and refine an answer over several rounds rather than producing it in a single pass. AgenticLU is one such system, refining its answers through a Chain of Clarifications (CoC) workflow. Clarification is widely assumed to be beneficial, but this is not always so. There exists a control problem with no settled answer: further clarification can repair a weak answer, but it can equally waste computation, entrench a wrong reasoning path, or push a correct answer off course. Existing approaches make the stop or continue decision from a model’s confidence in its answer, a signal that is known to be unreliable.

A case study on AgenticLU continuation trajectories in the Natural Questions dataset, together with the surrounding literature, shows that further reasoning is not always beneficial in agentic systems. Across the trajectories examined, a second round of clarification predominantly preserves the first answer, but it can also repair or degrade one. This investigation yields key insights about these outcomes that motivate the approach the thesis takes to deciding when to stop.

At the first clarification boundary, where a completed round has produced a provisional answer to be assessed, a well-defined control decision arises: stop, or continue. The central proposal of this thesis is that this decision should be made not from the model’s confidence in its answer, but from the evidence carried in the agentic system state that produced it.

This thesis proposes State Aware Risk Controller (STARC), an architecture which evaluates the agentic state and produces a stop or continue decision. STARC is implemented through three progressively layered policies,

- i. Verification and Refinement Baseline (VRB);
- ii. Verification and Refinement with Risk Signals (VRRS); and
- iii. Verification and Refinement with Risk Signals and Borderline Review (VRRS-BR).

The policies are evaluated offline on a fixed replay suite drawn from four Retrieval-Augmented

Generation (RAG) QA datasets: Natural Questions, PopQA, TriviaQA, and HotpotQA. Replaying identical saved states through each policy isolates differences in stopping logic from variation in the underlying system. The strongest, VRRS-BR, raised continue recall from 45% to 67% over the simplest policy. This gain does not come at the expense of overall judgement quality: balanced accuracy rose from 66% to 74% and the Matthews Correlation Coefficient (MCC) from 0.35 to 0.45 alongside it.

Together these establish a single claim: knowing when to stop in agentic QA is a judgement to be read from the agentic system state, not from confidence in the answer.

Declaration

I, Harry Kember, declare that this thesis and its work are my own. Information obtained elsewhere has been correctly acknowledged.

I have read and understood the University of Sydney Student Plagiarism: Coursework Policy and Procedure;

I understand that failure to comply with the Student Plagiarism: Coursework Policy and Procedure can lead to the University commencing proceedings against me for potential student misconduct under Chapter 8 of the University of Sydney By-Law 1999 (as amended);

This Work is substantially my own, and to the extent that any part of this Work is not my own I have indicated that it is not my own by Acknowledging the Source of that part or those parts of the Work.

To the best of my knowledge and belief, this work contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of The University of Sydney or other institute of higher learning, except where due acknowledgement has been made in the text. I acknowledge that Generative-AI tools, including OpenAI's ChatGPT and GitHub's Copilot, assisted in the development of this thesis by aiding in sentence structuring, spell checking, grammatical checking and overall response formatting. However, the research, ideas, critical thought, results and analysis presented in this thesis are entirely my own.

A handwritten signature in black ink that reads "H Kember". The signature is stylized with a large, sweeping initial "H" and a long, horizontal flourish extending from the end of the name.

Harry Kember

May 2026

Disclaimer: The views expressed in this thesis are those of the student and do not necessarily express the views of their supervisor or The University of Sydney

Acknowledgements

This thesis would have been far more challenging without the invaluable guidance, support, and encouragement of the following people, to whom I am deeply grateful.

I am sincerely thankful to my academic supervisor, Prof. Ying Zhou, for her insightful feedback and guidance in shaping and developing my ideas.

I express my heartfelt thanks to my parents, Christine and Warren, for their unwavering support throughout my academic journey. Their patience, belief in me, and constant encouragement have been a source of strength. Without their dedication, hard work, and sacrifice, completing this thesis would not have been possible.

To my wonderful partner, Pauline: thank you for your patience and constant support throughout. I couldn't have done this without you.

I am grateful to my grandfather, Harry, whose wisdom and lectures shaped how I think in ways that reach well beyond this thesis.

I also thank my brother, Samuel, for his generous help and support throughout this process; his guidance and advice were greatly appreciated.

Contents

Abstract	i
Declaration	iii
Contents	v
List of Figures	xii
List of Tables	xiv
List of Acronyms	xviii
List of Abbreviations and Terms	xx
1 Introduction	1
1.1 Research problem	2
1.2 Research question	2
1.3 Contributions	3
1.4 Scope	4
1.5 Thesis structure	5
2 Literature Review	6
2.1 Long context question answering and the evidence use problem	7
2.1.1 The U curve and what it implies	8

2.1.2	Why longer windows do not solve it	9
2.1.3	Retrieved evidence is not predictive on its own	11
2.2	Agentic reasoning and the observability of intermediate state	11
2.2.1	Final success is the wrong lens	13
2.3	Clarification as a reasoning move, and what it does <i>not</i> answer	14
2.4	Verification, refinement, and the conditional value of further reasoning	15
2.4.1	Why model confidence is not a safe stopping signal	17
2.4.2	Structured judgement: an instrument with its own biases	19
2.5	Stopping, selective prediction and inference control	20
2.5.1	How prior agentic and sampling systems decide when to stop	21
2.6	Evaluating a stopping controller: replay, imbalance and cost	23
2.6.1	Premature stopping and unnecessary continuation are not symmetric	24
2.7	Where the literature stops, and what this thesis adds	25
3	Background: Long Context Setting and Evaluation Datasets	26
3.1	The AgenticLU framework	26
3.2	The Chain of Clarifications workflow	27
3.2.1	The four operations of a single round	28
3.2.2	Single round and multi round configurations	29
3.3	The HELMET long context evaluation setting	30
3.3.1	Context variants and answer judging	31
3.4	The evaluation datasets	31
3.4.1	The four source datasets	31
3.4.2	From short context collections to long context RAG inputs	33
3.4.3	Why these four	34
3.4.4	Provenance and limitations	34

4	Methodology	36
4.1	Data and saved objects	37
4.1.1	Base questions, context variants and replay examples	38
4.1.2	Saved round states and the first clarification boundary	38
4.2	Stage 1: round state instrumentation	39
4.2.1	Purpose	39
4.2.2	Inherited runtime and added instrumentation	39
4.3	Stage 2: continuation case study	41
4.3.1	Purpose and unit of analysis	41
4.3.2	Transition categories	41
4.3.3	Manual judging	42
4.3.4	Family and example level analysis	43
4.3.5	Contribution to the controller design	43
4.4	Stage 3: State Aware Risk Controller (STARC)	44
4.4.1	Replay suite and decision context	45
4.4.2	Threshold bands	46
4.4.3	Verification and refinement baseline (VRB)	48
4.4.3.1	System overview	48
4.4.3.2	Verifier and falsifier	49
4.4.3.3	Verifier	49
4.4.3.4	Falsifier	52
4.4.3.5	Decision construction	53
4.4.4	Verification and refinement with risk signals (VRRS)	54
4.4.4.1	System overview	55
4.4.4.2	Hard and soft risk definitions	56
4.4.4.3	Signal families	57

4.4.4.4	Decision construction	60
4.4.5	Verification and refinement with risk signals and borderline review (VRRS-BR)	62
4.4.5.1	System overview	62
4.4.5.2	Semantic reviewer	63
4.4.5.3	Final continuation filter	65
4.5	Evaluation protocol	67
4.5.1	Procedure	67
4.5.2	Evaluation objects	67
4.5.3	Stopping task and outcome cells	68
4.5.4	Metrics	69
4.5.5	Judging the labels	71
4.5.6	Validity controls and limits	72
4.6	Chapter summary	73
5	Results	75
5.1	Evaluative framing	76
5.2	Case study results: the effect of a second clarification round	77
5.2.1	Trajectories of agentic continuation states	79
5.2.1.1	Changed round outcomes	79
5.2.1.2	Preserved round outcomes	81
5.2.2	Family distribution of the continuation mechanisms	83
5.2.3	What the case study establishes for the controller evaluation	85
5.3	The offline replay suite	86
5.4	Main stopping gate policy results	86
5.4.1	Aggregate comparison	86
5.4.2	Where the improvement comes from	88

5.4.3	The borderline review mechanism	90
5.5	Disaggregated policy behaviour	92
5.5.1	Dataset results	92
5.5.2	Which layer carried the improvement	96
5.6	Statistical robustness and uncertainty analysis	98
5.6.1	Cluster bootstrap intervals	99
5.6.2	Paired cluster comparisons	100
5.6.3	Cluster collapsed McNemar check	101
5.6.4	Robustness of band placements	102
5.6.5	Interpretation of statistical robustness	103
5.7	Chapter summary	103
6	Discussion	105
6.1	What the case study's typology says about controller design	106
6.2	STARC as the central contribution	107
6.3	Cost asymmetric redistribution and why it is appropriate	109
6.4	Broader applicability of the staged evaluator pattern	111
6.5	Conclusions	112
7	Limitations	113
7.1	Replay design	113
7.1.1	The scale of the empirical evaluation	113
7.2	Judge dependence at every stage	114
7.3	Scope of evaluation	115
7.4	Binary correctness as the desired action signal	115
7.5	What the limitations do not threaten	116

8 Conclusion	117
8.1 What the thesis established	117
8.2 What generalises beyond this setting	118
8.3 The central argument	118
9 Future Work	120
9.1 The staged evaluator at deeper clarification boundaries	120
9.2 Refining the deterministic stage for routing rather than adjudication	121
9.3 Transfer to other agentic control boundaries	121
9.4 Isolating the dual perspective split	122
9.5 Cost weighted evaluation that does not flatten answer value	123
9.6 Closing	123
List of References	124
A Literature Review Supporting Material	132
A.1 Summary of reviewed literature	132
B Methodology and Reproducibility Objects	136
B.1 AgenticLU round state saving method specification	136
B.2 Answer judging protocol	139
B.3 Case study method specification	144
B.4 VRB method specification	145
B.5 VRRS method specification	154
B.6 VRRS-BR method specification	155
B.7 Policy implementation	162
C Results and Supporting Objects	163
C.1 Case study supporting results	163

C.2	Controller evaluation result tables	168
C.3	Policy example bank: gate decision and failure mode evidence	171
C.4	Statistical robustness and supplementary figures	174

List of Figures

2.1	The lost in the middle U curve.	9
2.2	Three inference styles on a single HotpotQA question.	12
2.3	Confidence elicitation disagrees across regimes.	18
3.1	AgenticLU’s Chain of Clarifications (CoC) workflow: single and multi round clarification.	28
4.1	The CoC system, with instrumented state saving.	40
4.2	VRB system architecture.	48
4.3	Verifier prompt structure.	50
4.4	Falsifier prompt structure.	52
4.5	VRB deterministic decision construction.	53
4.6	VRRS system architecture.	56
4.7	VRRS-BR system architecture.	63
4.8	Semantic reviewer prompt structure.	64
4.9	VRRS-BR deterministic decision construction.	65
4.10	Outcome cells of the stopping task.	68
5.1	Agentic system states over two rounds of AgenticLU’s Chain of Clarifications (CoC) for changed round outcomes.	80
5.2	Agentic system states over two rounds of AgenticLU’s Chain of Clarifications (CoC) for the preserved round outcomes.	82

5.3	Distribution of the four continuation mechanisms across the six question families.	84
5.4	Controller trade off profile across the combined replay suite.	89
5.5	The borderline review pipeline applied to the replay suite.	91
5.6	Continue recall by dataset and policy.	93
5.7	Dataset level effect of each added policy layer.	97
5.8	Cluster bootstrap balanced accuracy intervals.	100
5.9	Matthews Correlation Coefficient (MCC) range under value perturbations of each band, grouped by policy.	102
C.1	Cluster bootstrap continue recall intervals	179
C.2	Cluster bootstrap MCC intervals	180
C.3	Confusion cell counts under sweep of each band placement.	182
C.4	Detail sweep for the VRRS regression risk placement.	183
C.5	Detail sweep for the VRRS-BR reviewer’s false answer risk placement.	184

List of Tables

2.1	Claimed versus effective context length on the RULER benchmark [1].	10
2.2	Mapping prior work phenomena onto the four continuation outcomes.	16
2.3	Stopping criteria in prior agentic and sampling systems.	22
3.1	Four operations in a single Chain of Clarifications round.	29
3.2	The four inherited QA datasets used for evaluation.	33
4.1	Case study transition categories.	42
4.2	Manual case study judging rules.	43
4.3	Construction of the offline first round replay suite.	45
4.4	Shared first round decision context fields used by all three policies.	46
4.5	Band definitions and their role in the stopping decision rule.	47
4.6	Verifier fields and output types.	51
4.7	Falsifier fields and output types.	53
4.8	QA form family: signals, construction procedures and trigger conditions. . . .	58
4.9	Grounding family: signals, construction procedures and trigger conditions. . .	59
4.10	Clarification quality family: signal, construction procedure and trigger condition.	59
4.11	VRB risk family: signals, construction procedures and trigger conditions. . . .	60
4.12	Ordered VRRS decision rule.	60
4.13	VRRS-BR final continuation filter.	66
4.14	Controller metrics and their interpretive role in the stopping gate evaluation. .	71

5.1	The six question families that produced the case study trajectories, with short identifiers used in the rest of the section.	78
5.2	Aggregate transition category counts across the case study trajectories.	78
5.3	Offline replay suite composition used for stopping gate evaluation.	86
5.4	Stopping gate outcome counts by policy.	87
5.5	Aggregate stopping gate metrics by policy.	87
5.6	Incremental effect of each added policy layer.	89
5.7	Per dataset controller performance.	94
5.8	Cluster bootstrap 95% intervals for the main controller metrics.	99
5.9	Paired cluster comparisons on gate success rate.	100
5.10	Cluster collapsed McNemar exact checks.	101
A.1	Summary of literature relevant to adaptive stopping in clarification driven, long context question answering.	135
B.1	Saved AgenticLU round state fields	137
B.2	Clarification state subfields	138
B.3	Representative manual case study judging examples	140
B.4	Known limitations of the automated round correctness judge	144
B.5	Effective VRB first round inputs	146
B.6	Verifier prompt inputs and sources	146
B.7	Verifier JSON field meanings	149
B.8	Falsifier prompt inputs and sources	150
B.9	Falsifier JSON field meanings	152
B.10	Clean stop conditions over the parsed verifier and falsifier records.	153
B.11	Ordered continuation triggers used by VRB.	153
B.12	Deterministic gate inputs	155
B.13	Semantic review trigger condition	156

B.14 Semantic review prompt inputs	156
B.15 Semantic review JSON fields	158
B.16 Semantic review issue types	159
B.17 Semantic review continuation filter	159
B.18 Worked VRRS-BR semantic review stop example	160
B.19 Worked VRRS-BR semantic review continue example	161
C.1 Case study question family reference table.	163
C.2 Case study family transition counts.	164
C.3 Full case study transition table, grouped by family: F1-F2.	165
C.4 Full case study transition table, grouped by family: F3.	166
C.5 Full case study transition table, grouped by family: F4.	166
C.6 Full case study transition table, grouped by family: F5-F6.	167
C.7 Representative case study transition examples.	168
C.8 Combined controller comparison: summary metrics.	168
C.9 Combined controller comparison: confusion cells.	169
C.10 Per dataset controller confusion counts.	169
C.11 Per dataset controller performance metrics.	170
C.12 Borderline review decision outcomes.	170
C.13 Final outcomes among borderline cases.	171
C.14 Semantic review issue distribution.	171
C.15 Stratified gate decision example bank.	172
C.16 Example identifiers for the stratified gate decision example bank.	172
C.17 Borderline review example bank.	173
C.18 Failure mode and trade off examples.	174
C.19 Base question cluster structure of the replay suite	175

C.20 Cluster bootstrap intervals	176
C.21 Observed controller point estimates on the 408 example replay suite	177
C.22 Paired cluster comparisons on mean gate success rate	177
C.23 Cluster collapsed McNemar exact checks	178
C.24 Summary of the band placement sweep over the offline replay suite.	181

List of Acronyms

AUROC Area Under the Receiver Operating Characteristic Curve

CoC AgenticLU's Chain of Clarifications

CoT Chain-of-Thought

ECE Expected Calibration Error

ICL In Context Learning

KILT Knowledge Intensive Language Tasks

LLM Large Language Model

MCC Matthews Correlation Coefficient

NIAH Needle-in-a-Haystack

QA Question Answering

RAG Retrieval-Augmented Generation

STARC State Aware Risk Controller

VRB Verification and Refinement Baseline

VRRS Verification and Refinement with Risk Signals

VRRS-BR Verification and Refinement with Risk Signals and Borderline Review

List of Abbreviations and Terms

Agentic

Describing AI systems that set goals, form multi step plans, and take actions autonomously to complete complex tasks.

AgenticLU

Agentic Long Context Understanding; the long context QA framework, built on a fine tuned Llama-3.1-8B-Instruct model, whose Chain of Clarifications workflow this thesis instruments and controls.

AmbigQA

A QA task and dataset for handling ambiguous questions.

Brier score

A proper scoring rule measuring the mean squared error between predicted probabilities and outcomes.

CRITIC

A framework in which an LLM evaluates and revises its own outputs using external feedback.

DeeBERT

An early exit technique for accelerating BERT inference.

FastBERT

A speed tunable BERT variant with adaptive inference.

FEVER

Fact Extraction and VERification; a fact verification dataset.

Found-in-the-Middle

A calibration technique addressing the lost-in-the-middle positional bias.

Google Colab

A cloud based platform for running Python in the browser; the compute environment used for this work.

GPT-4o

The OpenAI model used as the verifier, falsifier, and semantic reviewer in STARC’s LLM based stages (run at temperature 0.0).

HELMET

How to Evaluate Long context Models Effectively and Thoroughly; the long context benchmark suite providing the evaluation setting.

HotpotQA

A multi hop reasoning QA benchmark.

IRCoT

Interleaved Retrieval with Chain-of-Thought reasoning.

KILT

Knowledge-Intensive Language Tasks; a benchmark collection.

LLM-as-a-judge

The paradigm of using one language model to evaluate another’s outputs; the basis for STARC’s verifier, falsifier, and reviewer stages.

Lost-in-the-middle

The phenomenon whereby LLMs use information less reliably when it sits in the middle of a long context.

LooGLE

A long context language-understanding benchmark.

McNemar’s test

A paired statistical test for whether two classifiers differ on the cases where their decisions disagree; used in the statistical robustness analysis.

MuSiQue

Multi hop Questions via Single hop Question composition; a multi hop QA dataset.

NarrativeQA

A reading comprehension benchmark testing deep narrative understanding over long texts.

Natural Questions

A large scale QA dataset of real Google search queries with Wikipedia based answers.

NoLiMa

No Literal Matching; a long context probe that defeats lexical overlap shortcuts.

PopQA

An entity centric dataset testing factual recall across common and long tail entities.

Qasper

A QA benchmark over scientific papers requiring multi section synthesis.

QuAC

Question Answering in Context; a conversational QA dataset.

QuALITY

Question Answering with Long Input Texts, Yes; a multiple choice long context QA benchmark.

ReAct

A prompting paradigm (Yao et al.) that interleaves reasoning and action; a canonical example of agentic inference.

RULER

A configurable benchmark (NVIDIA) for measuring a model’s effective long context length.

Self-RAG

A retrieval-augmented generation method that decomposes the pipeline into learned reflection tokens governing retrieval and self critique.

TriviaQA

A large reading comprehension dataset of question answer evidence triples.

Chapter 1

Introduction

Question answering (QA) systems built on Large Language Models (LLMs) are increasingly agentic: rather than generating an answer in a single forward pass, they clarify the question, reason over retrieved evidence, and refine their answer across multiple rounds before responding. The premise underlying these systems is that another round of reasoning improves the answer. Often it does, but not always: a further round can equally waste computation, entrench a wrong reasoning path, or take an answer the system already had right and push it wrong. Existing systems rarely decide whether the round is worth running.

AgenticLU [2] is one such system. It runs an explicit Chain of Clarifications (CoC) workflow at inference time, executing four operations per round (clarification question, pointback, clarification answer, provisional answer) and looping for one or more rounds before producing a final response. The workflow is designed to close the gap between a model’s nominal and effective use of a long context input: rather than relying on the base model to extract evidence in a single pass, the system iterates, clarifying what the question is asking and grounding the answer in the long context. The design assumes that each additional round localises evidence, sharpens the question, and improves the response. For ambiguous, incomplete, or weakly grounded first round answers, the assumption holds and clarification is a useful intervention. It is not universally correct.

This thesis treats the decision of whether to continue clarifying as a problem in its own right. It builds the evidence that an additional clarification round is only conditionally beneficial, and it develops the **State Aware Risk Controller (STARC)**, an architecture that decides whether to stop or continue by assessing the agentic system state a completed round leaves behind, rather than trusting the model’s confidence in its answer.

1.1 Research problem

A clarification driven Question Answering (QA) system that has completed a first clarification round and produced a provisional answer faces a control decision: accept that answer, or continue clarifying. The decision is treated implicitly in current implementations, with the system setting fixed depth clarification regardless of the state produced. Adjacent literature on adaptive computation uses model confidence instead, stopping when the model is confident and continuing when it is not. Neither approach investigates the full state at the boundary between rounds. Fixed depth scheduling cannot, because no state is consulted. Confidence based scheduling can, but confidence is unreliable as a signal of correctness in long context QA, where the calibration failures of LLM confidence are well established [3, 4].

The control problem this thesis studies is the first clarification boundary in the CoC workflow. The moment immediately after the first clarification round, when the system has produced a provisional answer, is the point at which a controller should decide whether to accept the answer or to continue to a second round. Treating that moment as an explicit decision requires three things the literature leaves undefined: a way to observe the boundary state, an understanding of what continuation does to an answer, and a controller that reads the state to decide. These map to the research question and contributions that follow.

1.2 Research question

The thesis answers one question: given the saved state at the first boundary of a clarification driven workflow, can a stopping policy reliably decide whether to accept the provisional answer or continue, and do so consistently across the heterogeneous question types an agentic QA system encounters?

The question breaks into three operational components, which map to the three core contributions.

- i. The boundary state must be made observable in a form that supports principled decisions;
- ii. the behaviour of continuation must be characterised across the mechanisms an additional round can produce; and
- iii. a controller must be built to answer the proposed boundary question effectively, and rigorously evaluated.

1.3 Contributions

This thesis makes three contributions that share one idea: that stopping at a clarification boundary is best treated as an evaluation of the saved system state, rather than answered by reasoning depth or model confidence alone.

The first contribution makes the stopping problem observable: a saved agentic state framework that instruments AgenticLU’s Chain of Clarifications (CoC) runtime to preserve the full state of the workflow at each clarification boundary. This converts an internal computation into a structured object that can be read, classified, and compared, and the offline replay design used to construct and compare controllers rests on it directly. Without an observable boundary, the stopping problem in clarification driven QA cannot be evaluated cleanly.

The second contribution establishes that continuation is only conditionally beneficial, and characterises the four mechanisms by which a second clarification round affects a first round answer: repair of a wrong answer, preservation of an already correct answer, persistent failure that cycles through the same evidence path, and degradation of a correct answer the first round had right. The largest category is preservation, not repair: a significant part of the value of continuation is from accepting correct answers cheaply. Thus, a controller must focus on detection of wrong answer cases. Each mechanism leaves a different signature in the first round state, and the four place different demands on any controller built to detect them, which motivates the layered design of the third contribution.

The third is the **State Aware Risk Controller (STARC)**, a staged evaluator architecture designed and evaluated as three progressively layered policies on a replay suite drawn from Natural Questions, PopQA, TriviaQA, and HotpotQA.

- i. Verification and Refinement Baseline (VRB) produces dual perspective Large Language Model (LLM) judgements through a verifier and a falsifier;
- ii. Verification and Refinement with Risk Signals (VRRS) adds a deterministic stage that constructs risk classifications; and
- iii. Verification and Refinement with Risk Signals and Borderline Review (VRRS-BR) adds a semantic reviewer invoked when the deterministic stage cannot resolve a case.

Across the full replay suite, VRRS-BR holds the highest mean on every class balanced metric: continue recall rises from 44.66% to 66.99%, balanced accuracy from 66.10% to 73.99%, and Matthews Correlation Coefficient (MCC) from 0.346 to 0.450. It catches 67.0% of wrong answers against VRB’s 44.7%.

STARC’s organising principle is that the LLM evaluates the agentic system trace rather than producing the answer, with deterministic logic composing the resulting judgements into a decision. The pattern it instantiates aligns with directions emerging independently in adjacent work on adaptive RAG, agentic verification, and process reward modelling.

The methodology developed in this thesis is released as a public repository. It contains the instrumented AgenticLU runtime, the construction of the replay suite, the prompts and decision logic for the three policies VRB, VRRS and VRRS-BR, and the statistical analysis pipeline used in Chapter 5. Due to size constraints, the saved AgenticLU states cannot be included.

- Public mirror: <https://github.com/haz123456/STARC-Stopping-in-Agentic-Systems>
- University of Sydney: <https://github.sydney.edu.au/Hkem6751/STARC-Stopping-in-Agentic-Systems>

1.4 Scope

Three choices fix the scope of the work.

The control point is the first clarification boundary of AgenticLU’s Chain of Clarifications (CoC) workflow, where a provisional answer has been produced and the system must decide whether to continue. The first boundary is the one at which state has been produced exactly once, so the controller’s input space is well defined. Deeper boundaries inherit history that the present design does not handle.

The evaluation runs on Natural Questions, PopQA, TriviaQA, and HotpotQA through the HELMET long context setting [5]. These four are the datasets AgenticLU is calibrated against; using the same set holds the underlying system’s behaviour constant across the controller comparison.

The base model is AgenticLU’s calibrated model. The contribution is a controller built around the CoC workflow; switching models would change the workflow under study.

Within those choices, the thesis claims controller behaviour at this boundary, on these datasets, with this model, and nothing wider. What transfers beyond them is the staged evaluator pattern itself.

1.5 Thesis structure

The remainder of the thesis is organised as follows. Chapter 2 reviews the literature that constrains the stopping problem (long context QA, agentic reasoning, clarification, verification and refinement, selective prediction and evaluation under class imbalance), identifying what each literature contributes and the gap the thesis fills.

Chapter 3 defines the three inherited components on which the technical work builds: the AgenticLU framework, its CoC workflow, and the HELMET long context evaluation setting, and introduces the datasets to be used.

Chapter 4 presents the methodology in three stages, each delivering one contribution. Stage 1 delivers the round state framework: it instruments the CoC runtime so that round states are preserved at explicit boundaries. Stage 2 delivers the continuation case study: it uses saved trajectories to characterise how an answer changes when the system continues from the first to the second clarification round. Stage 3 delivers STARC, the staged evaluator architecture: it treats the first clarification boundary as a stopping decision and evaluates the three progressively layered policies (VRB, VRRS, VRRS-BR) over saved first round states.

Chapter 5 reports the empirical results, including the aggregate stopping metrics, the per dataset attribution showing which stages of STARC do the work on which question types, and the case study findings on the four continuation mechanisms.

Chapter 6 develops the STARC argument across the policy family, situates the architecture against adjacent literature, and works through the cost asymmetric framing that connects the empirical results to deployment regimes.

Chapter 7 states the bounds of the empirical work.

Chapter 8 revisits the three contributions and frames what the thesis adds to the broader question of stopping in agentic QA.

Chapter 9 sets out the next studies that the present work makes possible.

Chapter 2

Literature Review

Adaptive stopping in clarification driven question answering sits at the intersection of several literatures: long context Question Answering (QA), agentic and clarification reasoning, verification and refinement, selective prediction and adaptive computation, and evaluation under class imbalance. These threads are reviewed here against a single criterion. Each contributes a constraint that any first round stop or continue policy must respect. No single literature addresses whether a clarification system should halt after producing a provisional answer, but collectively the threads motivate the question and constrain how it can be answered.

Six threads frame the problem, and this chapter takes them in turn. Long context QA (Section 2.1) establishes that in these settings evidence is not only hard to find but hard to use: a fluent answer can sit over evidence the model never properly attended to, so answer quality cannot be read from the answer’s surface. Work on agentic reasoning (Section 2.2) shows why this is tractable rather than hopeless: once inference is decomposed into explicit steps, the intermediate state becomes observable, and an observable state is something a controller can inspect. The clarification literature (Section 2.3) establishes that a follow-up reasoning step is, in some cases, a genuinely useful move, while the verification and refinement literature (Section 2.4) establishes its converse: further reasoning is not always beneficial, and can entrench or degrade an answer that was already adequate. Selective prediction and adaptive computation (Section 2.5) supply the framing that resolves this tension: inference should be controlled per example, deciding case by case whether to commit or to do more work. Finally, the literature on evaluation under class imbalance (Section 2.6) supplies the apparatus, offline replay, cost asymmetry, and class balanced metrics, needed to compare such decisions fairly.

Read together, these threads converge on three claims that motivate this thesis. First, *evidence quality is the bottleneck*: the limiting factor in long context QA is whether an answer is genuinely grounded, not whether a passage was retrieved. Second, *intermediate state is the*

lever: because agentic systems expose their reasoning, the state at a clarification boundary carries the evidence a stopping decision would need. Third, *continuation has conditional value*: another clarification round helps in some cases and harms in others, so whether to continue is a real decision rather than a foregone conclusion.

What no existing literature provides is a way to act on these claims at the point where they meet. There is no established account of how to evaluate the first clarification boundary as a per example stop or continue decision, assessed on the quality of that decision itself rather than on the final answer accuracy that follows from it. This is the gap the thesis addresses and is developed in Section 2.7.

2.1 Long context question answering and the evidence use problem

What counts as a long context QA task. Long context question answering is the family of QA problems in which the input the model receives is large enough that it cannot be fully attended to in a single, undifferentiated pass. The defining property is not the absolute size of the prompt, but rather the relationship between input size and the model’s effective use of it. The answer to a long context question depends on evidence that lies somewhere in the input but is not necessarily in a localised, easily retrieved position. As such, the model must do two things at once: locate the relevant evidence and then reason over it to produce a grounded answer. Long context QA combines retrieval and reasoning in a setting where the two cannot be cleanly separated.

The benchmarks in this space were originally constructed for shorter contexts and then extended into the long context regime through the use of prompt packing or retrieval. Natural Questions established the format of an information seeking question paired with a long Wikipedia passage from which the answer should be extracted [6]. HotpotQA introduced the *multi hop* setting, in which a single question requires combining facts from two distinct paragraphs and decoupling the question from any one passage [7]. MuSiQue tightens this further, composing *two hop* and *three hop* questions from chains of *single hop* questions so that solving the composed question requires faithfully traversing each hop rather than relying on shortcuts in any individual paragraph [8]. Knowledge Intensive Language Tasks (KILT) consolidates this body of work as a unified knowledge intensive evaluation setup, treating retrieval and grounded answering as inseparable parts of a single task definition [9]. Across these tasks, the model is consistently asked to do more than generate fluent text. It must locate evidence, combine it across passages, and ground its answer in what it has read.

A parallel task focuses on long inputs in which the evidence containing the answer is dis-

tributed across an entire document rather than across a small set of retrieved passages. Qasper constructs questions over academic papers for which the answer requires reading and synthesising material from multiple sections of the article [10]. QuALITY uses long narrative passages of 5,000 words or more, with multiple choice questions designed to require integration of details from across the passage rather than retrieval of a single span [11]. CLAPNQ extends Natural Questions into long form Retrieval-Augmented Generation (RAG), asking systems to produce cohesive long answers grounded in citations from retrieved passages, with an explicit focus on grounding and faithfulness rather than on surface accuracy alone [12]. The shared feature across these benchmarks is that they cannot be reduced to the extraction of a single span. The system must attend coherently to information spread across the input.

The recurring failure visible in both tasks stems from an inability to attach a fluent answer to the right piece of evidence. A model can produce a confident, fluent answer over a long input that has only weak support in the evidence the model actually used. The broader hallucination literature has documented exactly this dissociation between surface fluency and evidential grounding across a wide range of generation tasks [13]. This has a direct implication for any stopping policy applied in a long context QA setting: the surface form of an answer carries limited information about whether the answer is actually grounded.

2.1.1 The U curve and what it implies

The clearest demonstration of the evidence use problem comes from Liu et al. [14], who show that retrieval style QA accuracy depends sharply on *where* the passage containing the answer sits in the prompt. Their multi document QA task places one gold passage among nineteen so called “distractors” and varies the gold passage’s index in the prompt. They report that “performance is often highest when relevant information occurs at the beginning or end of the input context, and significantly degrades when models must access relevant information in the middle of long contexts.” Figure 2.1 reproduces the position by accuracy numbers from their Appendix Table 6 for three systems. A clear *U shape* appears: evidence in the middle of the prompt is attended to substantially less reliably than evidence at the beginning or end.

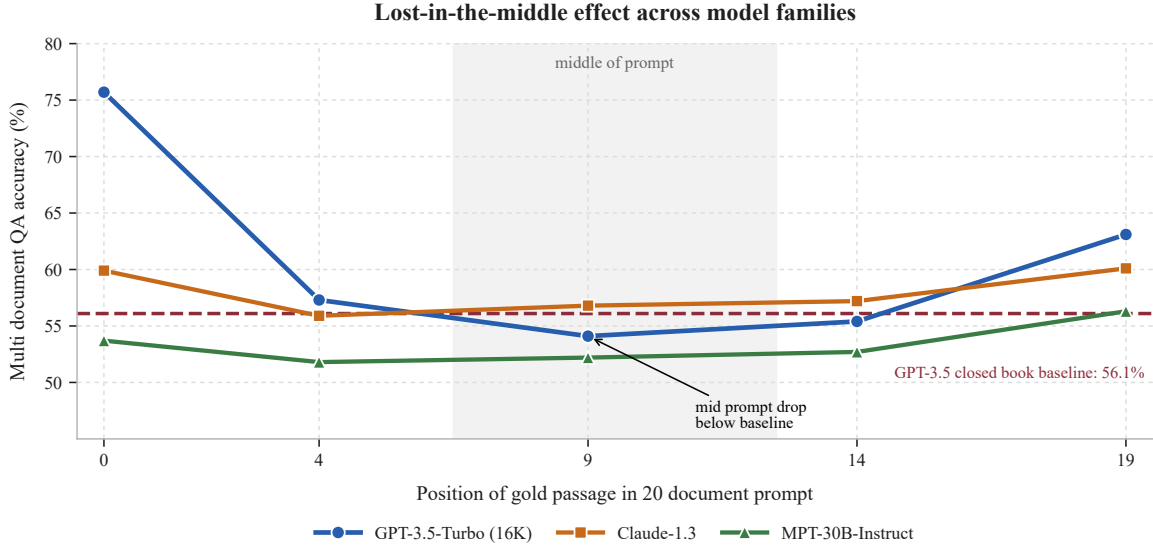


Figure 2.1 – The lost in the middle U curve.

Reproduced from the numbers in Liu et al. [14], Appendix Table 6. For GPT-3.5-Turbo, accuracy in the middle of the prompt falls below the closed book baseline of 56.1% that the same system achieves with no documents at all. The U shape is reduced but not removed in Claude-1.3 and is essentially flat for the weaker MPT-30B-Instruct, indicating that the effect interacts with overall model capability rather than being a universal artefact.

Two consequences follow as a direct result of this. Firstly, the model’s apparent ability to handle long context is a function of where the relevant information sits within the prompt. A first answer produced over a 128K input may have come from a 5K token window the model considered. The rest of the input functions as decorative padding. Secondly, the model’s confidence in its answer is silent about which of these situations holds. The answer in the strong case and the answer in the weak case have the same surface form. As a result, stopping policies that depend on the surface answer alone will systematically misclassify the weak case as if it were the strong one.

2.1.2 Why longer windows do not solve it

Subsequent work has shown that the U curve is not an artefact of one model or one prompt format. HELMET, RULER, LooGLE and Found-in-the-Middle each extend the original finding, showing that positional effects persist across model families, vary with task structure and dependency length, and are not eliminated by enlarging the nominal context window [1, 5, 15, 16]. This conclusion is evident in RULER and NoLiMa, both of which separate *claimed* context length (the figure printed on the model card), from *effective* context length (the longest input at which the model maintains performance against a fixed baseline).

The gap between claimed and effective context length is the empirical content of the long

context problem. Table 2.1 reports claimed versus effective context length on the RULER benchmark [1], where effective length is defined as the longest input at which the model exceeds the Llama2-7B-chat-at-4K baseline (85.6%). The authors note that of the ten evaluated models with claimed lengths of 32K or more, only four maintain satisfactory performance at 32K. A model that nominally accepts a 128K input may, on a given question, attend reliably only to a fraction of it.

Model	Claimed length	Effective length	Avg. score
Gemini 1.5 Pro	1M	>128K	95.8
GPT-4-1106-preview	128K	64K	91.6
Llama 3.1 70B	128K	64K	89.6
Mistral-Large-2411	128K	64K	86.0
Command-R-plus	128K	32K	87.4
Qwen2 72B	128K	32K	85.9
LWM-1M	1M	<4K	below threshold

Table 2.1 – Claimed versus effective context length on the RULER benchmark [1].

Effective length is the longest input at which each model exceeds the Llama2-7B-chat-at-4K baseline (85.6%) on the RULER benchmark. Avg. score is the mean across all input lengths tested. Of the ten models with claimed lengths $\geq 32K$ in the original evaluation, only four maintain satisfactory performance at 32K.

NoLiMa makes the same argument with a stricter probe [17]. RULER and similar benchmarks rely on probes in which the answer span appears verbatim in the long context, allowing a model to exploit lexical overlap between the question and the surrounding text as a shortcut. NoLiMa removes that shortcut, evaluating questions in which the answer is present in the long context but only via paraphrase rather than literal lexical match. Across 13 evaluated models, 11 fall below half of their baseline at short context by 32K tokens. Even GPT-4o, the strongest exception, falls from 99.3% to 69.7%. The key consequence is that the headline drops reported by RULER would, if anything, be larger under a probe that controls for shortcuts based on lexical matching.

These findings develop a critical point that directly impacts the AgenticLU pointback step¹. Xiao et al. document that transformer language models concentrate disproportionate attention on the first few tokens of their input regardless of semantic relevance, a phenomenon they call *attention sinks* [18]. The clearest evidence is the perplexity collapse of a cache with

¹In AgenticLU’s CoC system, the pointback step is the context retrieval based on the generated clarification question. The full system is explained and visualised in Chapter 3

a sliding window the moment those initial tokens are evicted. On the first book of PG19 with Llama-2-13B, dropping the first four tokens raises perplexity from 5.40 to 5158.07, a thousandfold increase, with reinjection restoring performance close to baseline. The model cannot allocate its attention freely across a long input, because a substantial fraction is committed to positions that carry no semantic content. Peng et al. describe a complementary mechanism in positional encoding: in standard rotary embeddings, some frequency dimensions never complete a full rotation within contexts of training length, so extrapolation to longer inputs systematically distorts attention that depends on position [19].

2.1.3 Retrieved evidence is not predictive on its own

If the bottleneck were retrieval, then better retrieval would solve the problem. RAG has substantially improved closed book QA on knowledge intensive tasks and remains the dominant approach for grounding answers in external evidence, but the supporting literature has clearly shown that good retrieval does not, by itself, guarantee correct answering. Shi et al. show that LLMs can be “easily distracted” by irrelevant context. Inserting a single unrelated sentence into a math problem materially degrades accuracy even when all the needed facts remain in view [20]. Yoran et al. document the converse failure across five open domain QA benchmarks, where RAG systems lose accuracy whenever irrelevant passages are returned alongside relevant ones. A small amount of stability focused fine tuning recovers most of the loss but does not eliminate it [21]. Self-RAG goes further still, decomposing the standard RAG pipeline into reflection tokens for each segment (*Retrieve?*, *IsRelevant*, *IsSupported*, *IsUseful*) and showing that the decomposition outperforms vanilla RAG on the same tasks [22]. As such, the quality of the pinned context and answer correctness are not predictive of each other: a first answer can be wrong over good evidence, and right over thin evidence. The implication for a stop or continue decision in this setting is that it must operate over the evidence *and* the answer jointly, not over either alone.

2.2 Agentic reasoning and the observability of intermediate state

What “agentic” inference means here. In the recent literature the term *agentic* is applied to inference procedures that run more than one operation per question, where each operation produces an output that is itself input to the next. The contrast is with the standard one shot system in which the model receives a prompt and produces an answer in a single forward pass. The defining property of an agentic procedure is the existence of intermediate steps whose

outputs are explicit and can be inspected. This shift from a single hidden computation to a sequence of explicit operations influences the design of stopping policies as each operation is a place where a controller could intervene, and each output is a candidate input to that intervention.

The canonical example of the shift is ReAct, introduced by Yao et al., which prompts the model to interleave *reasoning* and *acting* steps so that an external observation can be brought into the inference loop and used to update the reasoning trace [23]. The authors describe the contribution as a way for the model to “interact with the external environments (e.g., Wikipedia) to incorporate additional information into reasoning,” with the resulting reasoning trace acting as a running plan (or *state memory*) that the model can update as new observations arrive and that can record handling of exceptions. The structural lesson is that the reasoning trace becomes an artefact in its own right. The model’s intermediate thoughts are no longer a private internal computation but a sequence of observable steps that the rest of the system can inspect, store, and act on. Figure 2.2 contrasts the trajectory of a single example question under three inference styles.

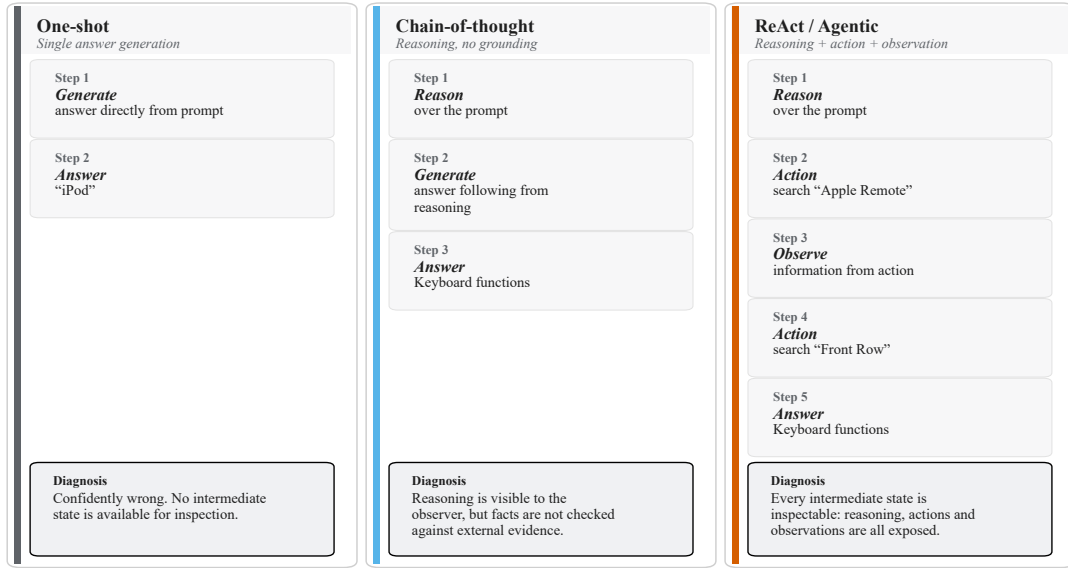


Figure 2.2 – Three inference styles on a single HotpotQA question.

Three inference styles applied to a single multi hop QA question, recreated and abbreviated from Yao et al. [23], Figure 1. The controllers developed here operate over exactly this kind of saved intermediate state at the first clarification boundary.

IRCoT, the design introduced by Trivedi et al., applies the same structural insight to multi hop retrieval-augmented QA by interleaving retrieval with Chain-of-Thought (CoT) reasoning [24]. Each CoT sentence is used to retrieve evidence for the next reasoning step, and each retrieved passage is used to ground the next sentence. The result is a tighter connection between reasoning and retrieval than either component would achieve alone, and a trajec-

tory whose intermediate states, reasoning sentences and retrieved passages, are individually inspectable. The same pattern of breaking inference into explicit, structured steps appears across recent studies of agentic benchmarks and multi document reasoning [25, 26].

This decomposition produces a key result: the reasoning becomes observable. What was once a single hidden generation step is now an ordered sequence of operations, and each operation produces an artefact. The clarification question is one. The retrieved passage is another. The clarification answer is a third. None of these is observable in a one shot setup, and each is a candidate input to a downstream decision. AgenticLU’s Chain of Clarifications (CoC) workflow, documented in Section 3.2, produces such artefacts at every round boundary.

2.2.1 Final success is the wrong lens

The published evaluations of agentic and iterative reasoning systems almost always report a single number per system, namely whether the agent answered correctly or completed the task. That measure is convenient and unambiguous, but in some cases it is the wrong lens for the present problem. Three concrete cases illustrate why.

First, we consider a Reflexion style agent that runs three trials on a task [27]. The reported empirical evidence is focused on if the third trial succeeded. From that number alone it is not recoverable whether trial one was already correct, if trial two was correct independently of trial three, or if trial three recovered from a wrong trial two. Each of these is a separate decision a controller could inspect, and each is invisible to the headline number, because no per trial accuracy is reported. The trial level dynamics of correction and degradation are hidden.

Consider next self-consistency sampling with the Adaptive-Consistency stopping rule of Aggarwal et al. [28]. The system draws samples until a Dirichlet based stability criterion is met, and final accuracy is reported jointly with the sample count saved. Thus, Adaptive-Consistency is, in effect, a stopping policy that operates over the distribution of sample votes. However, since its evaluation reports only the joint outcome, it cannot say how often the rule correctly halted on a stable majority, and how often it would have benefited from sampling further. The stopping rule cannot be evaluated separately from the sampling it controls.

Finally, consider, AgenticLU itself. The published configuration uses tree search at fixed depth three with branching factor eight, achieving 97.8% answer recall on NarrativeQA [2]. That number cannot distinguish between examples where: the answer at depth one would have sufficed, the branch at depth three was strictly necessary, deeper search eventually hurt. The case study in this thesis makes exactly those distinctions visible. The final task lens is not wrong, but does not consider the boundary decisions that occur inside the trajectory.

2.3 Clarification as a reasoning move, and what it does not answer

Clarification as a reasoning move. Clarification in a QA system refers to an explicit, internally generated step in which the model formulates a follow up sub question, gathers evidence relevant to it. The result is used to inform its answer to the original question. The clarification step is distinct from retrieval, which selects passages, and from CoT reasoning, which articulates intermediate inferences. Clarification produces a new *question* as its artefact, and that sub question can be re grounded against the same input. As a reasoning move, clarification trades a single hard problem for several easier ones, and lets the model isolate and resolve the parts of the original question that it cannot answer directly. As such, clarification is a natural fit for questions that are ambiguous or under specified, and for questions whose answers must be assembled from multiple pieces of evidence.

The foundational works in this area treat clarification as a turn taking phenomenon. QuAC reframes question answering as an inherently interactive process, in which an information seeker and an information provider work in tandem to negotiate the seeker’s information need across multiple turns of dialogue [29]. The result is a corpus in which questions are often under specified on their own and become answerable only when read in context with the preceding turns. AmbigQA extends this insight to the single turn setting, showing that open domain questions, considered in isolation, frequently admit multiple plausible interpretations [30]. The authors argue that systems should expose and resolve that ambiguity rather than silently collapsing it to a single answer, and the clarification mechanism in their proposed system makes the ambiguity explicit before answering.

More recent work develops clarification as a structured search step rather than a turn of dialogue. Tree-of-Clarifications, introduced by Kim et al., generates a tree of candidate disambiguations for an open domain question and uses retrieval to score each branch against retrievable evidence [31]. The result is a single answer that reflects the most strongly supported interpretation, with the disambiguation step preserved as part of the trajectory. AgenticLU, the base on which this thesis builds, sits in the same family but applies clarification to long context understanding rather than to ambiguity. Its CoC procedure (Figure 3.1) runs a self generated clarification, an evidence selection over the long input, and an intermediate answer, and may repeat that round before producing a final answer [2]. As such, the clarification step in AgenticLU is not a single move that resolves ambiguity once: it is a repeatable reasoning operation embedded in the inference workflow.

This literature establishes that clarification is sometimes the right move for a QA system. However, it does not provide an answer as to when clarification should *stop*. The literature

asks upstream questions almost exclusively. What clarification question should be generated? How should ambiguity be represented? How should a clarified answer be produced? The downstream question is rarely asked.

Once a clarification process has produced a provisional answer, should that answer be accepted or challenged with another round?

The clarification process inherited from AgenticLU can be run for any number of rounds, and nothing in the published procedure prescribes when a particular trajectory should halt. Several plausible defaults exist: always one round, always a fixed number of rounds, or running until a confidence threshold is reached. None is justified by the clarification literature itself: an inspection instrument is established without specifying a stopping policy for it.

2.4 Verification, refinement, and the conditional value of further reasoning

A standing assumption in clarification work is that another reasoning step, if it is well formed, will not hurt. Verification and refinement research has steadily eroded that assumption.

The first claim of this literature is that generated content can be assessed against evidence. FEVER, introduced by Thorne et al., operationalises this claim at scale: each claim in the dataset is labelled as SUPPORTS, REFUTES or NOTENOUGHINFO against a fixed Wikipedia evidence store, and systems are scored on whether they retrieve the correct evidence sentences and reach the correct verdict [32]. The dataset establishes that fact verification is well defined as a classification problem grounded in evidence, separate from the generation of the underlying claim. More recently, LLM as judge work has shown that structured evaluation prompts can produce useful, if imperfect, correctness signals for free form generations [33, 34], extending the verification logic from short factual claims to long form generated text.

Refinement followed naturally from this idea. If outputs can be assessed, they can be improved by iterating on that assessment. Self-Refine, Reflexion and CRITIC each operationalise this idea differently. Self-Refine has the model generate an output, critique that output, and revise it, all using the same underlying model [35]. Reflexion adds an episodic memory of verbal self feedback across trials, so that later trials can learn from earlier failures [27]. CRITIC introduces an external check, allowing the model to invoke tools to verify its claims before revising [36]. In each case, the refinement step is conditioned on a critique signal, and the assumption is that revision guided by critique is at least as good as not revising.

Since this, investigations have become more cautious. Huang et al. show that, without an external signal, LLMs do not reliably identify their own errors and can degrade initially correct answers when asked to revise them [37]. The literature on test time compute reaches the same conclusion from a different direction. Chen et al. characterise *overthinking*, in which extending the reasoning budget eventually abandons previously correct answers [38]. Hassid et al. show, across multiple reasoning models, that shorter thinking chains often outperform longer ones on the same question [39]. Work on long context stability reports that additional retrieval or reasoning steps can introduce drift in QA settings [21].

As such, the marginal value of an additional reasoning step is a function of the current state: when the current answer is under grounded or ambiguous, another step can repair it. But when the current answer is already adequate, another step is at best redundant and at worst introduces new sources of error. Table 2.2 maps the phenomena above onto the four ways a continuation can conclude.

Transition	Phenomenon named in prior work	Implication for stopping
Wrong $\rightarrow$ correct	Useful refinement on under grounded outputs [24, 35]	Continuation has value, and the case should be sent forward.
Correct $\rightarrow$ correct	Safe but redundant elaboration. “More reasoning” as a no op.	Continuation is pure cost, and the case should be stopped.
Wrong $\rightarrow$ wrong	Failure to self correct without an external signal [37]	Continuation may not help, but stopping on a wrong answer remains the error to avoid.
Correct $\rightarrow$ wrong	Overthinking. Abandonment of previously correct answers [38, 39]	Continuation is actively harmful, and correct answers should be preserved by stopping.

Table 2.2 – Mapping prior work phenomena onto the four continuation outcomes.

The verification, refinement and overthinking literatures collectively name all four ways a continuation can conclude. The asymmetry across these rows disqualifies any fixed always stop or always continue policy and motivates a per example controller.

2.4.1 Why model confidence is not a safe stopping signal

If continuation is conditional, the logical conclusion is that the controller needs a signal on which to condition. The most readily available candidate is the model’s own confidence in its provisional answer, expressed either as an output probability or as a verbalised estimate. However, three established lines of work rule this candidate out.

The first line demonstrates that internal probability is partially informative for the right model on the right task, and otherwise unreliable. Kadavath et al. probe Anthropic internal models with the $P(\text{True})$ prompt shown in Listing 2.1, which asks the model to estimate whether its own proposed answer is correct [4]. They report that a 52B parameter model achieves a Brier score of 0.145 on TriviaQA against a chance Brier of 0.25, with Area Under the Receiver Operating Characteristic Curve (AUROC) 0.873. The Brier score is substantively informative against the chance baseline, and the AUROC sits well above the 0.5 that random ranking would produce. The model’s internal probability does track correctness on this task. The authors also introduce a value head probe $P(\text{IK})$, a learned scalar trained to predict whether the model would answer a question correctly, which they describe as the model attempting to assess “the probability that I know the answer to this question.” The mechanism works *only with training time access* to the model, and downstream users do not have that access. The authors tested a natural language variant accessible to any user, but it did not yield a comparable signal.

```

Question: <Q>
Proposed Answer: <A>
Is the proposed answer:
  (A) True
  (B) False
The proposed answer is:

```

Prompt 2.1: The $P(\text{True})$ prompt of Kadavath et al. [4]. Confidence in the proposed answer is read off as $P((A))$.

The second line examines the verbalised alternative, in which the model is asked to state its confidence in words. Three findings together show that this alternative performs well only under narrow conditions. First, verbalised confidence *can* be informative under the right setup. On TriviaQA, SciQ and TruthfulQA, Tian et al. report an approximate halving of expected calibration error relative to internal probability, but only on RLHF fine tuned models and only when the elicitation prompt is constructed carefully [40]. Second, without those conditions, verbalised confidence overconfides badly. Xiong et al. evaluate vanilla verbalised confidence prompts across GPT-3, GPT-3.5, GPT-4 and Vicuna and find that expected calibration error exceeds 0.377 for three of the four models, with GPT-4’s AUROC

for distinguishing correct from incorrect answers at 62.7%, close to chance [3]. Third, the prompt itself, rather than the model, is the dominant source of variance. Yang et al. extend the comparison across seventeen prompt methods and four model families, and find that “the reliability of these scores strongly depends on how the model is asked” [41]. Figure 2.3 summarises the disagreement.

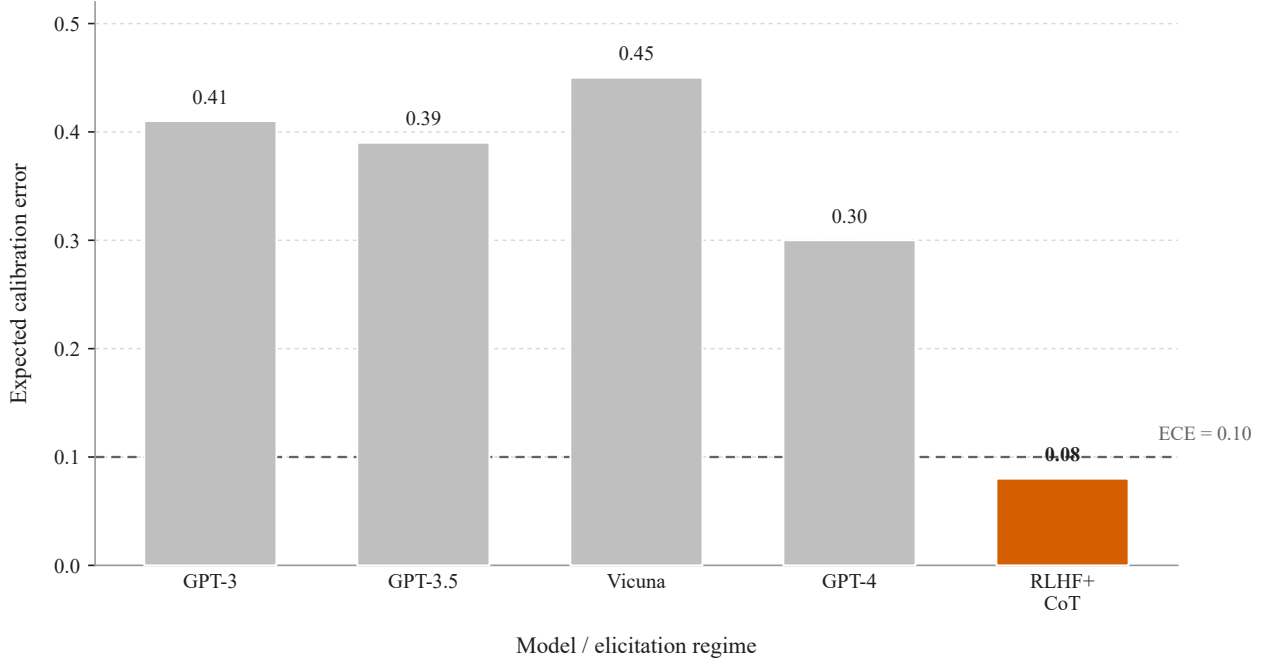


Figure 2.3 – Confidence elicitation disagrees across regimes.

Expected calibration error under representative confidence elicitation regimes, summarising the disagreement in the calibration literature. The four left bars give confidence numbers reported by Xiong et al. [3] (averages across eight tasks, rounded to two decimal places). The right bar is the order of magnitude reduction observed by Tian et al. [40] on RLHF fine tuned models with verbalised CoT elicitation. The dashed reference line is the conventional well calibrated threshold at $ECE = 0.10$. The lesson for a stopping controller is not that confidence is useless. It is that confidence is fragile. A single prompt produces an unreliable signal across model families, while careful elicitation produces a usable signal on only some of them.

A third line of work attacks the assumption itself. Yin et al. construct the SelfAware benchmark of questions designed to be unanswerable, and show that LLMs systematically attempt to answer rather than abstain [42]. Even the strongest models match only a small fraction of the human abstention rate. Feng et al. document the same gap in a knowledge gap framing and show that collaboration across multiple LLMs improves abstention accuracy by up to 19.3% over the strongest single model baseline, leaving substantial residual error [43]. This yields the following conclusion: an LLM’s surface confidence is not a safe stopping signal. It does not consistently track correctness, and on wrong answers, where the signal would matter most, it does not track the model’s own ignorance either. As such, stopping policies built

on confidence have a structural ceiling. The next subsection considers structured judgement against the evidence as the alternative.

2.4.2 Structured judgement: an instrument with its own biases

If confidence is not a safe primary signal, the available alternative is to score the provisional answer against its supporting evidence using a separate model as a structured judge. Zheng et al. formalise the LLM as judge approach and report that capable judges agree with human preferences at levels close to human inter annotator agreement [33]. The approach is now widely adopted, including by HELMET [5]. For a stopping controller it has an additional virtue. A judge can be asked to score an answer against the round’s pinned context, which is exactly the evidence available at the first clarification boundary.

The same literature, however, is clear that the judge is not a neutral instrument. Shi et al. analyse around 150,000 judging instances and show that “position bias in capable LLM judges is not due to random chance” and is “significantly impacted by the quality gap between solutions” under comparison [44]. Tam et al. report a complementary effect at the prompt level. Enforcing strict JSON output formats produces “a significant decline” in reasoning accuracy², with the degradation increasing as the schema becomes more restrictive [45]. They note an important asymmetry: format restriction degrades *generative* reasoning more than it degrades *decisional* classification, and on some classification tasks structured output actually helps.

This asymmetry matters for stopping controllers as the controller’s task is decisional rather than generative. The judge is being asked to output a short stop or continue verdict together with structured fields that justify it, not a long text dump containing the rationale. As such, a controller built on structured judgement should expect a usable signal, provided it acknowledges and mitigates the biases the judging literature documents. Biases of position and order can be reduced by swapped or symmetric evaluation and fragility across prompt methods can be reduced by combining multiple decision aspects. The residual case, in which a single judge call is not conclusive, calls for a deferral layer rather than a verdict: when the structured judgement neither cleanly accepts nor cleanly rejects, the controller should defer rather than commit. Sections 4.4.4 and 4.4.5 take up exactly this design.

²Recent reasoning systems trained for structured output (Anthropic’s models for XML, OpenAI’s for JSON) do not exhibit the Tam et al. degradation, suggesting that the effect being documented is specific to retrofitting structured output onto general purpose models rather than a property of structured output per se.

2.5 Stopping, selective prediction and inference control

Selective prediction as a framing. *Selective prediction* is the family of methods that allow a model to refuse to make a prediction on inputs where its expected risk is too high. The system answers when it is confident and abstains when it is not, accepting a reduction in coverage in exchange for an improvement in conditional accuracy on the answered subset. These systems make prediction a control problem rather than a generation problem: the question is no longer only *what answer should be produced*, but also *whether an answer should be produced at all*.

Selective prediction was first formalised for general deep neural network classifiers by Geifman and El-Yaniv, who give a procedure for selecting a confidence threshold such that the resulting selective classifier achieves a chosen risk level on the answered subset with high probability [46]. The framework was then extended to question answering under domain shift by Kamath, Jia and Liang, who show that when the very confidence estimates produced by the model are unreliable, a naive “answer everything” policy reduces overall reliability and selective answering recovers it [47]. The next extension reaches inherently ambiguous questions: Cole et al. argue that when the underlying question admits multiple legitimate answers, decision rules that account for ambiguity outperform naive confidence thresholding [48]. These three settings share a structural move from generation to control: a system that can choose not to answer is a system that can be evaluated on the *quality of its choosing*.

In a parallel stream, literature on adaptive computation makes the same control move along the cost axis. Network architectures such as DeeBERT, FastBERT, and the “right tool for the job” framework allow different inputs to receive different amounts of inference compute, exiting earlier from the network on easy inputs and running the full forward pass on harder ones [49–51]. The final mechanism is different but the framing is the same: per example decisions about how much work to do at inference time.

The selective prediction signal is graded, not binary. A core defining feature of the selective prediction framework is that the model produces a graded risk indicator and the decision rule is a threshold over it and the binary refuse or answer output is derived from this indicator [46, 47]. The verbalised confidence literature reviewed in Section 2.4.1 rests on the same premise that Expected Calibration Error (ECE) and AUROC are defined only over a graded confidence score, and a binary self report would leave nothing for either metric to measure. The cost of that collapse is visible the moment the indicator is reduced to a single bit at the model boundary. Trajectories with substantial risk and those with only weak

concern produce the same flag and can no longer be separated, even when the underlying state differs sharply on the axis the flag is supposed to summarise. The finding of Tam et al. on format restriction reinforces the point from a different direction. Format restrictions degrade generative reasoning more than decisional classification, with the strongest collapse coming from the most restrictive output formats [45], of which a binary classification is the limit case. Across these ideas, the literature treats graded indicators as the input to the decision rule rather than as a substitute for it, and ultimately reserves the binary verdict for the deterministic stage that consumes those indicators rather than for the model’s own output.

2.5.1 How prior agentic and sampling systems decide when to stop

Stopping has been studied within agentic and sampling systems, but always coupled to generation rather than as a separable controller. Table 2.3 surveys the main designs.

System	State the policy decides over	Stopping criterion
Self-consistency, Wang et al. [52]	Sample vote distribution	Fixed sample budget. Halt after N samples.
Adaptive-Consistency, Aggarwal et al. [28]	Running Dirichlet posterior over sample votes	Halt when posterior probability of majority winner exceeds a threshold.
Reflexion, Shinn et al. [27]	Verbal self feedback memory across trials	Fixed trial budget plus environment success signal.
Self-Refine, Madaan et al. [35]	Model generated feedback on previous output	Halt when the model declares no further refinement needed.
Scaling test time compute, Snell et al. [53]	Process reward or outcome reward signal over a search tree	Compute optimal allocation of samples versus depth, halt by budget.
AgenticLU CoC, Zhuang et al. [2]	Clarification trace (question, evidence, answer)	Fixed depth bound (for example, three rounds). No boundary level criterion.
This thesis (VRB / VRRS / VRRS-BR)	Saved first round textual state	Per example stop or continue decision via structured judgement, deterministic risk signals, and borderline review.

Table 2.3 – Stopping criteria in prior agentic and sampling systems.

Stopping designs across the agentic and sampling literature, classified by the state each policy inspects and the rule it applies. Every prior system either decides over scalar confidence or vote distributions, or runs to a fixed budget. None decides over an explicit textual round state with structured per example judgement, which is the design space the thesis occupies.

Two patterns are visible across the table. Firstly, prior systems decide over scalar or distributional summaries: a vote count, a Dirichlet posterior, and a process reward score. The state that is used is compact and this yields a simple policy. Secondly, the policy is almost always tuned jointly with the generator that produces the samples and there is no clean separation between which samples were produced and when the system halted. The two are entangled in the published numbers, thus, there can be no individual evaluation.

Both patterns differ from the clarification boundary setting. The state at the first clarification boundary is a textual trace consisting of a clarification question, a pinned evidence span, an intermediate clarification answer, and a provisional answer. As such, a controller in this

setting must decide over those textual fields directly, or over structured summaries derived from them, rather than over a single number. Furthermore, the resulting policy is naturally separable from the generator: the trace already exists, and the controller’s job is to inspect it. The replay design developed in this thesis, for these controllers exploits this separability.

The selective prediction and adaptive computation threads supply the conceptual framing for that design but not the inputs. Selective classifiers decide over hidden state representations or scalar confidence and early exit networks decide over per layer logits. However, neither inspects an explicit textual reasoning trace and the resulting control structure is the same. This thesis constructs a well defined decision problem over that trace, with controllers that decide on its textual fields directly rather than on a scalar confidence signal.

2.6 Evaluating a stopping controller: replay, imbalance and cost

Treating stopping as control rather than generation changes how it should be evaluated. Final answer accuracy alone conflates three things: the stopping decision, the quality of repair on subsequent rounds, and the quality of the clarification that preceded it. A controller that correctly continues on a wrong answer has succeeded even if the next clarification round fails to repair it. A controller that stops on a correct answer has succeeded even if a further round would have preserved it. Three tools have been designed to address these outcomes.

The first is offline replay. Offline policy evaluation has a long history in contextual bandit settings, where logged interactions let researchers compare policies without deploying each one online [54]. The same logic applies to a stopping evaluation. If the saved state at the decision boundary is available, each candidate policy can receive the same reconstructed state, and correctness is judged only after the decision is made. Variation across runs in the upstream workflow is then isolated from differences in stopping logic. The doubly robust estimator, developed by Dudík, Langford and Li, combines model based and importance weighted estimators to reduce variance and bias when comparing policies on a fixed log [55]. The broader counterfactual risk minimisation framework of Swaminathan and Joachims supplies the statistical justification for comparing multiple policies against a single fixed log without rerunning each one online [56]. In this framing, the replay suite used here is a logged bandit dataset of saved first round states, and the three controllers are three policies evaluated against it.

The second is evaluation that accounts for class imbalance. When most first round answers are correct, a policy that stops frequently can achieve a respectable raw success rate while missing most of the cases where continuation was warranted. Prior work addresses this skew with

metrics that weight the minority class explicitly. Class balanced accuracy averages per class recall rather than pooling outcomes [57], and the Matthews Correlation Coefficient (MCC) weights all four cells of the confusion matrix symmetrically, where raw accuracy is dominated by the majority class [58]. Precision recall analysis is preferred to ROC analysis when the minority class carries most of the cost [59].

The third is uncertainty estimation under clustered data. When the evaluation suite contains multiple related observations of the same underlying question, treating those observations as independent produces confidence intervals that are too narrow. The cluster bootstrap, in which whole clusters rather than individual observations are resampled, is the standard response, and it yields intervals that respect the within cluster dependence rather than ignoring it [60].

2.6.1 Premature stopping and unnecessary continuation are not symmetric

A standard accuracy metric treats every classification error as equally costly. For a stopping controller this is the wrong default. Elkan’s foundational analysis of cost sensitive learning shows that under asymmetric costs the Bayes optimal decision threshold moves away from 0.5 even when the underlying probability estimate is perfectly calibrated [61]. The same logic applies here. A controller that stops prematurely on a wrong answer accepts an unsafe output and prevents repair, and the error is silent because the user receives a fluent but incorrect response. A controller that continues on an already correct answer wastes one clarification round and may, in the worst case, introduce drift, and the error is loud but recoverable, because the next round’s answer can preserve the previous one if continuation is well designed. These two errors are not symmetric, and an evaluation that weights them equally will rank controllers wrongly.

The prescription that follows is an evaluation discipline rather than a new metric. A controller’s headline number should be balanced accuracy, since raw accuracy is dominated by the majority class. Balanced accuracy alone, however, hides premature stopping inside the average, because it reweights classes but not error *types*. As such, this thesis reports balanced accuracy alongside recall on wrong answers, MCC, and the full confusion matrix. These metrics expose premature stopping specifically: recall on wrong answers as a single scalar, MCC as a balanced summary across the four cells of the matrix, and the full confusion matrix as an audit trail. Section 4.5.4 formalises this metric set in the Methodology.

2.7 Where the literature stops, and what this thesis adds

Each thread provides essential information that contextualises this thesis. Long context QA research establishes that evidence use is the bottleneck and that surface fluency cannot be trusted [1, 13, 14, 17]. Agentic reasoning illuminates that intermediate states exist and can be inspected, but that final task evaluation hides the decisions made over them [23, 24, 27]. Clarification research shows that explicit follow up is sometimes the right move yet does not address when to stop [2, 30, 31]. Verification, refinement, and overthinking research together show that further reasoning has conditional value, and that it can repair or degrade the current answer depending on its state [37–39]. Calibration and abstention research rules out model confidence as a safe stopping signal [3, 4, 40–43]. The LLM as judge literature provides the alternative signal together with the biases the controller must mitigate [33, 44, 45]. Selective prediction and adaptive computation provide a control framing for per example decisions [46, 47]. Together, four tools supply the apparatus to compare controllers fairly: replay, metrics that handle class imbalance, the cluster bootstrap, and cost sensitive evaluation [54–58, 60, 61].

Despite the examined works framing the core issue, they do not provide:

- i. a definition of the first clarification boundary as an explicit, persistent decision object;
- ii. an evaluation protocol that judges stopping policies on their boundary decision rather than on the success of downstream repair attempts;
- iii. a controller design that consumes the saved round state of a clarification driven workflow and produces an interpretable decision to stop or continue; and
- iv. a layered comparison of controllers based on structured judgement, deterministic signals, and borderline review against a shared replay log, evaluated with metrics that reflect the asymmetric costs of the two error types.

This thesis studies that gap. We establish the first clarification boundary as a well defined control problem in its own right, using the clarification setting proposed by AgenticLU [2]. Instrumentation exposes the boundary as an observable object, saved trajectories from that instrumentation characterise what continuation does at the boundary, and three stopping policies are designed, built, and evaluated against a shared replay log.

A consolidated reference table mapping every cited work above to its research area and main finding is provided in the chapter Appendix A.1.

Chapter 3

Background: Long Context Setting and Evaluation Datasets

The technical work of this thesis builds on three inherited components: the AgenticLU framework for long context understanding [2], its Chain of Clarifications (CoC) workflow, and the HELMET long context evaluation setting [5]. These three are exercised on four inherited question answering datasets, Natural Questions, HotpotQA, TriviaQA and PopQA, which supply the questions, answers and long context inputs the system operates over.

3.1 The AgenticLU framework

AgenticLU is an agentic, long context understanding framework introduced by Zhuang et al. [2]. A large language model’s *nominal* context window (the maximum input length it accepts) is typically larger than its *effective* context window (the portion of that input the model can actually use to produce an answer). It has been established that enlarging the prompt does not guarantee that the model will use the right evidence within it. As such, AgenticLU gives the model an explicit inference procedure for selecting a localised evidence span before producing an answer.

The framework has two stages. The first builds offline clarification trajectories by tree search over candidate clarification questions and corresponding evidence selections. These trajectories are then distilled into the model through supervised fine tuning followed by direct preference optimisation, producing a model that runs the clarification procedure as a single, agentic workflow at inference time. The second stage is inference time deployment: the fine tuned model processes a long context question through its learned clarification workflow, which may run for one or several rounds at runtime.

Across seven long context tasks, AgenticLU maintains consistent performance as context length grows to 128K tokens, while baseline prompting strategies and specialised long context models degrade at extreme lengths [2].

3.2 The Chain of Clarifications workflow

AgenticLU’s Chain of Clarifications (CoC) is the inference procedure. A single clarification round consists of four operations applied in order, and the workflow runs that round either once or several times before producing a final answer. Figure 3.1 shows both configurations.

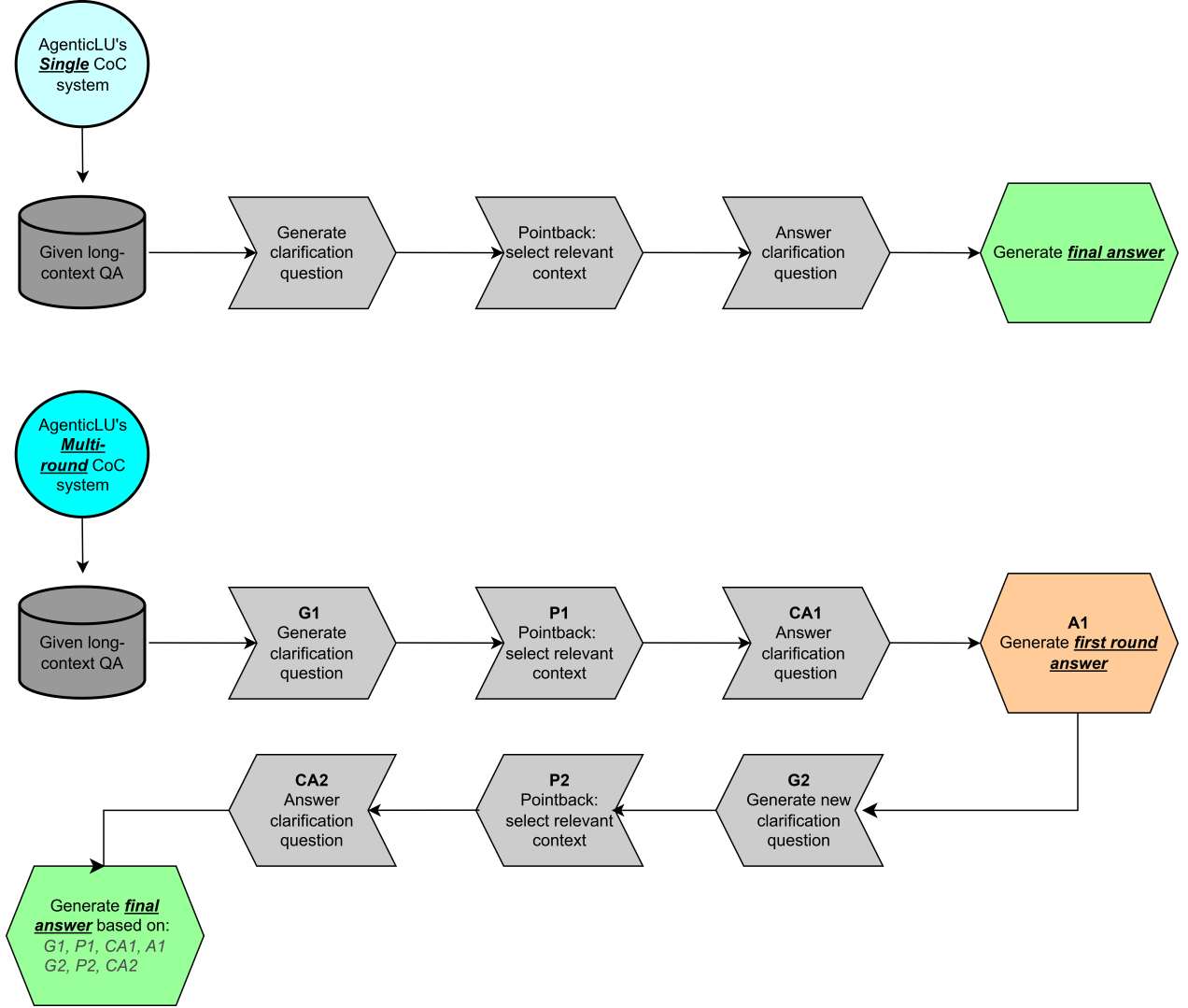


Figure 3.1 – AgenticLU’s Chain of Clarifications (CoC) workflow: single and multi round clarification.

Top: the single round configuration generates one clarification question, performs pointback to select the relevant context, answers the clarification question, and then produces a final answer to the original question. **Bottom:** the multi round configuration repeats the same four operations, with the first round answer A_1 feeding into a second round clarification question G_2 , and a final answer produced from the full trace.

3.2.1 The four operations of a single round

Table 3.1 lists the four operations of a single CoC round and the symbol assigned to each artefact in round k .

Operation	Symbol	Role in round k
Clarification question generation	G_k	Generates a clarification question for the original query, conditioned on the available question and context, and on prior round artefacts when $k > 1$.
Pointback / context selection	P_k	Selects or pins the span of context judged most relevant to answering the clarification question G_k .
Clarification answer generation	CA_k	Answers the clarification question using the pinned span P_k , producing an intermediate answer grounded in the selected context.
Original question answer generation	A_k	Produces a provisional answer to the original question using the current round trace, including G_k , P_k , and CA_k , together with any prior round artefacts.

Table 3.1 – Four operations in a single Chain of Clarifications round.

Each round produces four artefacts in order: a clarification question G_k , a pinned context span P_k , a clarification answer CA_k , and a provisional answer A_k to the original question.

The first round state instrumented in Chapter 4 (the input that all three controllers consume) consists of these four artefacts together with the original question and ground truth answer.

The pointback stage grounds the system in the given context. AgenticLU does not expect the model to attend uniformly to a 128K token input, rather the clarification question is used to generate a *span* of the context directly relevant to that question. This question is then answered and used to generate the final provisional answer. As such, the quality of CA_k , and through it A_k , is coupled to the quality of P_k : a pinned span that contains the relevant evidence yields a well grounded round answer; a span that is too broad, off topic or missing the relevant fact yields an answer that inherits that weakness.

3.2.2 Single round and multi round configurations

In the single round configuration the workflow runs the four operations once and returns A_1 as the final answer. In the multi round configuration it applies the same four operations a second time before producing a final answer. The second round clarification question G_2 is generated from the first round trace, including A_1 ; pointback yields P_2 , the clarification answer yields CA_2 , and the final answer returned to the caller is conditioned on the full trace $(G_1, P_1, CA_1, A_1, G_2, P_2, CA_2)$. The number of rounds is a specified runtime parameter in the AgenticLU configuration.

The workflow can be halted at the end of any round. At the end of round k the model has already produced an answer A_k to the original question. Continuing past round k is a runtime configured choice, not a requirement.

3.3 The HELMET long context evaluation setting

Introduced by Yen et al. [5], HELMET is the long context evaluation harness in which AgenticLU is evaluated. As developed in their work, previously proposed long context benchmarks gave an incomplete signal of long context ability, as they relied on synthetic tasks such as Needle-in-a-Haystack (NIAH) ¹, used inconsistent context lengths, and applied surface metrics that did not always agree with human judgement. HELMET addresses these problems through overarching task application categories, controllable context lengths up to 128K tokens, and higher value evaluation performed by LLMs in place of rudimentary string match metrics.

These tasks are organised into seven categories: synthetic recall, long context QA, RAG, generation with citations, passage re ranking, many shot In Context Learning (ICL)², and summarisation. The RAG category supplies the setting used in this thesis. Inside each QA example a question is paired with a long retrieved context window. The task is clear; the system must locate the relevant passages containing information related to the answer. AgenticLU is evaluated on four QA datasets from this category: Natural Questions, HotpotQA, TriviaQA, and PopQA [6, 7, 62]. The first three are part of the KILT benchmark family focused on knowledge-intensive³ language tasks; PopQA targets longer tail factual knowledge⁴ [9, 62]. Together the four span single hop factoid, multi hop and long tail factual question styles, and under HELMET they share a common form: each example is a (question, long context, gold answer set) triple delivered through the same prompting interface.

¹NIAH refers to a synthetic, long context evaluation setup in which a target item, or “needle”, is placed inside a much longer distractor context, or “haystack”, and the model is evaluated on whether it can retrieve that item. HELMET argues that such tests provide an incomplete signal for long context ability because they primarily measure isolated retrieval from long inputs, while downstream long context tasks may require reasoning over, comparing, or synthesising information across the full context.

²*many-shot ICL* in HELMET is an evaluation of whether a model operating on long context can infer a specific task from many labelled examples placed in context, rather than fine tuning. HELMET varies the number of examples to control context length and uses numbered labels to test task learning rather than reliance on pre-trained label meanings [5].

³In KILT, knowledge-intensive language tasks are tasks that require access to external factual resources, such as open-domain question answering, fact checking, slot filling and entity linking.

⁴in PopQA, Long-tail factual knowledge refers to facts about less popular entities, where language models tend to perform worse than on high-popularity facts.

3.3.1 Context variants and answer judging

There are two essential properties constructed by HELMET that define what the AgenticLU system sees at runtime.

First, the same underlying question can be presented at multiple context lengths, with retrieved passages padded or trimmed to a target token budget. Each such presentation is a distinct example with the same answer set; the thesis refers to it as a **context variant** of the underlying question. The motivation for these variants is based on studies that illustrate that performance of long context QA systems depends not only on context length but also the position of the passage containing the answer [14]. As such, a single base question paired with different context windows can produce different intermediate workflow states.

Second, candidate answers are scored by a separate LLM judge rather than by string matching. Thus, the judge is more robust to paraphrase and to answer form variation. However, this judging method introduces a dependency on the model configuration [5].

3.4 The evaluation datasets

The four QA datasets used in AgenticLU: Natural Questions, HotpotQA, TriviaQA and PopQA, are the base data over which this thesis is run. They are *inherited* rather than constructed: each is supplied through the AgenticLU configuration in its HELMET RAG form, and no part of the thesis modifies the questions, answer sets or evidence corpora. Together, these sets stress a different aspect of factual question answering which makes them a solid base for studying QA behaviour.

3.4.1 The four source datasets

Natural Questions [6] is built from real, anonymised queries issued to the Google search engine, each paired with a Wikipedia page drawn from the top search results. Annotators mark a long answer (typically a paragraph) and a short answer (one or more entity spans), or null where the page supports neither. Its questions are therefore *naturally* information seeking: they originate from genuine user information needs rather than being authored against a passage, and they carry the ambiguity and under specification that real queries exhibit. The public release contains 307,373 training, 7,830 development and 7,842 test examples [6]. Within this thesis it supplies single hop, information seeking factoid questions.

HotpotQA [7] was constructed to require *multi hop* reasoning. Crowdworkers were shown

pairs of linked Wikipedia introductory paragraphs and asked to author questions that can only be answered by combining facts from both, and to mark the sentence level supporting facts used. The dataset contains 112,779 question answer pairs and distinguishes two question types, *bridge* questions, which chain through an intermediate entity, and *comparison* questions, which contrast properties of two entities [7]. It is the only one of the four whose questions are designed so that no single passage is sufficient.

TriviaQA [63] differs from the others in the order of its construction. Questions are pre authored by trivia enthusiasts and harvested from quiz sources; evidence documents from Wikipedia and the web are gathered *afterwards*, providing distant supervision rather than passage annotation. The dataset comprises over 650K (question answer evidence) triples derived from roughly 95K question answer pairs, with on average six evidence documents per question [63]. Because the questions are written independently of any passage, they exhibit considerable lexical and syntactic distance from their supporting evidence, which is part of what makes them a distinct test of grounding.

PopQA [62] is an entity centric dataset of roughly 14K questions, each generated by converting a Wikidata (subject relation object) triple into a natural language question using a per relation template, across sixteen relation types. Its defining design choice is the deliberate sampling of the *long tail* of entity popularity, measured by Wikipedia monthly page views, so that many questions concern entities a language model is unlikely to have memorised [62]. Thus, it probes the boundary between what a model knows parametrically and what it must recover from the provided context.

Table 3.2 summarises the four along the dimensions relevant to this thesis.

Dataset	Question source	Reasoning type	Original size	Evidence corpus
Natural Questions	Real Google queries	Single hop information seeking	307,373 / 7,830 / 7,842	Wikipedia
HotpotQA	Crowd authored from paragraph pairs	Multi hop (bridge / comparison)	112,779 total	Wikipedia
TriviaQA	Pre authored trivia	Compositional, distantly supervised	650K+ triples (95K Q-A)	Wikipedia + web
PopQA	Templated from Wikidata	Long tail entity recall	≈14K total	Wikidata / Wikipedia

Table 3.2 – The four inherited QA datasets used for evaluation.

Sizes for Natural Questions are train / development / test. The first three datasets belong to the KILT knowledge intensive family [9]; PopQA is constructed in the same spirit from Wikidata. Reasoning types span the single hop, multi hop and long tail styles referenced in Section 3.3.

3.4.2 From short context collections to long context RAG inputs

None of the four datasets is natively a long context task. Each began as a short context collection and is extended into the RAG regime by HELMET, which constructs the long inputs. For each question, HELMET begins from a gold passage that contains the answer, then retrieves further passages from a Wikipedia corpus to act as distractors, and packs the gold passage together with these distractors into a single context window. Input length is controlled by the number of retrieved passages, which is how the same underlying question is presented at the several context lengths that produce the context variants of Section 3.3.1. The position of the gold passage within the window is varied rather than fixed, reflecting the finding that long context performance depends on where the answer sits, not only on how long the input is [14]. The consequence for this thesis is that the datasets contribute the questions, answer sets and gold evidence, while the surrounding long context, the retriever, the distractor corpus and the placement protocol are inherited properties of the HELMET

setting rather than of the datasets themselves.

3.4.3 Why these four

The value of the four lies in their differences. TriviaQA’s pre authored questions lean on knowledge a strong model has often memorised, and tend to be answered correctly at the first clarification round; HotpotQA’s bridge and comparison questions enforce reasoning across two passages; Natural Questions carries the ambiguity of real user queries; and PopQA is constructed precisely to fall outside a model’s parametric memory. A stopping controller is only interesting where first round answers are sometimes right and sometimes wrong, and the four datasets supply markedly different base rates of first round correctness for exactly that reason. The effect is visible in the replay suite used in the final evaluations: the proportion of first round answers that are wrong ranges from roughly one in ten on TriviaQA to roughly one in three on Natural Questions, with HotpotQA and PopQA in between, so the same controller faces a very different stopping problem on each. This spread, rather than any single dataset, is what lets the later analysis distinguish a controller that stops well from one that merely inherits an easy or a hard distribution, and it is why results are reported per dataset rather than pooled.

3.4.4 Provenance and limitations

The evidence corpora are public and openly licensed. Natural Questions, HotpotQA and TriviaQA are anchored to Wikipedia text under its Creative Commons licensing, PopQA derives from Wikidata together with Wikipedia page view statistics, and TriviaQA’s release explicitly notes that the originating institution does not claim ownership of the underlying trivia questions or gathered evidence documents. As the datasets are inherited and used unmodified, no further data collection or annotation was performed for this thesis.

Three properties of the datasets bear on the interpretation of later results. First, all four are Wikipedia centric and fixed to mid 2010s snapshots, so entities that became prominent later are systematically under represented. This matters most for PopQA, whose long tail entities are the most sensitive to which Wikipedia snapshot is used. Second, each dataset carries construction artefacts. HotpotQA contains documented *reasoning shortcuts*, where a question can sometimes be answered by matching a single sentence rather than genuinely combining two passages [64]. PopQA’s templated entity form can be partially solved from surface features of an entity’s name rather than from retrieved evidence [65]. TriviaQA’s distant supervision means that not every gathered document actually supports its answer. Third, Natural Questions and TriviaQA both admit several valid surface forms for the same

answer, which is one reason the inherited setting scores answers with an LLM judge rather than by string matching. None of these properties undermines the datasets for the present study, but each is a reason to read per dataset behaviour in light of how the data was built, rather than treating the four as interchangeable.

AgenticLU runs an explicit clarification procedure at inference time to close the gap between nominal and effective context use. Its CoC workflow executes four operations per round (clarification question, pointback, clarification answer and provisional answer) and runs for one or more rounds at runtime. HELMET supplies the long context execution harness, the controllable length protocol, the context variants produced by that protocol, and the four QA datasets through which AgenticLU is evaluated. These datasets, Natural Questions, HotpotQA, TriviaQA and PopQA, span single hop, multi hop and long tail factual question styles, and their differing construction is evident in the later results analysis.

Chapter 4

Methodology

A clarification driven QA system that has completed a first clarification round and produced a provisional answer faces a control decision: accept that answer, or continue clarifying. This thesis establishes that decision as a well defined boundary controller problem, fixed at one specific point in the CoC workflow: the moment immediately after the first clarification round, when the system has returned its first provisional answer.

At the first clarification boundary, should the system accept the provisional answer, or continue to another clarification round?

STARC, the architecture designed to address this problem, proposes that this question is best answered through a consideration of the system state at that boundary.

The methodology rests on one decision: separate the runtime that produces clarification behaviour from the analysis that evaluates it. The underlying agentic CoC system is left unmodified, and its runtime is instrumented so that the state produced at each clarification boundary, the clarification question, pinned context, clarification answer and provisional answer, is captured as a recoverable, structured object. That agentic state then becomes the object of study. Both the case study and the stopping policies operate offline over saved states rather than over live runs of the system, so the same boundary is examined repeatedly under identical conditions. This decoupling is what makes the central comparison valid: as every policy decides over the same saved states, any difference in their behaviour is attributable to the stopping logic itself and not to run variation in the underlying system.

The chapter proceeds through three stages, each delivering one of the thesis's contributions. Stage 1 (Section [4.2](#)) makes the boundary observable: it instruments the CoC runtime to preserve agentic states at explicit boundaries, producing the saved state framework all later analysis depends on. Stage 2 (Section [4.3](#)) establishes that continuation is only conditionally

beneficial: through a trajectory case study it characterises the four mechanisms by which a second round affects a first round answer, repair, preservation, persistent failure, and degradation, and shows that which one occurs is already fixed by the first round state. Stage 3 (Section 4.4) designs, implements and evaluates STARC as three progressively layered policies:

- i. Verification and Refinement Baseline (VRB);
- ii. Verification and Refinement with Risk Signals (VRRS); and
- iii. Verification and Refinement with Risk Signals and Borderline Review (VRRS-BR).

STARC’s three policies are evaluated on a fixed replay suite of 408 saved first round states, drawn from four RAG QA datasets, Natural Questions, PopQA, TriviaQA, and HotpotQA, chosen because they present the controller with markedly different rates of first round error. The evaluation protocol (Section 4.5) holds the saved states constant across all three policies and separates the information available to a policy at decision time from the correctness labels used to score it, so that no policy can see the answer it is being asked to judge. The metrics are chosen to remain interpretable under the class imbalance of the suite, with the Matthews Correlation Coefficient (MCC) reported as the principal summary. The result of this design is a like-for-like comparison of three stopping strategies over identical evidence, reported in Chapter 5.

The instrumented runtime, the construction of the replay suite, and the implementations of the three stopping policies VRB, VRRS and VRRS-BR are released in the public repository accompanying this thesis (Section 1.3), together with the prompts used by their LLM stages.

4.1 Data and saved objects

The methodology uses the four QA benchmarks established in Section 3.4, Natural Questions, HotpotQA, TriviaQA and PopQA, run through the inherited HELMET long context setting [2, 5]. The benchmarks supply the original questions, accepted answer sets and long context inputs. The objects under analysis, however, are not the benchmarks themselves but the saved CoC round states produced when those benchmarks are run through the instrumented clarification runtime, together with two further objects projected from those states:

- i. a case study continuation set; and
- ii. an offline first round replay suite.

4.1.1 Base questions, context variants and replay examples

Three terms are used throughout the methodology:

- a **base question** is the underlying semantic question;
- a **context variant** is a particular long context instance associated with that base question; and
- a **replay example** is the saved first round clarification state for a specific (base question, context variant) pair.

This distinction is important as the same underlying question, paired with different context variants, can produce different saved round states; the clarification question, the pinned context and the provisional answer can vary across variants [14]. As such, repeated base questions are treated as context varied observations of the same semantic question rather than as independent ones, and the later stability analysis takes the base question cluster as its unit of analysis.

4.1.2 Saved round states and the first clarification boundary

Evaluation focuses on the saved state produced at the end of the first clarification round, the point where the system first faces a stop or continue decision over a provisional answer. This is the **first clarification boundary** used throughout the thesis.

A distinction is maintained between two related state objects. The *saved* state contains everything needed for traceability and evaluation, including the benchmark answer set. The *controller visible* replay state, projected from the saved state for offline policy evaluation, is restricted to the information available at the decision boundary, namely the five fields listed in Table 4.4. Gold answers, correctness labels and later round outcomes are attached to the replay row only after policy decisions have been produced, and serve solely as inputs to metric computation.

4.2 Stage 1: round state instrumentation

4.2.1 Purpose

Studying what happens at the first clarification boundary requires that the boundary itself be observable.

The inherited CoC runtime returns only a final answer, with intermediate round states produced and consumed internally. The released system does not expose the items produced during each round: the clarification question, pinned context, clarification answer or provisional answer. As such, Stage 1 instruments the runtime so that the state produced at each clarification round is captured as a structured, recoverable object. These saved states are the empirical foundation for Stage 2 (the continuation case study) and Stage 3 (the offline stopping gate evaluation). Without them, neither analysis could be performed without re-running the underlying clarification system, which would itself introduce run variation in the saved states.

4.2.2 Inherited runtime and added instrumentation

The CoC system and the HELMET long context setting it builds over were both introduced in Chapter 3. Their core implementations are not modified.

Stage 1 builds two components that operate at runtime alongside the inherited system. The first is a **runtime extraction layer** that intercepts the system output at each step, as illustrated in Figure 4.1. Each extracted output is written to a structured state record, which enables full round recovery. The second component is a **persistent progress mechanism** that writes the completed round state record to *durable storage* as it is finalised, allowing extended clarification runs to be resumed across interrupted compute sessions without losing accumulated states.

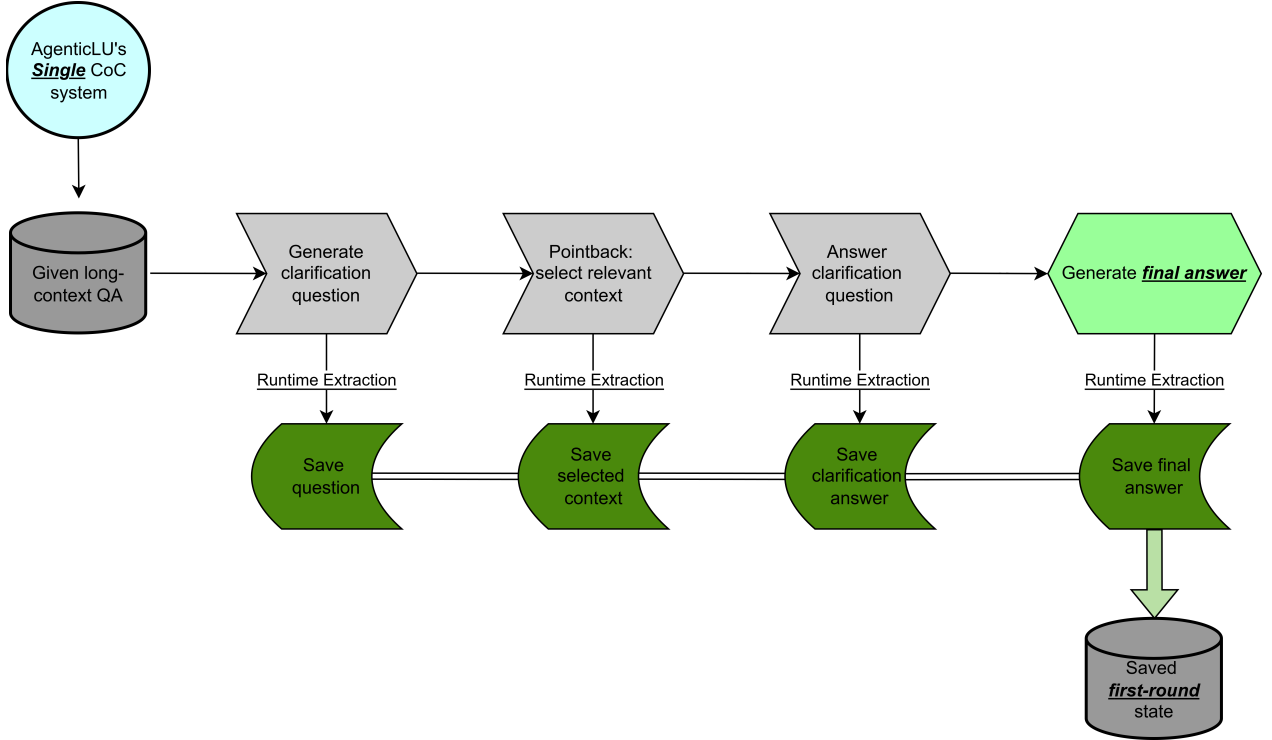


Figure 4.1 – The CoC system, with instrumented state saving.

The four CoC operations (clarification question, pointback, clarification answer, provisional answer) run as in the unmodified system. The instrumentation layer reads the artefacts each operation produces and writes them to a persistent record at the round boundary, without interfering with the operations themselves.

The objects produced at each stage are extracted and written into their saved records. The instrumentation does not interfere with the clarification mechanism. It reads outputs the inherited stages already produced rather than altering how those stages run.

Runtime extraction layer. The extraction layer reads the clarification question, pinned context, intermediate clarification answer and provisional answer at the round boundary as they are produced internally, and writes them into the saved state record alongside example and dataset metadata. Without this layer the first clarification boundary would not be observable as a recoverable object.

Persistent progress writing. The clarification runs use the maximum AgenticLU long context configuration of 128k tokens, executed on Google Colaboratory (Colab) with an A100 GPU. At this scale, individual runs span many hours, and Colab sessions are subject to interruption. The base AgenticLU system writes results only after every question in the testing set has been answered, so without explicit progress writing an interrupted run would lose every state generated since the previous checkpoint. The persistent progress mechanism writes each completed round state record to durable storage as soon as it is produced, so a resumed run picks up where it left off rather than restarting. The full runtime configuration and

resumability mechanism are specified in Appendix [B.1.3](#).

4.3 Stage 2: continuation case study

4.3.1 Purpose and unit of analysis

In the AgenticLU system, additional clarification rounds are designed to resolve ambiguity, recover missing information, and strengthen answer grounding. The reported gain from this multi round design is small but positive against a single round baseline [2], and further clarification is treated as a generally beneficial operation on that basis. However, the behaviour of individual continuation rounds falls outside the scope of that work, and the authors identify finer grained analysis of the clarification mechanism as future work. Prior work on multi step and long context reasoning has shown that further reasoning or retrieval steps can introduce drift, entrench earlier failures, or degrade answers that were already adequate [21].

The case study also plays a structural role in the methodology of this thesis. It establishes the motivation for adding stopping policies to CoC and provides the basis on which those policies are designed. Identifying which first round states benefit from continuation, which preserve correctness redundantly, which remain on a wrong evidence path, and which degrade under further clarification lets the design of the Stage 3 controllers be motivated by concrete continuation patterns rather than against an abstract assumption of uniform benefit.

The unit of analysis is the saved continuation trajectory: a single CoC run with both a first round and a second round answer state, together with the clarification trace needed to interpret the transition between them. The case study contains 26 such trajectories, sampled uniformly at random, without replacement, from the Natural Questions population of saved continuation trajectories that meet the inclusion criterion of containing recoverable first round and second round answer states. Sampling was at the trajectory level rather than stratified by base question family, so family sizes reflect the underlying distribution of recoverable trajectories rather than a sampling design. Manual adjudication at the level of detail required (per-round commitment, ambiguity space, supporting evidence) is labour-intensive; 26 was the maximum tractable within the project timeline. The full case study runtime configuration, inclusion criterion, and saved states are given in Appendix [B.3.1](#).

4.3.2 Transition categories

Each trajectory is classified by the correctness transition between its first and second round answer states. Table [4.1](#) lists the four categories and their relevance to the stopping problem.

Transition category	Interpretation	Relevance to stopping
Preservation (Correct→correct)	Second round preserved an already correct answer	Continuation was safe but potentially unnecessary
Repair (Wrong→correct)	Second round repaired an incorrect or unresolved answer	Continuation was beneficial
Persistent Failure (Wrong→wrong)	Second round failed to repair an incorrect answer	Continuation added cost without correcting the failure
Degradation (Correct→wrong)	Second round degraded an answer that was already correct	Continuation was harmful

Table 4.1 – Case study transition categories.

The categories distinguish beneficial, safe but redundant, ineffective and harmful continuation. The classification supports the question of the case study: does further clarification behave as a useful extension of the workflow, or as an intervention whose value depends on the current answer state. The four categories were specified as part of the methodology before the case study trajectories were examined, so the manual adjudication step assigns each trajectory to one of these predefined categories rather than producing categories from the trajectory sample.

4.3.3 Manual judging

Round level correctness in the case study is assigned by manual adjudication against the saved benchmark answer set. Manual adjudication is appropriate here because the analysis is interpretive: the goal is to characterise how each answer changes between rounds, not to produce a scalable label set. The full adjudication protocol and worked examples are in Appendix B.2.1, and the rules applied at each round are summarised in Table 4.2.

Judgement	Rule
Correct	The answer commits to an accepted gold answer
Correct	The answer gives an acceptable paraphrase of a gold answer
Wrong	The answer leaves multiple live candidates unresolved
Wrong	The answer refuses to commit to an answer
Wrong	The answer selects an answer not in the gold set
Wrong	The answer introduces a contradictory or unsupported claim
Wrong	The answer contains a gold string but does not commit to a single answer

Table 4.2 – Manual case study judging rules.

4.3.4 Family and example level analysis

The transition categories are supplemented by two levels of analysis.

Family analysis groups trajectories by their underlying base question, so that repeated context variants of the same semantic question are interpreted together rather than as independent observations. This grouping tests whether transition patterns concentrate by question family rather than occurring uniformly.

Example analysis takes the complementary, single trajectory view. A transition category records only that an answer’s correctness changed between rounds; it does not record how. Reading individual trajectories recovers the mechanism behind the label, including what the first round committed to, what the clarification step introduced, and which of the two the second round answer followed.

4.3.5 Contribution to the controller design

The case study supports the controller work that follows by establishing that continuation beyond the first clarification boundary produces qualitatively different outcomes (repair, preservation, persistent failure, and degradation) rather than uniform improvement. Each outcome is produced by an identifiable kind of continuation behaviour, and their detailed treatment, including representative trajectories and family level patterns, is presented in Chapter 5. The four outcome categories were specified as part of the methodology before

the case study trajectories were examined, so the trajectory analysis confirms the forecast typology rather than producing it inductively.

These mechanisms are taken as established for the following controller design in Stage 3, where they motivate the stopping policies and decision logic.

4.4 Stage 3: State Aware Risk Controller (STARC)

Stages 1 and 2 set up the stopping problem in concrete terms. Stage 1 makes the first clarification boundary recoverable as a saved object. Stage 2 shows that continuation past that boundary is not uniformly beneficial: depending on the trajectory, a second round can repair, preserve, fail to repair, or degrade the first round answer. Stage 3 builds the controllers that act on that boundary.

Stage 3 controller task

At the first clarification boundary, can the saved round state be used to decide whether to stop on the provisional answer or continue to another clarification round?

This thesis proposes State Aware Risk Controller (STARC), the architecture that answers this question. STARC treats the stopping decision as a risk assessment over the saved agentic state: a LLM evaluates that state, and a deterministic decision rule converts the resulting graded judgements into a stop or continue decision. STARC is implemented as three progressively layered policies. Each successive policy reuses the previous policy’s components and adds one new mechanism.

The policies share an approach that separates them from a different task. The literature on LLM judges shows they are unreliable when asked to assess answer correctness from the answer alone [33, 45]. The policies designed in this thesis operate differently: they evaluate the provisional answer and the agentic system state, and focus on what the risks and benefits of continuing are rather than whether the answer is correct. The structured LLM components emit signals on specific decision dimensions, and the deterministic layers convert those signals into a final *stop* or *continue* decision.

Verification and Refinement Baseline (VRB) is the structured LLM baseline. A verifier and a falsifier each assess the saved first round state and return indicators on specific dimensions: answer support against the pinned context, completeness for the question, expected gain from further clarification, and concrete failure risks such as a competing answer or evidence locked onto the wrong entity. A deterministic decision rule then consumes both records and emits a binary stop or continue decision.

Verification and Refinement with Risk Signals (VRRS) extends VRB by adding a deterministic risk signal layer that operates over the saved state and the parsed VRB fields. Hard risk signals capture concrete failure patterns, such as a multi candidate answer on a single factoid question or a form mismatch between the question and the answer, and force continuation on a single occurrence. Soft risk signals capture weaker concerns, such as an answer that appears only in the intermediate clarification answer or a pinned context that is unusually broad, and contribute towards a borderline decision. The policy emits three outputs: stop, continue, and borderline. Borderline cases are resolved as stops within VRRS itself, and used as the trigger for semantic review by VRRS-BR.

Verification and Refinement with Risk Signals and Borderline Review (VRRS-BR) extends VRRS by adding a constrained semantic review stage that is invoked only on borderline cases. Deterministic stop and continue decisions from VRRS are preserved and borderline cases are routed to a structured LLM reviewer that judges whether the answer contains a concrete, identifiable error, such as a contradiction, a wrong entity, or a competing answer that fits the question better. The reviewer examines the saved first round state together with the VRB record and the VRRS risk flags, applying semantic judgement to cases the deterministic signals could not resolve.

4.4.1 Replay suite and decision context

Stage 3 evaluates the stopping policies offline against a fixed replay suite built from the saved CoC round states. The suite contains 408 first round states drawn from four source datasets, organised into 85 base question clusters. Table 4.3 shows the per dataset breakdown.

Dataset	Base questions	Context variants	Replay examples
Natural Questions	17	6	102
PopQA	17	6	102
TriviaQA	17	6	102
HotpotQA	34	3	102
Combined	85	Mixed	408

Table 4.3 – Construction of the offline first round replay suite.

The unit of analysis is the first round boundary, represented by extracting the state from each record and projecting it into a *controller visible* row that exposes the information available at

that boundary. Correctness labels are attached to the row only after the policy has produced its decision. This conversion of a saved round state record into a *controller visible* replay row is the *replay projection*.

The five *controller visible* fields available to every policy are listed in Table 4.4.

Decision context field	Role in the replay decision
Original question	The benchmark question that the provisional answer must ultimately answer.
Clarification question	The clarification question generated by the clarification system.
Pinned context	The selected context available to the model at the first clarification boundary.
Intermediate clarification answer	The answer produced in response to the clarification question.
Provisional answer	The current answer to the original benchmark question at the first clarification boundary.

Table 4.4 – Shared first round decision context fields used by all three policies.

All three policies are evaluated on the same replay rows, and each returns a binary stop or continue decision. Holding the replay state fixed across policies isolates the effect of the stopping logic itself: any difference in performance reflects controller design rather than upstream variation in the clarification trace [54].

4.4.2 Threshold bands

The three stopping policies operate by checking graded risk indicators against thresholds. The structured LLM components of every policy emit graded scores in the range $[0, 1]$. These scores are projected into three named bands, **low**, **moderate**, and **high**, before each policy’s decision rule fires.

The bands are design parameters, not learned policy weights. Each band threshold is set by the logic of the field it scores. Every structured field poses a specific question to the model, and the model returns a graded output in $[0, 1]$, so the band cut points are placed to reflect what a **low**, **moderate**, or **high** output means for that question: no substantive evidence, present but not decisive, or strong evidence, respectively. The decision rules then operate over these bands rather than the raw scores, so that a rule such as a continuation trigger fires

on the band that corresponds to the output it is looking for.

The band ranges were confirmed during prompt development against the Stage 2 case study trajectories, whose answer behaviour was already characterised: the score distributions on those trajectories indicated where each band range should sit, so that a rule fired on core mechanism cases and not on background noise. The case study NQ trajectories and the replay examples were drawn from disjoint base question populations, so band placements derived from the case study were not evaluated against the same questions in the replay suite. The 408 row offline replay suite played no part in this calibration. It serves only as the downstream evaluation set, after the policy design is fixed.

Table 4.5 states what each band means.

Band	Range	Magnitude meaning
Low	$[0, 0.4)$	No substantive evidence to support the statement.
Moderate	$[0.4, 0.6)$	Evidence is present to support the statement, but it is not decisive.
High	$[0.6, 1]$	Strong evidence to support the statement.

Table 4.5 – Band definitions and their role in the stopping decision rule.

The same band name carries the same magnitude across all statements, but the operational consequence depends on whether the band sits on a risk statement or a confidence statement.

The choice to emit graded scores and discretise them into three bands, rather than asking the model for a binary stop or continue flag on each field, is deliberate. A binary classification at the model boundary collapses every degree of evidence on a given field to a single bit, so an answer with substantial risk and an answer with only weak risk produce the same flag and can no longer be separated. The three band discretisation preserves the magnitude information that distinguishes those two cases, while still giving the deterministic decision rule a small, fixed set of categories to reason over. The binary stop or continue output of the policy is then derived from the graded signal through the decision rule, rather than substituted for it at the model boundary. This structure follows the standard pattern of selective prediction, in which the model produces a graded risk indicator and the decision rule applies logic over it [46, 47].

The stability of the policy results to perturbations within the band placements is examined in Section 5.6.4.

4.4.3 Verification and refinement baseline (VRB)

VRB uses two complementary LLM assessments of the saved first round state: a **verifier** and a **falsifier**. Each evaluates the provisional answer against the shared decision context and returns a structured record of indicators across four dimensions: answer support, completeness, expected gain from continuing, and concrete failure risks.

4.4.3.1 System overview

The VRB system, illustrated in Figure 4.2, has two stages that operate sequentially over the saved first round state. The first is a structured LLM assessment, performed by the verifier and falsifier introduced above, that produces structured judgements about the provisional answer using the shared decision context. The second is a deterministic decision rule that consumes the parsed assessment and emits the policy’s stop or continue decision together with an inspectable justification.

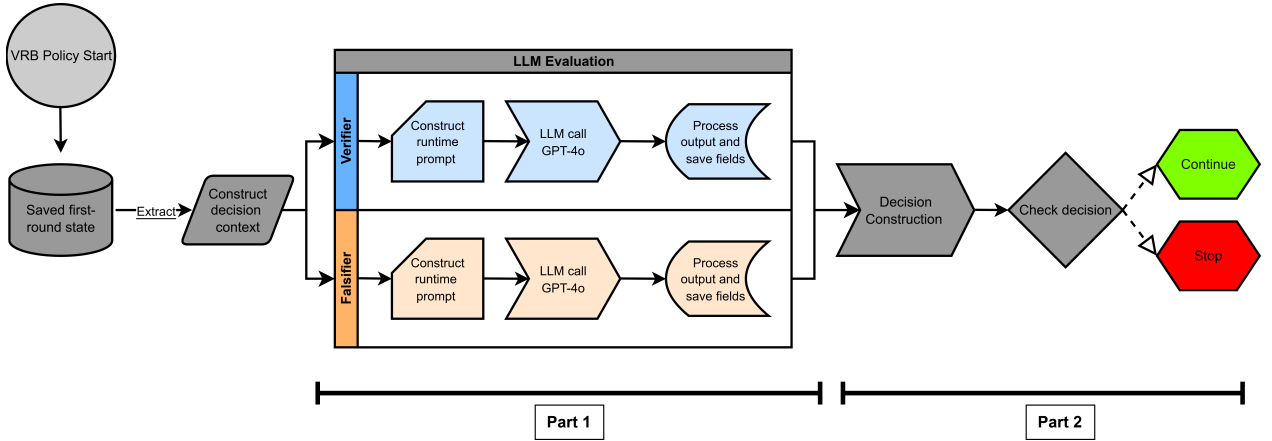


Figure 4.2 – VRB system architecture.

VRB runs two model calls (verifier and falsifier) over the saved first round state, producing structured assessments. The final stop or continue decision is made deterministically over the parsed assessments, not by a further model call. The LLM is the evaluator; the decision is the deterministic stage’s job.

The two stages together implement the policy’s central design choice. A structured LLM assessment evaluates the answer, but the final stop or continue decision is made deterministically over the parsed record rather than by another model call. This structure produces a multi component assessment whose outputs can be inspected individually, rather than a single binary verdict produced opaquely by the model.

4.4.3.2 Verifier and falsifier

Splitting the verification and falsification into two separate LLM calls is deliberate. Asking a single judge to assess both supporting and adversarial evidence within one prompt results in self consistency: the model resolves the tension between the two stances internally, and the output collapses to whichever frame the model finds more natural. Splitting the two perspectives forces each call to produce its strongest case for its assigned stance, and the deterministic stage reasons over the disagreement rather than a single resolved verdict.

A direct comparison of the dual perspective split against a single combined judge call filling the same field set is outside the scope of the present evaluation. The verifier and falsifier prompts that implement it were iterated against the Stage 2 case study trajectories until they emitted the required structured fields reliably. A study that instantiates a combined judge configuration over the same replay suite would isolate the split directly, and is identified as future work in Chapter 9.

The verifier evaluates the case for stopping: it judges the support of the provisional answer against the pinned context, the completeness of the answer for the original question, the expected gain from another clarification round, the regression risk that continuation would degrade an adequate answer, and the presence of a missing information type. The falsifier evaluates the case for continuing: it treats the provisional answer adversarially, judging the risk that the answer is false despite appearing coherent, and the risk that a plausible competing answer fits the question better.

Both assessments operate over the shared decision context, so the two records are produced from identical evidence. They differ in three respects: how each frames its task, the explicit judgement criteria each is given, and the structured output each is required to return. The raw prompt templates, exact input substitution, and full output schemas are given in Appendix B.4.

4.4.3.3 Verifier

The verifier prompt, outlined in Figure 4.3, frames the call as a stopping gate task. An explicit constraint forbids the model from inventing ambiguity around unspecified time periods, positions, or roles, which is otherwise a common source of unjustified refinement recommendations. The prompt closes with a strict JSON contract that determines the required output. The verifier also returns a text reason, which provides an inspectable record of the model’s reasoning for any decision that needs to be reviewed.

Table 4.6 details what each field entails, the motivating question and the output type.

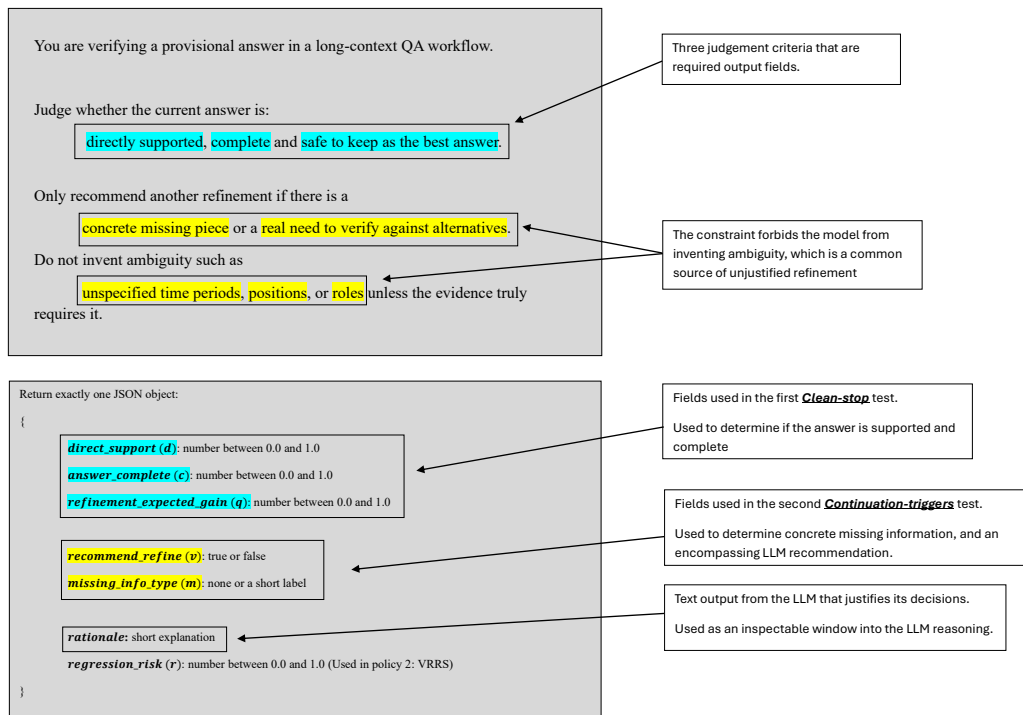


Figure 4.3 – Verifier prompt structure.

The verifier reads the saved first round state and emits structured fields; each field is a graded score in $[0, 1]$, projected into the bands defined in Section 4.4.2.

Sym.	Decision question	Interpretation	Output type
<i>d</i>	Is the provisional answer directly supported?	Assesses whether the provisional answer can be derived from the pinned context as given.	Band
<i>c</i>	Is the provisional answer complete?	Assesses whether the answer carries plausible information that could constitute an answer, and whether it matches the form demanded by the original question.	Band
<i>q</i>	Would another clarification round materially help?	Estimates the expected refinement gain from continuing. This field asks whether further clarification would produce material gain or whether the question is already simple enough to stop.	Band
<i>v</i>	Should the system run another refinement round?	Records the model's direct recommendation about whether the provisional answer is insufficient and another answer refinement round is beneficial.	Binary (true / false)
<i>m</i>	Is a concrete information type still missing?	Identifies whether the answer lacks a specific information type required by the question; for example, where the question requires a year but the answer gives none.	Binary (true / false)
<i>r</i>	Could continuation degrade the answer?	Estimates the regression risk of continuing: that is, the risk that a further clarification round would make the current answer worse.	Band

Table 4.6 – Verifier fields and output types.

4.4.3.4 Falsifier

The falsifier prompt, outlined in Figure 4.4, mirrors the verifier in input structure but reframes the task adversarially. The falsifier identifies whether the answer may still be wrong even when it appears coherent. Three known QA failure modes¹ are presented for the model to assess: an answer unsupported by the right evidence, a plausible competing answer that fits the question better, and an evidence selection locked onto the wrong entity, date, role, or relation. The prompt closes with a strict JSON contract that determines the required output.

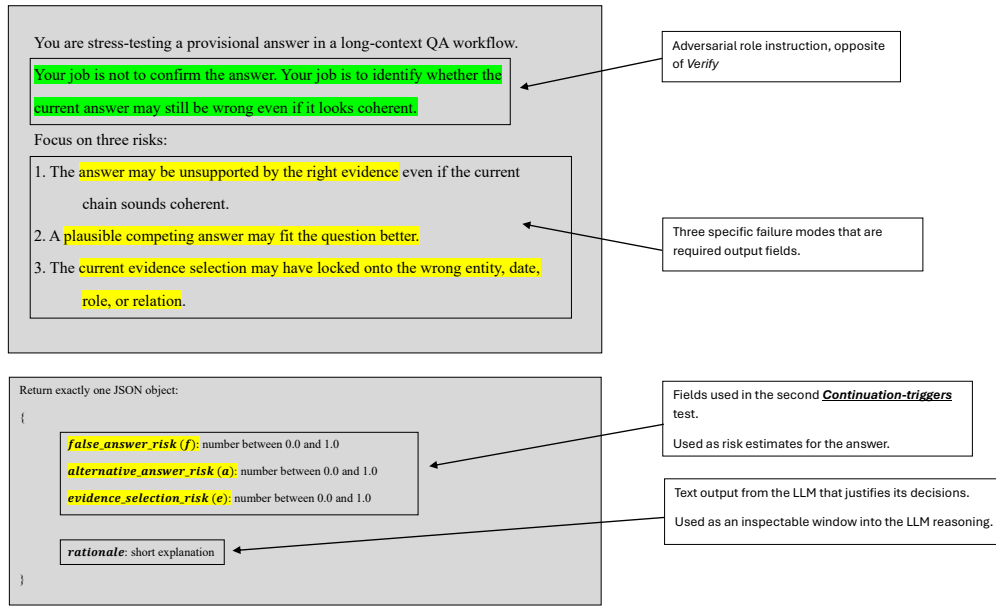


Figure 4.4 – Falsifier prompt structure.

The falsifier reads the same saved first round state as the verifier but reframes the task adversarially: it asks whether the answer is likely wrong despite appearing coherent. Fields emit graded scores in $[0, 1]$, projected into the bands defined in Section 4.4.2.

Table 4.7 details what each field entails, the motivating question and the output type.

¹These failure modes are established in the literature review and substantiated by the case study results. They are not derived from the evaluation suite.

Sym.	Decision question	Interpretation	Output type
f	Is the provisional answer likely to be incorrect?	Assesses the risk that the provisional answer is false, given the saved state context, including the clarification question, pinned context and provisional answer.	Band
a	Is there a plausible better alternative answer?	Assesses whether the pinned context leaves room for a competing answer that the clarification round did not adequately consider.	Band
e	Was the selected evidence weak or irrelevant?	Assesses whether the selected evidence was relevant to the original question.	Band

Table 4.7 – Falsifier fields and output types.

4.4.3.5 Decision construction

The parsed verifier and falsifier records feed into an ordered decision rule that emits the policy’s stop or continue decision. Figure 4.5 illustrates this rule.

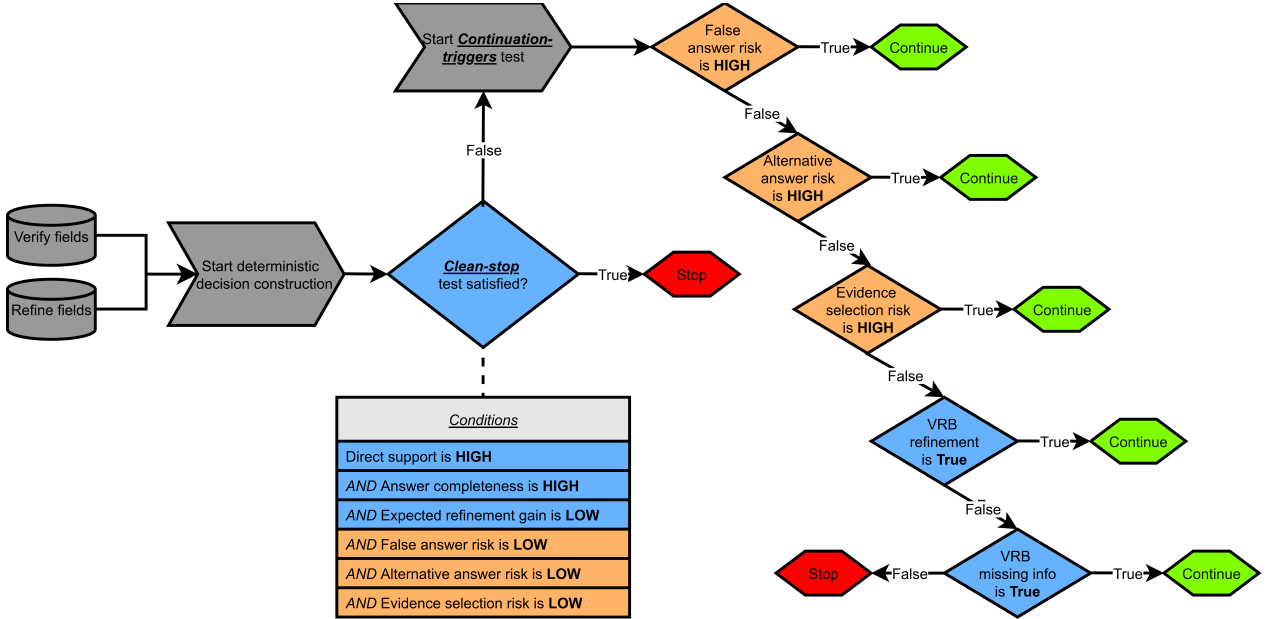


Figure 4.5 – VRB deterministic decision construction.

The **clean stop** test is evaluated first over the joint verifier and falsifier evidence and emits stop only when every condition holds. If any condition fails, an ordered cascade of **continuation triggers** fires on the first match.

The rule’s two stages relate asymmetrically. The clean stop test demands strong evidence on multiple axes at once because not stopping costs only an extra clarification round, whereas stopping risks accepting a wrong answer. The continuation triggers fire on a single risk field,

so the rule continues on moderate evidence of risk more readily than it stops on moderate evidence of correctness.

The cascade defaults to stop rather than continue. A controller that defaulted to continuation would treat every uncertain replay row as warranting another CoC round, inflating operational cost with no measurable gain in correctness. Thus, continuation is emitted only where a specific trigger has identified a reason for it. Both choices keep the policy auditable: every decision traces cleanly to the rule branch that produced it, and any failure case can be analysed against that branch.

Algorithm 4.1. VRB construction over a saved first round state

Require: Saved first round replay state

Ensure: Final stop or continue decision, verifier record, falsifier record, final decision reason

Phase A: Structured assessment

- 1: Construct the verifier and falsifier prompts from the shared decision context (Table 4.4).
- 2: Make two model calls and parse the JSON responses into the record:

Phase B: Ordered decision rule

Clean-stop test.

3: **if** Clean-stop test **then**

4: **Stop**

5: **return**

6: **end if**

Continuation triggers, evaluated in order. The first match emits continue.

7: **if** (False answer risk is High) **then**

8: **Continue**

9: **else if** (Alternate answer risk is High) **then**

10: **Continue**

11: **else if** (Evidence selection risk is High) **then**

12: **Continue**

13: **else if** (Verifier recommends refinement) and (Expected refinement gain is High) **then**

14: **Continue**

15: **else if** (Missing information type is valid) and (Expected refinement gain is High) **then**

16: **Continue**

17: **else**

18: **Stop**

19: **end if**

Replay output

20: Store the final decision, the final reason, the verifier record and the falsifier record in the replay row.

4.4.4 Verification and refinement with risk signals (VRRS)

VRRS extends VRB by adding a deterministic risk signal layer over the saved first round state and the parsed verifier record. The motivation is from the Stage 2 case study, which

establishes that continuation after the first clarification round is conditional rather than uniformly beneficial. Some first round answers are already supported well enough to accept. Others remain ambiguous, are weakly grounded in the pinned context, are overspecific to the clarification trace, or rest on evidence that does not fully resolve the original benchmark question. The deterministic layer makes these failure patterns explicit at the stopping boundary, so the controller can distinguish cases where continuation is likely to add value from cases where it only adds computation.

The design also addresses several weaknesses of LLMs applied to QA. Generated answers can be fluent despite being factually unsupported [13]. Factual reliability is uncertain for knowledge that is poorly represented in the context [62]. Long context models do not always use available evidence reliably, with use degrading when the relevant information is buried in the input [14]. VRRS operationalises these risks as decision signals over the replay state, so the controller inspects both whether the provisional answer appears plausible and whether the saved state contains concrete markers of unreliable answer quality.

4.4.4.1 System overview

The architecture has three deterministic stages: a construction stage that constructs risk signals from the saved state and the parsed VRB record, a classification stage that separates those signals by decision strength, and an aggregation stage that combines those signals into the policy’s decision. Figure 4.6 shows the corresponding flow.

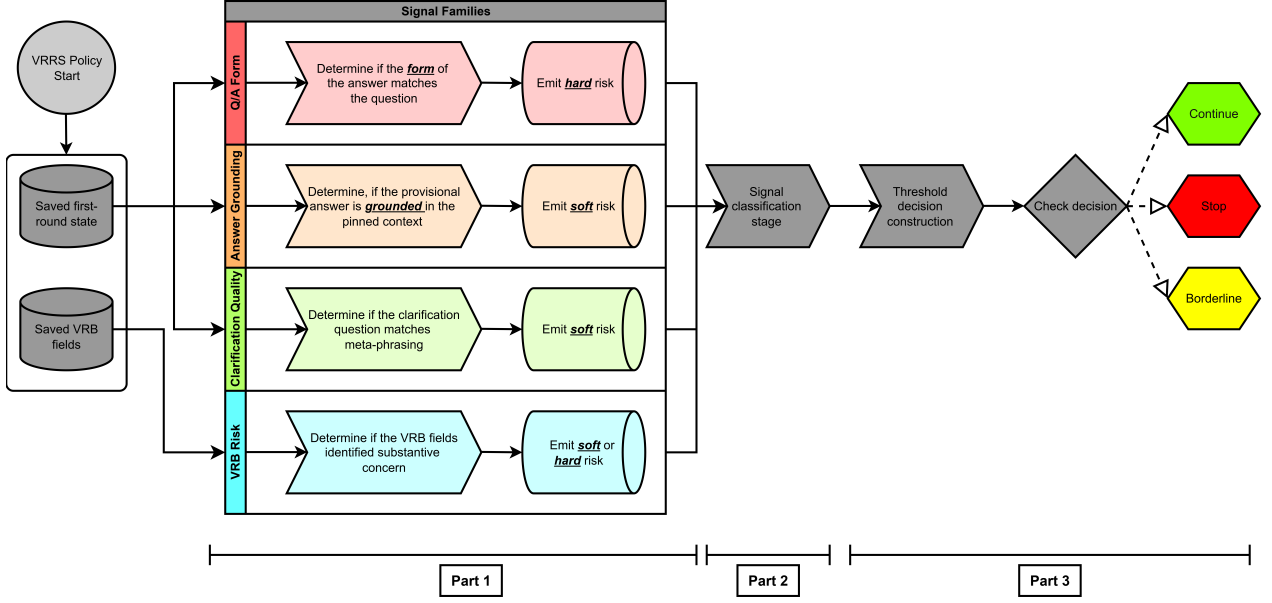


Figure 4.6 – VRRS system architecture.

The construction stage takes two inputs: the shared first round decision context (Table 4.4) and the parsed verifier record (Table 4.6). The constructed signals are classified into concrete **hard** and **soft** risks, and the final aggregation produces three decision outputs: continue, stop or borderline. Borderline cases are retained within VRRS as stop decisions, but are subsequently used as the trigger for semantic review in VRRS-BR.

The design has two notable properties. VRRS introduces no additional LLM calls: the verifier and falsifier calls inherited from VRB remain the only sources of model nondeterminism in the pipeline, and every step downstream of the parsed verifier record is a fixed computation over the replay row. The construction stage is also decomposed into four signal families rather than a single extractor. Each family targets a distinct slice of the replay state and a distinct category of risk.

4.4.4.2 Hard and soft risk definitions

The deterministic layer separates its signals into two classes by how decisive each is on its own. The verification, refinement, and overthinking literature together name the four ways a continuation can land, ranging from useful repair on under grounded answers to active degradation of already correct ones [35, 37–39]. Not every indicator on the saved first round state is equally strong evidence that the round has landed in one of those failure cases, and, consequently, the controller treats each class of signal under different conditions.

Hard risks are concrete failure patterns established in the QA literature. A single factoid question whose provisional answer has not committed to one candidate, an answer in the wrong form for a year seeking or count seeking question, and an answer that echoes the question rather than identifying an entity are all defects in how the answer relates to the

question. The hallucination literature shows that fluency and grounding can come apart, and that fluent surface form will not flag any of these defects [13]. The VRB record contributes a fourth hard signal: when the structured judgement reports substantive concern alongside expected gain, the round has already been assessed as one in which continuation has value. Each hard signal is calibrated so that, in isolation, it is strong enough to make stopping unsafe. Thus, a single active hard flag forces continuation. This treatment follows cost sensitive evaluation under asymmetric error costs, in which the decision logic reflects the consequence of each error type rather than weighting them equally [61]. A wrong stop on a concrete defect silently accepts an unsafe answer. A wrong continue costs a single clarification round and a recoverable subsequent decision.

Soft risks are weaker warning signs whose individual occurrence is not evidence that continuation is warranted. An answer absent from the pinned context may be a valid paraphrase of the supporting evidence. A meta phrased intermediate clarification answer may sit alongside other parts of the trace that did supply useful evidence. A diffuse pinned context need not point to any specific failure. Each soft signal aligns with what the long context literature predicts: the lost in the middle effect [14] shows that evidence use degrades with position and breadth, so a broad pinned context or an answer not visible in the selected span is a circumstantial indicator rather than a defect. As such, a single soft risk is rarely decisive. The joint occurrence of several soft indicators on the same replay row is informative, and the policy requires soft risks to accumulate and to be corroborated by an independent indicator of expected gain from the verifier before they justify another clarification round.

The separation also makes VRRS a layered combination of evidence rather than a single judgement. VRB establishes the structured LLM judge as the primary instrument once surface confidence is ruled out [33]. The same literature documents that even capable judges carry biases of position, format, and schema strictness that a single call does not fully mitigate [44, 45], and recommends combining decision aspects over relying on any one verdict [45]. VRRS combines three such aspects: deterministic hard signals catch the concrete defects the judge can overweight or underweight, soft signals accumulate evidence over independent slices of the replay state, and the VRB fields contribute the structured judgement itself. The hard and soft separation keeps these three sources from collapsing into a single bit at the controller boundary, and distinguishes VRRS from an uncertainty maximising controller that would continue on any weak evidence of concern.

4.4.4.3 Signal families

The deterministic layer organises its inspection of the replay state into four signal families. Each family targets a different source of stopping risk and is constructed from a different slice of the available inputs. The decomposition keeps the controller inspectable: every

constructed signal can be traced to the input from which it was derived.

These signals do not prove that a given answer is wrong. They identify evidence that stopping on the first round is unsafe.

QA form signals examine the relationship between the original question and the provisional answer. The family infers the expected answer type from the question² and tests whether the provisional answer matches it. Concrete cases include a *year seeking* question with no four digit year in the answer, a *count seeking* question with no digit in the answer, and a *single factoid* question with multiple unresolved candidates in the answer. A separate case is an answer containing *hedging* between several possibilities, regardless of question type. These are *Hard* signals: each case is a concrete defect in the answer.

Signal	Type	Construction procedure	Trigger
Multi-candidate answer	H	(i) Test single factoid form: question begins with <i>who</i> , <i>when</i> , <i>where</i> , <i>which</i> , <i>what</i> or <i>how many</i> . (ii) Count candidates in the answer using a heuristic over quoted items, <i>or/and</i> splits, comma cues, and title spans. (iii) Detect hedge cues: <i>possible</i> , <i>possibly</i> , <i>may be</i> , <i>might be</i> , <i>could be</i> , <i>likely</i> .	single-factoid and (candidate count ≥ 2 or hedge present)
QA form mismatch	H	(i) Question matches <i>what year</i> or <i>when</i> as year-seeking, or <i>how many</i> as count-seeking. (ii) Answer is scanned for a four-digit year using regex <code>\b(1[5-9]\d\d 20\d\d 2100)\b</code> , and for any digit using <code>\d</code> .	year/count expected and answer has no year and no digit
Question echo behaviour	H	(i) Answer token count is too low. (ii) The question is folded into the answer, indicating the final answer is incomplete.	Token counting and question/answer inclusion.

Table 4.8 – QA form family: signals, construction procedures and trigger conditions.

Grounding signals inspect whether the provisional answer is supported by the pinned context. The family tests three properties of the answer’s relationship to the pinned evidence: presence of the answer text in the pinned context, presence of the answer text only in the intermediate clarification answer, and breadth of the pinned context itself. These checks target replay states in which the answer may have been produced from weak, indirect, or insufficient evidence. These are *Soft* signals: each case is circumstantial evidence rather than a concrete defect.

²Answer types are established in Chapter 2 and in the case study results (Chapter 5). They include *year*, *count*, *country*, *person*, *city*, and *single factoid*.

Signal	Type	Construction procedure	Trigger
Answer only in intermediate	S	Is the answer contained within (i) the pinned context and (ii) the intermediate clarification answer.	answer in intermediate and answer not in pinned context
Answer missing from pinned context	S	Is the answer contained within the pinned context.	answer not in pinned context
Broad pinned context	S	Count of <para N> markers in the pinned context.	marker count ≥ 20 , set as the broad-context threshold beyond which the pinned evidence becomes too diffuse for the model to use reliably

Table 4.9 – Grounding family: signals, construction procedures and trigger conditions.

Clarification quality signals inspect the clarification trace itself. The family flags meta phrasing in the clarification question or intermediate clarification answer, such as a request for additional context, that signals the clarification step did not substantively narrow the answer space. A first round provisional answer can appear fluent even when the clarification step has produced no useful evidence, and this family makes those cases visible to the controller. This is a *Soft* signal: the case is circumstantial evidence that the clarification trace itself is unreliable.

Signal	Type	Construction procedure	Trigger
Meta clarification answer	S	Matching of the clarification answer against known meta phrasing that indicates the pinned context did not contain relevant material.	any cue matches

Table 4.10 – Clarification quality family: signal, construction procedure and trigger condition.

VRB risk signals reuse the parsed verifier record. The family extracts support, completeness, expected refinement gain, missing information type, and the refinement recommendation, and converts them into risk signals over the verifier fields. It is the bridge between VRB’s structured assessment and VRRS’s deterministic decision, covering cases where the string matching and form checking of the first three families lack discriminating power and the verifier’s judgement carries the signal instead. The family produces one signal of each type: a *Hard* signal when the verifier reports substantive concern alongside expected gain, and a *Soft* signal when high verifier confidence sits alongside an active grounding-family flag.

Signal	Type	Construction procedure	Trigger
VRB high risk	H	Band comparisons over the parsed verifier fields d (support), c (completeness), q (expected gain), r (regression risk), and categorical checks on v (recommend refine) and m (missing info type).	q is Moderate or High and (v is true or m is concrete or r is Moderate or High)
High confidence with low grounding	S	Band comparison on d , c and q , combined with the output of the grounding family.	d is High and c is High and q is Low and any grounding-family soft flag is active

Table 4.11 – VRB risk family: signals, construction procedures and trigger conditions.

4.4.4.4 Decision construction

An ordered decision rule consumes the classified risks and assigns each replay row to one of three decisions: continue, stop, or borderline. The rule is ordered because the categories of evidence are not symmetric. Hard risks are decisive on their own, the clean stop conditions require strong evidence on multiple fronts, and borderline classification is reserved for cases that neither resolves. Table 4.12 states the full decision rule in evaluation order.

Step	Output	Rule
1	Continue	If any Hard signal is active.
2	Stop	If no Hard signal is active, only minimal Soft signals are active, and the VRB fields indicate high confidence and low expected gain.
3	Continue	If no Hard signal is active, but the VRB fields indicate moderate or high expected gain and a substantial number of Soft signals is active.
4	Borderline	If no Hard signal is active, but VRB High Risk is active and some Soft signals are active.
5	Stop default	If no prior rule fires.

Table 4.12 – Ordered VRRS decision rule.

Rules are evaluated top to bottom; the first matching rule determines the decision.

The asymmetric treatment of the two risk classes is the core of this policy. Step 1 emits continue on a single hard flag because hard signals are concrete defects whose individual occurrence already makes stopping unsafe under the cost asymmetry of the controller task [61]. Step 3 by contrast emits continue only when soft flags have accumulated *and* the VRB ex-

pected gain corroborates them. Each soft signal is individually circumstantial, but their joint occurrence on the same replay row, combined with VRB’s independent assessment, is the layered evidence the judging literature recommends in place of any single verdict [45].

Borderline classification at step 4 is reserved for replay rows in which the verifier reports substantive concern through VRB **High risk** but the deterministic layer has not accumulated enough soft evidence to justify continuation on its own. A single structured judgement is not conclusive in this case, and the LLM as judge literature recommends a deferral layer rather than a verdict for it [33, 44]. Within VRRS, borderline cases default to stop so that the policy emits the binary decision required by the evaluation protocol. The borderline classification is also recorded explicitly in the replay row, so that VRRS-BR can use it as the trigger for semantic review without altering the decisions VRRS has already resolved. Step 5 then defaults to stop on the same principle as VRB: continuation is emitted only where the rule has identified a specific reason for it. All five steps preserve auditability: every classification is traceable to a specific rule branch and a specific set of active signals.

The full VRRS construction is specified as two algorithms. Algorithm 4.2 is the system coordinator: it extracts fields, constructs the signal families, computes the aggregate hard and soft counts, and dispatches to the decision stage. Algorithm 4.3 consumes the assembled signal bundle and runs the ordered logic of Table 4.12.

Algorithm 4.2. VRRS system construction

Require: Saved first round replay state s ; parsed VRB record

Ensure: Decision band, stop or continue decision, decision reason

- 1: Extract verifier fields from the VRB record.
 - 2: Evaluate VRB **High risk** using the rule in Table 4.11.
 - 3: Initialise $H \leftarrow \emptyset$, $S \leftarrow \emptyset$.
 - 4: $(H, S) \leftarrow \text{QAFORMFAMILY}(H, S, s)$
 - 5: $(H, S) \leftarrow \text{GROUNDINGFAMILY}(H, S, s)$
 - 6: $(H, S) \leftarrow \text{CLARIFICATIONQUALITYFAMILY}(H, S, s)$
 - 7: $(H, S) \leftarrow \text{VERIFIERRISKFAMILY}(H, S, d, c, q, v, m, r)$
 - 8: Compute $|H|$ and $|S|$.
 - 9: **return** $\text{DECISIONCONSTRUCTION}(H, S, \text{VRB High risk}, d, c, q)$ ▷ Algorithm 4.3
-

Algorithm 4.3. VRRS decision construction (DECISIONCONSTRUCTION)

Require: Hard flags H ; Soft flags S ; VRB High risk; VRB d, c, q ;**Ensure:** Decision band, stop or continue decision, decision reason*Step 1: hard-risk continuation.*1: **if** $|H| \geq 1$ **then**

2: Continue

3: **end if***Step 2: clean stop test.*4: **if** d is High **and** c is High **and** q is Low **and** $|S| \leq 2$ **then**

5: Stop

6: **end if***Step 3: VRB continuation gain and soft risk continuation.*7: **if** q is Moderate or High **and** $|S| \geq 3$ **then**

8: Continue

9: **end if***Step 4: borderline.*10: **if** $|S| \geq 2$ **and** VRB High risk **then**

11: Stop

▷ Borderline case; recorded for use by VRRS-BR

12: **end if***Step 5: default stop.*13: Stop

4.4.5 Verification and refinement with risk signals and borderline review (VRRS-BR)

VRRS-BR adds a *semantic review* stage to VRRS, invoked only on the borderline cases. This policy tests if semantic judgement can improve the stopping decision in the uncertain region left by deterministic risk signals. Deterministic stop and continue decisions from VRRS are preserved, and borderline cases are routed to an additional structured LLM call whose conclusion enters the final decision.

4.4.5.1 System overview

The policy takes the *borderline* decision produced by VRRS as its input. The *stop* and *continue* decisions from VRRS pass through unchanged. Borderline decisions are routed to the semantic review stage, which produces a structured evaluation over the shared context. The controller then parses the evaluation and applies the final continuation filter, yielding a *stop* or *continue* decision. Figure 4.7 illustrates the process.

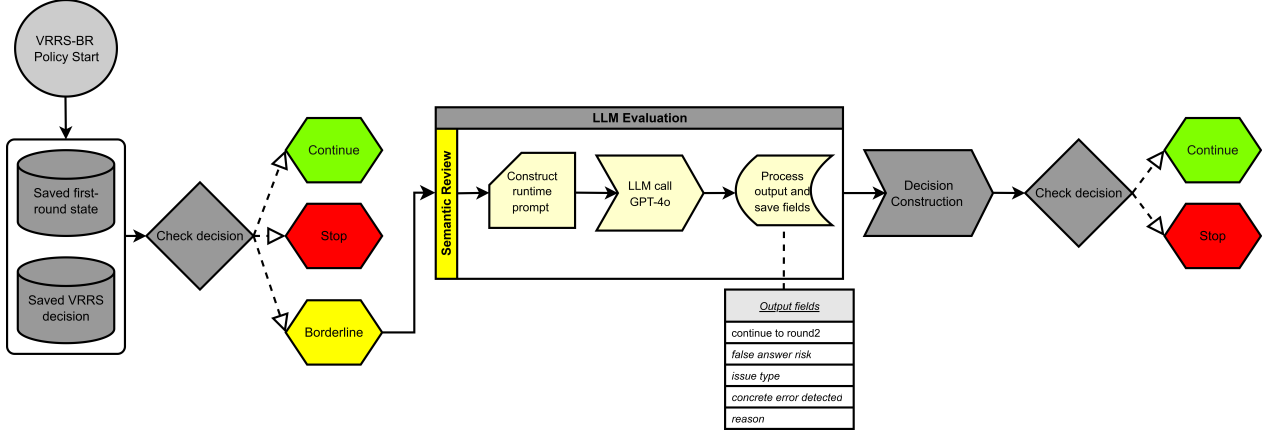


Figure 4.7 – VRRS-BR system architecture.

Selective routing of **borderline** cases through a semantic reviewer and final continuation filter; deterministic stop and continue decisions from VRRS are preserved.

The design has two notable properties. The semantic stage is invoked selectively rather than universally: routing only the borderline cases through the reviewer confines the additional model cost and latency to only those examples where the deterministic signals are uncertain. Stop and continue decisions from VRRS are protected and never routed to further review. This restriction preserves the controller interpretation of the policy and isolates the later comparison with VRRS to the question of whether semantic review improves decisions in the borderline region.

4.4.5.2 Semantic reviewer

The semantic reviewer is a structured LLM call with specific instructions. The review is not an open evaluation of answer quality. The task is to judge whether the existing first round answer contains a concrete error that would make stopping unsafe. The prompt explicitly instructs the model to recommend continuation only where it can identify such an error, and to treat under grounding alone as insufficient grounds for continuation. Two reasons motivate this restriction. VRRS has already investigated grounding through its grounding family, so asking the reviewer to repeat that check would add cost without adding new evidence. Tasking the model to thoroughly investigate the pinned context would also risk the same limitation that constrains the original CoC system: reasoning over long context is not always reliable, with evidence use degrading sharply when the relevant information is not at the start or end of the input [14]. Constraining the reviewer to concrete error identification rather than open grounding inspection keeps the call within the regime where structured LLM judgement is most reliable.

Figure 4.8 shows the prompt template and the structured output.

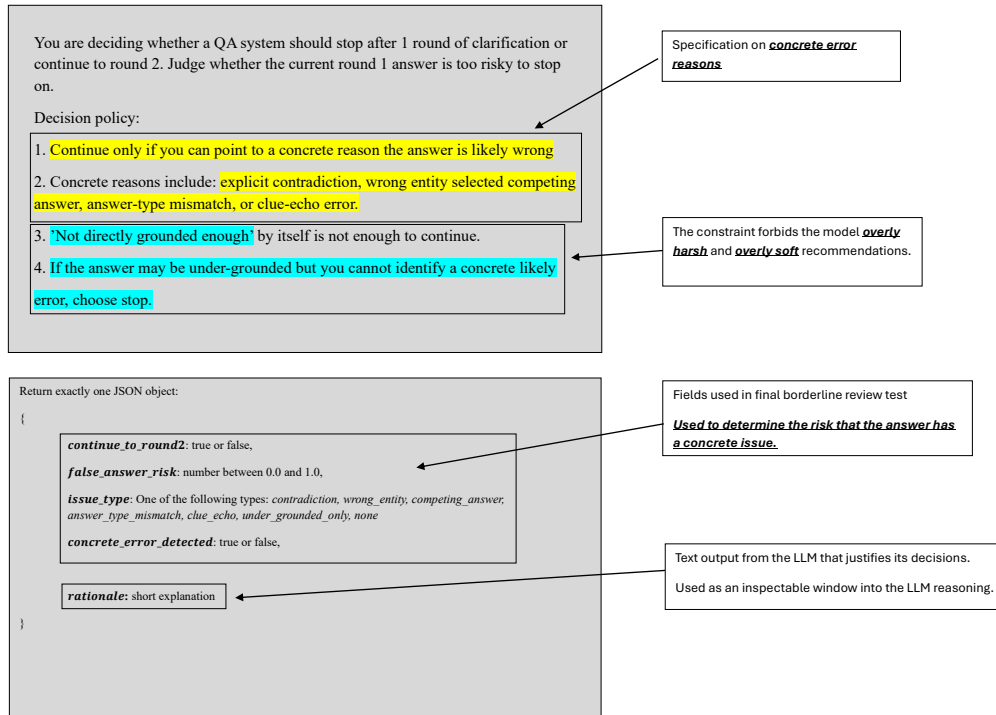


Figure 4.8 – Semantic reviewer prompt structure.

The reviewer is an evaluator: it is instructed to recommend continuation only where it can identify a concrete named error, and is forbidden from continuing on under grounding alone.

The reviewer receives more information than the verifier or the falsifier. In addition to the five shared decision context fields, it is given a summary of the parsed verifier record and the local deterministic risk flags constructed by VRRS. This expanded input set is a key design choice: the reviewer is being asked to resolve borderline cases, and that task requires knowledge of what the deterministic system has already concluded. Without the verifier summary and the risk flags, the reviewer would be reconstructing context that the controller already possesses.

The prompt then constrains how that information may be used. Continuation is permitted only where the reviewer can identify a concrete error that fits one of the established categories: *explicit contradiction*, *wrong entity*, *competing answer*, *answer type mismatch*, or *clue echo*. Under grounding alone, vague concern about coherence, and general uncertainty are explicitly forbidden as grounds for continuation. The prompt closes with a strict JSON contract that determines the required output. At runtime, the shared decision context fields, the verifier summary, and the risk flags are substituted into their corresponding slots before the call is made. The raw prompt template, exact input substitution, and full output schema are given in Appendix B.6.

4.4.5.3 Final continuation filter

The reviewer does not produce the final stop or continue decision directly. Its task is to investigate and construct a set of fields about the answer and the context that produced it. The controller then checks that they jointly indicate continuation is warranted. This follows the same pattern as the verifier and falsifier in VRB: the model produces structured evidence over the saved state, and a deterministic rule turns that evidence into the policy’s decision. The continue decision is licensed by the reviewer’s joint field values rather than by a single un-auditable recommendation. However, unlike VRB, the reviewer sits closer to the decision itself. Its structured fields include a direct continuation recommendation and a graded risk on the answer, and the filter cascade acts as a veto over that recommendation rather than as a constructor of the decision from independent evidence.

Figure 4.9 shows the rule as a sequential cascade over the parsed reviewer record.

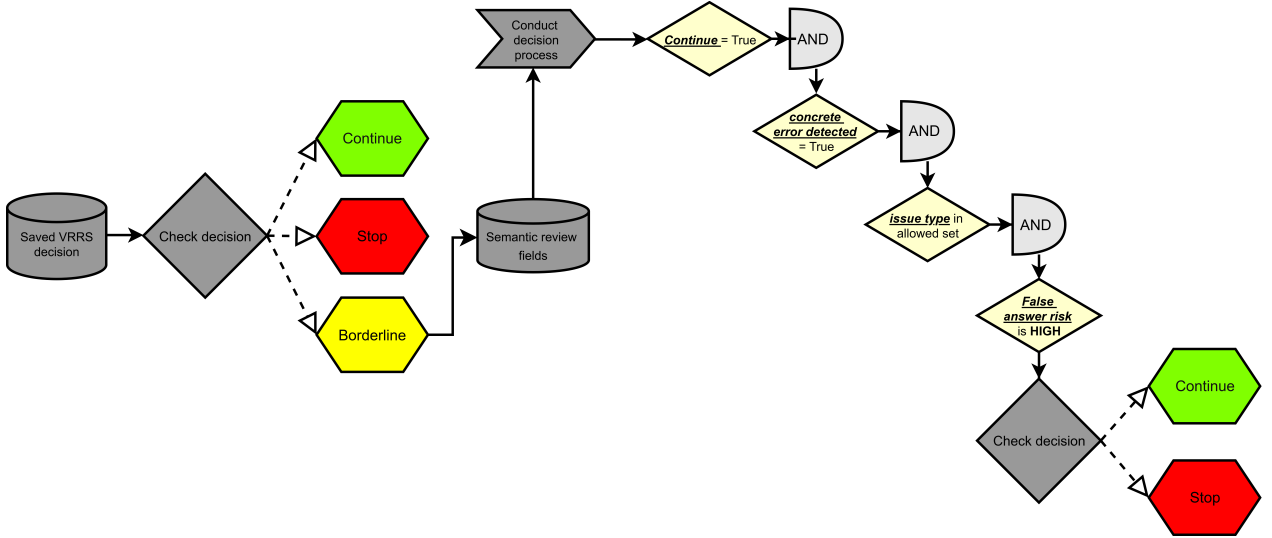


Figure 4.9 – VRRS-BR deterministic decision construction.

The semantic reviewer output is evaluated against the four filter conditions; any failed condition reverts the borderline case to a stop outcome.

This four condition construction defends against known failure modes of unconstrained LLM review. Condition 1 requires the model to express an intent to continue. Condition 2 prevents continuation on the basis of generalised concern that has not been localised to a known QA failure pattern. Condition 3 restricts the controller to those patterns, rather than permitting the reviewer to invent its own categories. Condition 4 requires the model’s false answer risk indicator to be in the *High* band rather than *Low* or *Moderate*, so that continuation is licensed only where the reviewer has placed substantial weight on the answer being wrong. The conditions together use the model to identify errors and risks while keeping the final decision auditable.

Table 4.13 states the four conditions for continuation, together with the concrete error categories admitted by the `issue type` condition.

#	Filter condition	Reviewer field	Required value
1	Continuation to a second round is warranted	<code>continue to second-round</code>	<code>true</code>
2	Concrete error detected	<code>concrete error detected</code>	<code>true</code>
3	Issue type in allowed set	<code>issue type</code>	One of: <code>contradiction</code> , <code>wrong entity</code> , <code>competing answer</code> , <code>answer type mismatch</code> , <code>clue echo</code>
4	False answer risk	<code>false answer risk</code>	High band

Table 4.13 – VRRS-BR final continuation filter.

All four conditions must be satisfied for a borderline case to convert into a continue decision; otherwise, the case reverts to a stop outcome.

The full VRRS-BR pipeline, composing the VRRS stage with the semantic review stage and the filter cascade, is specified in Algorithm 4.4.

Algorithm 4.4. VRRS-BR construction over a saved first round state

Require: Saved first round replay state; parsed VRB verifier record

Ensure: Final stop or continue decision, semantic review record (where invoked), final decision reason

Phase A: VRRS pre-pass

1: Run VRRS construction (Algorithms 4.2 and 4.3) on the replay row.

Phase B: Selective routing

2: **if** VRRS band is `Continue` or `Stop` **then**

3: **return** the VRRS decision unchanged.

4: **end if**

Phase C: Semantic review (borderline only)

5: Call the semantic reviewer and parse the JSON response into the reviewer record.

Phase D: Final continuation filter (Table 4.13)

6: **if** every filter condition is satisfied **then**

7: **return** `Continue`

8: **else**

9: **return** `Stop`

10: **end if**

Replay output

11: Store the final decision, the final reason and the reviewer record in the replay row.

4.5 Evaluation protocol

The evaluation protocol that produced the main findings reported in Chapter 5 follows the staged structure of the methodology. The case study and offline stopping gate evaluation are two layers of the same analysis, and a shared set of validity controls applies to both.

4.5.1 Procedure

The evaluation has two sections. The first is the Stage 2 case study, which examines realised continuation trajectories and classifies whether a second clarification round *repaired*, *preserved*, *failed to repair*, or *degraded* the answer state. The case study layer characterises how continuation behaves in CoC and establishes the motivation for treating the first clarification boundary as a control point. The second is the Stage 3 offline stopping gate evaluation, which treats the first round boundary as a binary controller decision and evaluates STARC at its three layers (VRB, VRRS, VRRS-BR) in sequence.

The two sections are sequential: the case study asks what happens when the system continues, and the stopping gate evaluation asks whether a policy can decide when continuation is appropriate. An investigation without the case study layer would have to assume that continuation is uniformly beneficial, and the appropriate policy would simply be *always continue*. The case study removes that assumption by showing that continuation has conditional rather than uniform value.

Furthermore, the order of the two sections is a validity choice. The case study develops the typology and informs the band placements before the replay suite is consulted, so the policy design and its evaluation set are separated in time.

4.5.2 Evaluation objects

The policy implementation is based on the dataset and saved state objects defined in Section 4.1, but the evaluated objects in either layer are not raw benchmark questions. In the case study, the evaluated object is the saved continuation trajectory, containing both the first round and second round answer states and classified by the manual judging procedure described in Section 4.3.3. In the stopping gate evaluation, the evaluated object is the first round replay row defined in Section 4.4.1, which preserves the CoC inference information available at the first clarification boundary.

Policy decisions are constructed exclusively from the saved state and the signals derived from

it. Gold answers and correctness labels are attached only afterwards. This separation between decision time and evaluation time is the main validity control of the protocol, discussed in Section 4.5.6. The answer judging procedure that produces the correctness labels is given in Appendix B.2.3.

Each policy produces a binary stop or continue decision for each replay row, together with a trace recording the active rule branch, the active signals, and the decision reason. The trace makes each decision attributable to a specific layer of the policy and lets differences between policies be localised when they are compared.

4.5.3 Stopping task and outcome cells

The stopping task is a binary classification over the first round replay rows. A row is correct if the saved first round answer is judged correct under the offline judging protocol (Section 4.5.5). Otherwise it is wrong. The desired controller action is to stop on correct rows and to continue on wrong rows. Crossing the desired action with the policy’s actual action produces the four mutually exclusive outcome cells shown in Figure 4.10.

	Correct first round answer	Wrong first round answer
Policy stops	safe stop	premature stop
Policy continues	unnecessary continuation	justified continuation

Figure 4.10 – Outcome cells of the stopping task.

The two correct controller outcomes are **safe stop** (correctly stopping on a correct answer) and **justified continuation** (correctly continuing on a wrong answer). The two controller errors are **premature stop** (stopping on a wrong answer, the unsafe failure) and **unnecessary continuation** (continuing on a correct answer, the inefficient failure).

The two errors are not symmetric in operational consequence.

A **premature stop** accepts an unsafe answer and is the failure the stopping gate is designed to prevent. It is the more costly of the two errors. An **unnecessary continuation** sends a correct answer to a further clarification round, incurring the operational cost of one additional inference call but not directly compromising answer quality. The metric set chosen in Section 4.5.4 keeps both error classes visible separately, so that no policy can be credited for reducing one error class by inflating the other.

The task evaluates the appropriateness of the boundary decision rather than the eventual success of any subsequent clarification round. A continue decision on a wrong answer counts as a successful gate decision because the controller has avoided premature acceptance of an

unsafe answer. This does not imply that the next clarification round will repair that answer.

4.5.4 Metrics

The metric set is chosen to expose both controller responsibilities: preserving correct answers through stopping and detecting wrong answers through continuation. The metrics remain interpretable under the class imbalance of the replay suite.

For compactness, the four outcome cells are written $SS = \text{safe stop}$, $JC = \text{justified continuation}$, $UC = \text{unnecessary continuation}$ and $PS = \text{premature stop}$. The total number of evaluated examples is

$$N = SS + JC + UC + PS. \quad (4.1)$$

Aggregate correctness. Gate success measures the proportion of replay rows on which the controller selected the desired action, and gate error is its complement:

$$\text{Gate success} = \frac{SS + JC}{N}, \quad \text{Gate error} = \frac{UC + PS}{N}. \quad (4.2)$$

Gate success on its own is not a sufficient principal metric. The replay suite contains more correct than wrong first round answers, so a policy that stops frequently can achieve a moderate gate success score while still failing to detect many unsafe wrong answers. The per class recalls and the class balanced summaries below address this.

Recalls. Stop recall measures the preservation of correct answers. Continue recall measures the detection of wrong answers:

$$\text{Stop recall} = \frac{SS}{SS + UC}, \quad \text{Continue recall} = \frac{JC}{JC + PS}. \quad (4.3)$$

Balanced accuracy averages the two recalls, so that performance on the majority class does not dominate the result:

$$\text{Balanced accuracy} = \frac{\text{Stop recall} + \text{Continue recall}}{2}. \quad (4.4)$$

Precisions. Stop precision measures the safety of accepting answers: the proportion of policy stops that are safe. Continue precision measures the usefulness of continuation decisions: the proportion of policy continuations that are justified:

$$\text{Stop precision} = \frac{SS}{SS + PS}, \quad \text{Continue precision} = \frac{JC}{JC + UC}. \quad (4.5)$$

Confusion aware summary. Matthews Correlation Coefficient (MCC) is reported as a principal summary because it evaluates the full binary confusion structure and is less misleading than raw accuracy under class imbalance [58]. In the controller framing used here, continuation on a wrong answer is the positive target action: JC is the true positive cell, SS is the true negative cell, UC is the false positive cell and PS is the false negative cell. The coefficient is

$$\text{MCC} = \frac{(JC \cdot SS) - (UC \cdot PS)}{\sqrt{(JC + UC)(JC + PS)(SS + UC)(SS + PS)}}. \quad (4.6)$$

Operational cost. Predicted continue count records the number of replay rows the policy sends to a further clarification round:

$$\text{Predicted continue count} = JC + UC. \quad (4.7)$$

Each continuation triggers an additional clarification round, so a policy that detects more unsafe answers may still be undesirable if it does so by sending too many already correct answers to that further round. Reporting recall, precision, balanced accuracy, MCC and the predicted continue count together makes the stopping trade off visible: preserving correct answers through stopping while detecting unsafe answers through continuation. Table 4.14 summarises the metric set and the interpretive role each metric plays in the comparison of policies.

Metric	Operational definition	Interpretive role
Gate success	$(SS + JC)/N$	Overall correctness of the stop/continue decision.
Gate error	$(UC + PS)/N$	Overall controller failure rate.
Stop recall	$SS/(SS + UC)$	Preservation of correct answers through stopping.
Continue recall	$JC/(JC + PS)$	Detection of unsafe answers through continuation.
Stop precision	$SS/(SS + PS)$	Safety of accepting answers (probability that a stop is safe).
Continue precision	$JC/(JC + UC)$	Usefulness of continuation (probability that a continuation is justified).
Balanced accuracy	Mean of stop recall and continue recall.	Reduces distortion from class imbalance.
Matthews correlation coefficient	Eq. 4.6	Summary over the full confusion structure that accounts for class imbalance.
Predicted continue count	$JC + UC$	Operational cost of the policy.

Table 4.14 – Controller metrics and their interpretive role in the stopping gate evaluation.

The metrics are reported per dataset (Natural Questions, HotpotQA, TriviaQA, PopQA), per policy and aggregated across the full replay suite, so that policy comparisons can be inspected at the level where any difference is most informative.

4.5.5 Judging the labels

The Stage 3 stopping gate evaluation uses the automated correctness judging procedure defined by the AgenticLU base protocol (Appendix B.2.2), applied to the saved first round answers in the replay suite. Correctness labels are produced for each first round answer by comparing the answer text against the gold answer set under a fixed prompt and a strict JSON output configuration. Automated judging is appropriate at this scale: the replay suite contains 408 examples across 85 base question clusters, manual judging at that volume is

impractical, and the binary correctness label is well suited to a structured judge.

However, using an LLM as a judge is itself an evaluation choice rather than a stable ground truth, and LLM judges have well defined limitations on long context QA, including position dependent behaviour and surface form sensitivity [33, 34]. Initial application of the inherited protocol surfaced concrete failure modes in the produced labels, including unstable judgements across context variants of the same base question and acceptance of unresolved multi candidate answers (Appendix B.2.3.1). To bound the effect of these failure modes on the committed labels, the automated judge was run twice over the full replay suite. Disagreements between the two passes were manually reviewed, and the reconciled output of that review is the label set used for every reported metric.

Two consequences follow. First, the absolute correctness numbers reported in Chapter 5 should be read as model judged estimates, not as the true answer correctness rate. Second, every metric and policy comparison in the thesis is reported on the same fixed correctness labels. Comparisons between policies are therefore comparisons under a single judging configuration rather than against an idealised ground truth. The automated judging prompt, the two pass review, and the failure modes surfaced during it are specified in Appendix B.2.3.

4.5.6 Validity controls and limits

The protocol imposes clear limits on the claims that can be made. The controls already introduced carry their own scope conditions: decision time and evaluation time separation (Section 4.5.2), fixed replay states across policies (Section 4.4.1), and the controller task framing (Section 4.5.3). The controls and limits specific to the evaluation follow.

Replay scope. The replay suite contains 408 first round states across 85 base question clusters, where each base question is materialised in multiple context variants. The total replay count should be read as the number of reconstructed states evaluated by the controller, not as the number of independent semantic questions in the suite. Clusters introduce correlation. A controller that fails on one variant of a base question is more likely than chance to fail on its sibling variants, so aggregate metrics overstate the effective sample size if read as independent observations. This scoping is appropriate to the controller framing, because the stopping decision operates over realised saved states rather than abstract questions in isolation. The scoping does, however, limit claims about performance across all possible question instances or all possible CoC trajectories.

Band selection and stability. The three policies use fixed band placements rather than learned parameters. The risk this raises is that placements selected against the replay suite would inflate reported performance. The case study, the prompt design iterations, and the band placement step were all completed before the replay suite was used as an evaluation set, and the case study trajectories are drawn from Natural Questions only while the replay suite spans four datasets and 85 base question clusters. The stability of the reported results to placement perturbation is examined in Chapter 5. Tuning the band placements on a formally separated development split is left as future work.

Judging uncertainty. Correctness labels for the Stage 3 evaluation depend on an automated LLM judging procedure (Section 4.5.5). Absolute correctness rates and policy gate success values should be treated as evaluation estimates produced under one judge configuration, rather than as a stable ground truth. Comparisons between policies are nonetheless made under the same two pass reviewed labels, which is the strongest internal comparison available without a manually adjudicated gold set across the full suite. The case study, which uses manual judging, provides the human grounded reference point for the stopping framing as a whole.

Replay only evaluation. The protocol evaluates the stopping decision over saved first round states rather than in a fully live adaptive system that decides repeatedly after each clarification round. The thesis therefore supports a direct claim about the first round stopping decision and the controller behaviour at that boundary. It does not estimate the final answer accuracy that would result from executing all continuation decisions to completion. This scoping is consistent with the staged contribution of the methodology, which positions the thesis as an investigation of the stopping problem rather than a complete solution to multi round adaptive reasoning.

4.6 Chapter summary

The methodology frames the first clarification boundary in the CoC workflow as a controller decision problem, and develops a three stage approach for studying it. Stage 1 instruments the CoC runtime so that the round state at each clarification boundary is captured as a recoverable structured object, with persistent progress writing so that extended clarification runs can be resumed across interrupted compute sessions. The empirical work is grounded in four QA benchmarks (Natural Questions, HotpotQA, TriviaQA, PopQA), instantiated under the maximum AgenticLU long context configuration of 128k tokens and resolved into saved round states by this instrumentation.

Stage 2 characterises continuation behaviour at the first clarification boundary through a trajectory case study, classifying each second round transition as repair, preservation, failed repair, or degradation under manual adjudication. The case study supplies the mechanism level evidence that defines what the stopping problem looks like and identifies the failure patterns STARC is designed to detect.

Stage 3 delivers the State Aware Risk Controller (STARC), evaluated as three progressively layered stopping policies: VRB, VRRS, and VRRS-BR. VRB is a structured LLM baseline combining a verifier and a falsifier under a deterministic decision rule. VRRS adds a deterministic risk signal layer over four signal families, separating hard from soft risks and producing a borderline category for cases the deterministic layer cannot resolve. VRRS-BR adds a constrained semantic reviewer invoked only on borderline cases, admitted to the decision only through a deterministic continuation filter.

The evaluation protocol holds the replay states constant across policies and separates the inputs available at decision time from the correctness labels used at evaluation time. The 408 saved first round replay states are drawn from 85 base question clusters and use a fixed replay design that isolates differences in stopping logic from differences in upstream clarification traces. The metric set is chosen to expose the stop or continue trade off under the class imbalance of the suite, with the Matthews correlation coefficient reported as the principal summary. The empirical results produced under this protocol are reported in [Chapter 5](#) below.

Chapter 5

Results

This chapter reports two findings. The first motivates the thesis; the second delivers it. From an analysis of continuation within the Chain of Clarifications (CoC), a second clarification round is shown to be only conditionally beneficial: across the case study trajectories it repaired some first round answers, preserved others, failed to repair others, and degraded a few that were already correct. This establishes the first clarification boundary as a genuine control point. The second finding is that a controller can act on that boundary well: STARC decides whether to stop or continue from the saved first round state alone, and its strongest layer substantially improves the decision over a structured judgement baseline.

This thesis proposes STARC architecture and is developed as three progressively layered policies. VRB produces structured verifier and falsifier judgements, VRRS adds a deterministic stage that aggregates those judgements into risk bands, and VRRS-BR adds a gated semantic reviewer on the cases the deterministic stage cannot resolve. Against VRB as the baseline, the full controller raises wrong answer detection (continue recall) by 22.3 percentage points.

Headline result

Across the 408 example replay suite, VRRS-BR raises continue recall from 44.66% to 66.99%, balanced accuracy from 66.10% to 73.99%, and MCC from 0.346 to 0.450 over the VRB. The full controller catches 67.0% of wrong first round answers against VRB's 44.7%, at the cost of 20 additional unnecessary continuations on 305 already correct cases.

This gain is a risk sensitive redistribution, not a free improvement: STARC catches substantially more wrong answers at the bounded cost of continuing a few already correct ones, and the class balanced metrics confirm the trade is favourable rather than a one sided shift along the error curve.

The chapter reaches these findings through two analyses. The first is the case study of what a second clarification round does to a first round answer, which characterises continuation as a control point and, with the established literature, motivates a stopping decision at the boundary. The second is an offline evaluation of STARC’s three policies over a fixed replay suite of saved first round states. Holding the inputs fixed isolates the comparison, so differences in behaviour read as differences in stopping logic rather than as noise from the underlying CoC system. Both rest on the replay apparatus developed in Chapter 4.

The aggregate improvement is not distributed evenly across the four datasets, and this is the strongest evidence the chapter offers for the layered design. The datasets were chosen to pose different kinds of question: TriviaQA leans on memorised knowledge, HotpotQA on multi-hop reasoning, PopQA on long-tail entities outside parametric memory, and Natural Questions on the ambiguity of real user queries. Because these question types produce different kinds of wrong answer, the progressively layered policies do not all earn their improvement on the same dataset; each carries a different part of the suite. An evaluation reporting only the aggregate would have seen a single improvement and missed this structure.

The chapter proceeds as follows. It first sets out the two objects of evaluation, continuation behaviour and controller quality, and the distinction between them (Section 5.1). It then reports the case study and the typology of continuation outcomes that motivates the stopping decision (Section 5.2). It describes the replay suite that holds the policy comparison fair (Section 5.3), then reports the aggregate policy comparison and locates where the improvement comes from (Section 5.4). It disaggregates that improvement by dataset, which attributes the gain to each policy (Section 5.5), before the statistical robustness of the findings is examined and the chapter is summarised.

5.1 Evaluative framing

The chapter contains two analyses, addressing two related but separate objects of evaluation: *continuation behaviour* and *controller quality*.

Continuation behaviour describes what happens to an answer when the CoC system proceeds from the first clarification round to the second. The case study characterises this behaviour by comparing first round and second round answer states.

Controller quality concerns whether a stopping policy makes the appropriate stop or continue decision at the first clarification boundary, given the saved first round state alone. The offline replay evaluation isolates controller quality by applying each policy to the same saved states and comparing its decision against the desired action defined previously.

The distinction matters because controller quality turns on the boundary decision, not on whether the continuation that follows succeeds. A continue decision on a wrong answer is the desired action even if the next round fails to repair it, and a stop decision on a correct answer is the desired action even if a further round would have preserved it. This lens applies throughout the controller results.

5.2 Case study results: the effect of a second clarification round

The case study addresses a question on which the rest of the chapter depends: *when the CoC system continues past the first clarification boundary, what does the second round do to the provisional answer?* Two simple answers would make the stopping problem trivial. If continuation reliably repaired wrong answers, the system should never stop at the first round. If continuation reliably preserved correct ones, it should always stop there. The trajectories examined below show that neither holds.

The analysis covers 26 saved continuation trajectories from the Natural Questions continuation set described in Section 4.3. Each trajectory preserves both a first round and a second round answer state, so changes in answer correctness across the first clarification boundary can be read directly from the saved record. Round level correctness was assigned using the manual judging procedure of Section 4.3.3. The full example transition table is reported in Appendix C.1.3.

The trajectories are not independent questions. They come from six underlying question families, each corresponding to a single original Natural Questions item. Table 5.1 introduces these families under the short identifiers F1 to F6, which are used throughout the section. Since the trajectories within a family share the same underlying question, any variation in second round behaviour within a family reflects the dynamics of that trajectory rather than a different question being asked.

ID	Short label	Original Natural Questions item	Trajectories
F1	Chris Cornell lyric	“chris cornell she’s going to change the world”	3
F2	Carthage leader	“leader of carthage in the first punic war”	6
F3	Pull tabs on beer cans	“when did they stop making pull tabs on beer cans”	5
F4	Person who jumped from space	“who is the guy that jumped from space”	6
F5	Evolution proposed in 1859	“who proposed evolution in 1859 as the basis of biological development”	1
F6	Singer of “A Thousand Years”	“who sings ive loved you for a thousand years”	5
Total			26

Table 5.1 – The six question families that produced the case study trajectories, with short identifiers used in the rest of the section.

For each trajectory, the second clarification round produced one of four possible outcomes, defined by the correctness of the first and second round answers. Table 5.2 reports the aggregate counts.

Transition category	Count	Share	Interpretation
Preservation	10	38%	Continuation preserved an already correct answer
Repair	7	27%	Continuation repaired an initially wrong or unresolved answer
Persistent Failure	7	27%	Continuation failed to repair an incorrect answer
Degradation	2	8%	Continuation degraded an initially correct answer
Total	26	100%	

Table 5.2 – Aggregate transition category counts across the case study trajectories.

Of the 26 trajectories, 17 ended correct. This overall count presents continuation as beneficial.

However when examined closer, the trajectories show a different story. Most of the examples were categorised as preservation, not repair: in 10 of the 17, the second round held onto an answer the first round had already produced correctly, and only 7 turned a wrong or unresolved answer into a correct one. Repair occurs in exactly as many trajectories as outright persistent failure, also 7. Consequently, the favourable count is carried by cases in which the second round changed nothing about correctness.

Conditioning on the first round state turns this into the asymmetry that defines the stopping problem. Of the 12 trajectories whose first round was already correct, 10 stayed correct and 2 degraded. Of the 14 whose first round was wrong or unresolved, 7 were repaired and 7 were not. The two rates point in opposite directions for a controller: continuation rarely harms a correct answer, so stopping early on a correct answer forfeits little, but continuation repairs only half of the wrong ones, so continuing is no guarantee of recovery. Neither a default stop nor a default continue is safe, which is what makes the boundary a decision rather than a fixed rule.

These counts describe what happened at the endpoints of the trajectories, but do not show what the second clarification round did between them. The subsections that follow examine that intermediate behaviour and show that the categories arise from qualitatively distinct continuation mechanisms.

5.2.1 Trajectories of agentic continuation states

The transition counts in Table 5.2 record what happened to each answer’s correctness, but not the effect of continuing clarification. This subsection recovers the mechanism behind each of the four categories through worked examples, drawn from the families in Table 5.1 and shown as a two round transcript of the CoC pipeline. The trajectories are grouped by whether the second round changed the correctness label: repair and degradation flip it, preservation and persistent failure leave it unchanged.

5.2.1.1 Changed round outcomes

Figure 5.1 reports the two trajectories in which the second round *flipped* the correctness label. The left panel shows answer *repair* in F4 (Person who jumped from space). The right panel shows answer *degradation* in F3 (Pull tabs on beer cans).

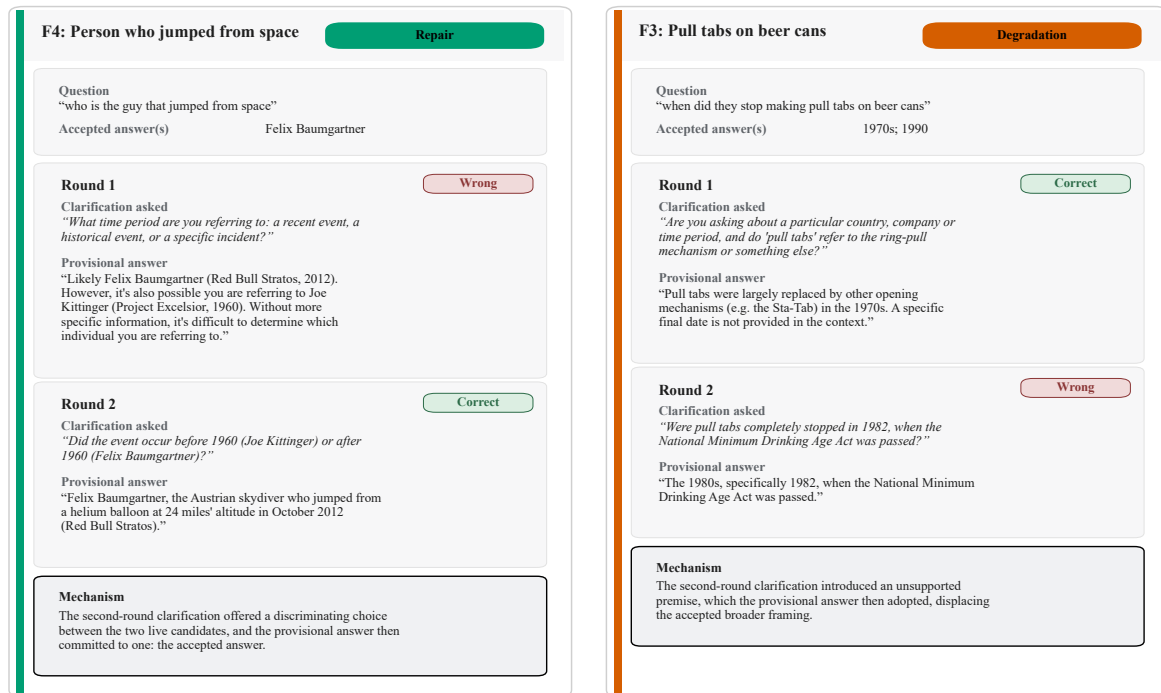


Figure 5.1 – Agentic system states over two rounds of AgenticLU’s Chain of Clarifications (CoC) for changed round outcomes.

The text shown is the condensed generations from the agentic system; the full generations are reported in Appendix C.1.4.

F4: repair by resolving an existing ambiguity. In F4, the first round answer left two live candidates unresolved. Both Felix Baumgartner and Joe Kittinger were named, the answer noted the difficulty of choosing between them, and committed to neither. Under the judging rubric, this satisfies the rule that “the answer leaves multiple live candidates unresolved” and is judged wrong. The second round clarification then asked a discriminating question between the two candidates the first round had already produced (“before 1960 or after 1960?”), and the second round provisional answer committed to one of them: the accepted answer. The work of the second round was clear: it did not retrieve new evidence or reconsider the entity space; it forced a commitment within a candidate set the first round had already identified but failed to collapse. This is the mechanism the methodology forecasts for repair, in which the second round “resolves ambiguity between competing candidates”.

F3: degradation by overspecification. The F3 trajectory inverts the relationship between the clarification question and the answer. The first round answer was a broader 1970s claim that the judging rubric accepted under gold string containment, since “1970s” appears as an accepted answer. The second round clarification then introduced a specific date (1982) tied to the National Minimum Drinking Age Act, which is unrelated to beer can opening

mechanisms and is not supported anywhere in the pinned context. The provisional answer absorbed both the date and the unsupported premise wholesale, producing a final answer that does not contain either correct answer and rests on a fabricated causal link. The mechanism is overspecification rather than random degradation: the clarification question moved the answer from an approximate but acceptable framing to a precise but fabricated one. This is the mechanism the methodology forecasts for degradation, in which the second round “overfits to a spurious detail”. It is also a concrete instance of a broader pattern, in which further reasoning steps entrench or introduce errors that earlier steps did not contain [21].

Across these two trajectories, the same two round process produced opposite outcomes. The difference lies entirely in the work the second round clarification question did. A discriminating question over named candidates converted an unresolved first round answer into the accepted one; a question carrying an unsupported premise converted an acceptable first round framing into a fabricated specific. The direction of an outcome changing continuation is set by the clarification question, not by the act of continuing.

5.2.1.2 Preserved round outcomes

Figure 5.2 reports the two trajectories in which the correctness label was preserved. The left panel shows F5 (Evolution proposed in 1859), where preservation kept a correct answer correct. The right panel shows F1 (Chris Cornell lyric), where persistent failure kept a wrong answer wrong, even though the surface text of the answer changed.

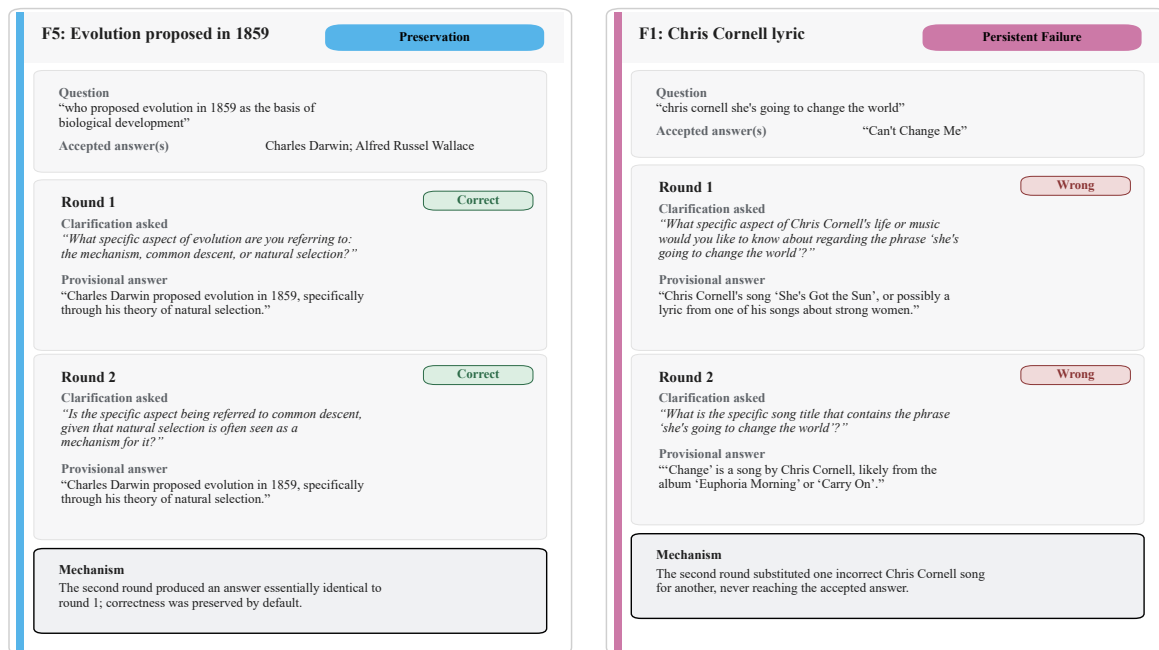


Figure 5.2 – Agentic system states over two rounds of AgenticLU’s Chain of Clarifications (CoC) for the preserved round outcomes.

The text shown is the condensed generations from the agentic system; the full generations are reported in Appendix C.1.4.

F5: preservation by literal repetition. The F5 first round answer committed to Charles Darwin, one of the two accepted gold answers, and was judged correct under the rule that “The answer commits to an accepted gold answer”. The second round clarification asked a meta question about which aspect of evolution was being referred to, and the second round provisional answer repeated the first round verbatim, ignoring the clarification entirely. Correctness was held by default, not by any second round work. This is preservation in its purest form: the clarification step produced nothing the answer state required, and stopping and continuing are observationally identical at the boundary.

F1: persistent failure on a wrong evidence path. The F1 trajectory differs from F5 in that the second round answer is not a verbatim repeat. The first round named the song “She’s Got the Sun”, matching neither the accepted answer (“Can’t Change Me”) nor any documented Chris Cornell title in the available context. The second round clarification asked the model to identify the specific song title, and the second round answer substituted “Change”, again neither the accepted answer nor a documented title. The surface text changed, but the underlying failure did not: both rounds stayed on the same wrong evidence path, attempting to identify a Chris Cornell song from a lyric fragment with no retrieval evidence supporting any specific title. This is the mechanism the methodology forecasts for persistent failure, in

which the second round “refines the wording of an incorrect answer without changing the underlying evidence path”. Persistent failure of this kind is distinct from preservation: the system does visible work between rounds, yet that work never escapes the failure mode.

Across these two trajectories, the second round contributed nothing of its own to the correctness of the final answer. In F5 it produced a repeat; in F1 it produced a different wrong answer on the same wrong evidence path. The value of a preserving continuation comes entirely from the first round state it inherited: safe when that state is already correct, a missed opportunity when it is not. In neither case does the second round itself supply the value.

What the four trajectories establish. Across all four trajectories, the second round introduced no new evidence and did not improve the answer on its own. Where the correctness label changed, the direction was set by the premise the clarification question carried; where it was preserved, the outcome was already fixed by the first round commitment the second round inherited. In every case the determining factor was present in the first round state before the second round ran: what the first round committed to, and what the clarification question then did with it. This is what makes the boundary a tractable decision rather than a gamble on the second round, and it is the regularity the rest of the chapter builds on.

5.2.2 Family distribution of the continuation mechanisms

The four trajectories of Section 5.2.1 establish that repair, degradation, preservation, and persistent failure are distinct mechanisms, not four labels on one underlying process. The question this subsection takes up is whether they distribute randomly across the trajectories or concentrate by question. Figure 5.3 reports the mechanism breakdown within each family.

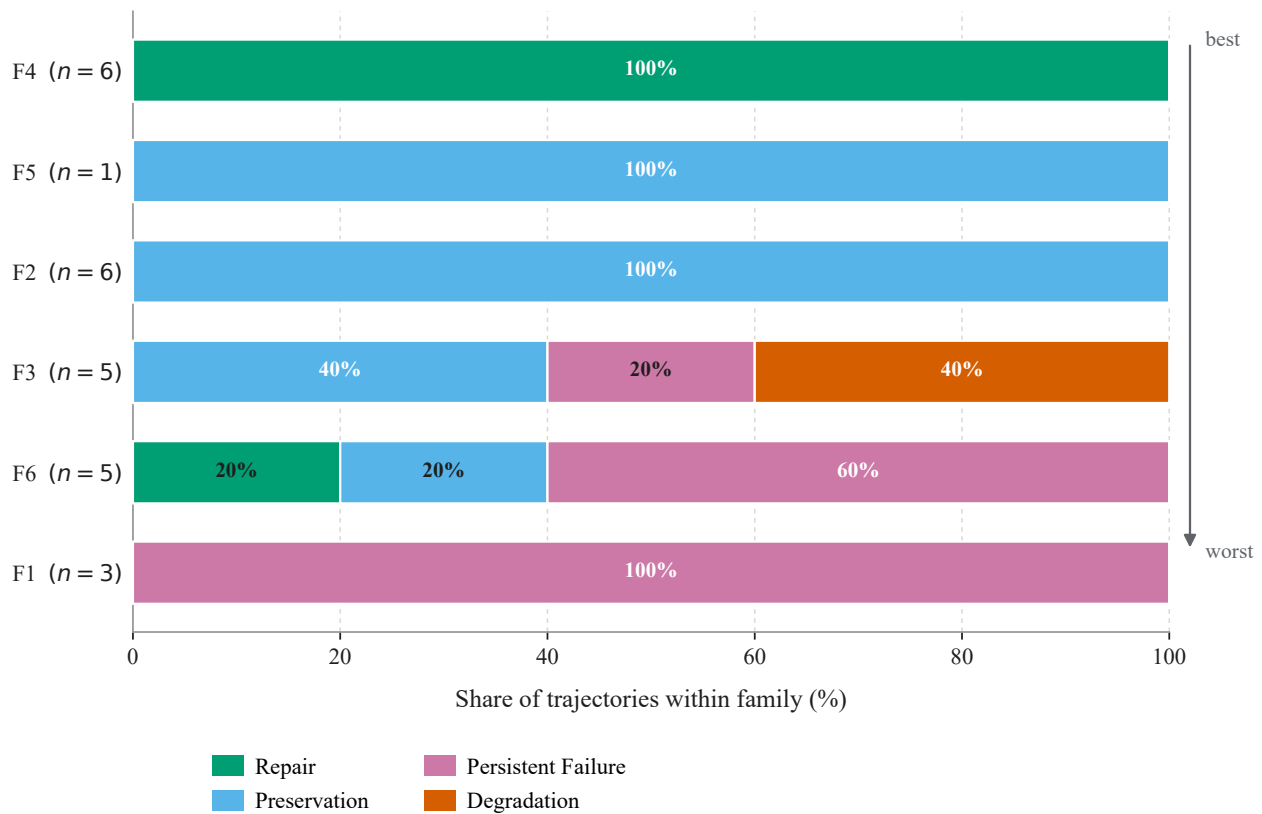


Figure 5.3 – Distribution of the four continuation mechanisms across the six question families.

Note: Families are ordered from worst continuation outcome (F1, bottom) to best (F4, top).

The mechanisms do not distribute randomly across the families. Four of the six families are homogeneous, with every trajectory falling into a single mechanism; the remaining two, F3 and F6, contain a mix. This division is itself informative, as the two groups support different inferences. The homogeneous families establish that the second round’s behaviour is determined by the saved first round state, while the mixed families isolate which property of that state determines it.

In the homogeneous families, the same mechanism recurs regardless of context variant. F4’s six trajectories all produce a first round answer naming both Felix Baumgartner and Joe Kittinger as live candidates, and the second round commits to Felix Baumgartner every time. F2’s six trajectories all commit to Hanno the Great, an accepted gold answer, and the second round preserves that commitment with only surface rewording. F1’s three trajectories all name a Chris Cornell song outside the gold set, and the second round substitutes a different out-of-set title each time, never reaching the accepted answer. The full trajectory records for every family are reported in Appendix C.1.4. Where the saved first round state is essentially constant across variants, the second round’s behaviour follows from it deterministically.

The mixed families are the more informative, because their internal variation isolates the

state property the second round is sensitive to. In F3 the split is on the first round answer’s specificity: where the first round committed clearly to a 1970s phase out, the second round preserved it; where the first round hedged between the 1970s and 1980s, or noted that no specific date was supported, the second round drifted into an unsupported 1980s claim. F6 shows the pattern more cleanly: where the first round named Christina Perri, the second round preserved that commitment and ended correct; where it named Celine Dion or Sarah McLachlan, the second round retained the wrong candidate and ended wrong. Across both families the discriminating property is whether the first round answer text contains the accepted entity at all, not whether it was confident, fluent, or grammatically committed. This is the dissociation between surface fluency and evidential grounding documented in the hallucination literature [13], surfacing here as a concrete property of the saved state: fluency is uninformative about whether the gold entity was named, and only the latter predicts what the second round can do. Where the first round named the gold candidate, the second round had a path to it; where it did not, the second round had no path to construct one.

The two degradation trajectories, both in F3, follow the same logic in the opposite direction. Each began from a first round answer carrying an acceptable broader framing (“the 1970s”) without a specific date, and the second round clarification injected a specific date that displaced it. Two trajectories are no basis for a general rate of degradation, but they identify the configuration that produces it: a noncommittal first round answer met by a clarification question that introduces a specific premise, the same configuration behind F3’s other unfavourable outcomes.

Taken together, the distribution points to a single organising property. Which mechanism a trajectory falls into is governed by what the first round answer commits to relative to the gold candidate: if the candidate is named, and if so whether the answer commits to it cleanly, leaves it among competitors, or buries it in a framing a later premise can displace. That property lives in the saved first round state, the one thing a controller can read at the boundary, and not in the second round it is deciding whether to run.

5.2.3 What the case study establishes for the controller evaluation

The investigation relocates the stopping decision. If the mechanism is fixed by the first round state, then deciding at the boundary is a judgement over evidence the boundary already contains. Such a judgement involves considering whether a further round will help, harm, repeat or fail. As a result, STARC focuses on the boundary state.

5.3 The offline replay suite

The controller evaluation operates on a fixed offline replay suite. Running the instrumented Stage 1 pipeline (Section 4.2) over the four source benchmarks produced the 408 example suite: each replay example is the saved first round CoC state from one (base question, context variant) pair, retrieved from storage rather than regenerated. Every policy receives the same saved state for the same example, so any difference in policy output is attributable to the stopping logic rather than run variation of the underlying clarification system [54].

Dataset	Base questions	Variants	Replay examples	First round correct	First round wrong
Natural Questions	17	6	102	67	35
PopQA	17	6	102	70	32
TriviaQA	17	6	102	91	11
HotpotQA	34	3	102	77	25
Combined	85	mixed	408	305	103

Table 5.3 – Offline replay suite composition used for stopping gate evaluation.

5.4 Main stopping gate policy results

STARC was designed under the core idea that the continuation decision is not a single step answer confidence test. Rather it is a layered risk analysis of the system state at the first round boundary. The policies were applied to the 408 example replay suite and their decisions produced were scored against the desired action: to stop when the first round answer is correct and continue when it is wrong. The decision correctness falls into the four confusion matrix cells: **safe stop**, **justified continuation**, **premature stop**, and **unnecessary continuation**, and is evaluated using class balanced metrics.

5.4.1 Aggregate comparison

Tables 5.4 and 5.5 report the outcome cells and the summary metrics for each policy across the full suite.¹

¹Full metric and confusion tables are provided in Appendix C.2.

Policy	Safe stop	Justified continuation	Premature stop	Unnecessary continuation
VRB	267	46	57	38
VRRS	248	53	50	57
VRRS-BR	247	69	34	58

Table 5.4 – Stopping gate outcome counts by policy.

Policy	Gate success	Continue recall	Balanced accuracy	MCC	Predicted continue
VRB	76.72%	44.66%	66.10%	0.346	84
VRRS	73.77%	51.46%	66.38%	0.321	110
VRRS-BR	77.45%	66.99%	73.99%	0.450	127

Table 5.5 – Aggregate stopping gate metrics by policy.

Note: The qualitative decision evidence underlying these outcome cells is reported in Appendix C.3, which records representative policy outputs and failure mode evidence for each outcome type.

The headline of Table 5.5 is the gap between gate success and the class balanced metrics. Gate success is nearly flat across the three policies (76.72%, 73.77%, 77.45%), while the class balanced metrics move sharply over the same rows: continue recall rises from 44.66% to 66.99%, balanced accuracy from 66.10% to 73.99%, and MCC from 0.346 to 0.450. The two lenses are not measuring the same thing. Gate success is dominated by the majority class, on which all three policies behave alike; the class balanced metrics weight the minority class equally and thus register the differences that fall on the wrong first round answers. The policies differ little in how often they decide correctly overall, but substantially in which class their correct decisions land on. That redistribution, from the majority class to the minority one, is the effect the rest of the section measures.

Under the class balanced lens, VRB is conservative: it accepts 87.5% of correct first round answers (267 cases) but challenges only 44.7% of wrong ones (46 cases). Thus, more than half of the wrong first round answers in the suite are accepted prematurely. VRRS-BR is risk sensitive: it accepts almost as many correct answers (80.98%, 247 cases) yet challenges 67.0% of wrong ones (69 cases). The trade off across VRB $\rightarrow$ VRRS-BR is concentrated on the safety side: the final policy makes 23 fewer of the more consequential errors (**premature stop**) at a cost of 20 additional less consequential ones (**unnecessary continuation**).

VRRS sits between the two. Continue recall (51.46%) is higher than VRB, suggesting added sensitivity to unsafe first round states. Gate success (73.77%), balanced accuracy (66.38%),

and MCC (0.321) are at or below VRB’s, indicating that the added sensitivity does not improve overall class balanced performance. The mechanism behind this divergence is visible in the outcome cells. **unnecessary continuation** rose by 19 across the VRB $\rightarrow$ VRRS transition, while **premature stop** fell by only 7. As such, the deterministic risk signals moved nearly three already correct answers into continuation for every wrong answer detection they recovered.

The lack of class balanced movement between VRB and VRRS carries a substantive reading. The deterministic risk signals added in VRRS are designed from a set of known QA failures, established in the literature and further supported by the case study. These include hedge tokens, source absence patterns, multi candidate enumeration, and other families introduced previously. VRB carries no such rules. Its evidence comes from a structured LLM verifier and falsifier call. If VRRS’s explicit signals were detecting failure modes that VRB’s verifier and falsifier missed, the class balanced metrics would move in VRRS’s favour. They do not. The most parsimonious reading is that the LLM based evidence in VRB is already capturing the same failure patterns the explicit signals were built to identify. The deterministic stage layered on top of it therefore adds sensitivity without adding new information. As such, the next question is whether a different addition, one that brings new information, can convert the redistribution into an improvement.

5.4.2 Where the improvement comes from

The aggregate result decomposes into the marginal effect of each added layer, since each policy extends the previous one. Two outcome cells capture the trade off most directly. **premature stop** is the principal safety failure, in which the controller accepts an answer that should have been challenged. **unnecessary continuation** is the principal efficiency cost, in which the controller continues despite the answer already being correct. Success for a policy layer means reducing **premature stop** without a disproportionate rise in **unnecessary continuation**.

Figure 5.4 positions the three policies in the corresponding two dimensional trade off space. The horizontal axis is stop recall, the rate of **safe stop** among correct first round answers. The vertical axis is continue recall, the rate of **justified continuation** among wrong first round answers. A policy that moves up and to the right improves on both axes simultaneously. Movement up and to the left trades stop recall for continue recall.

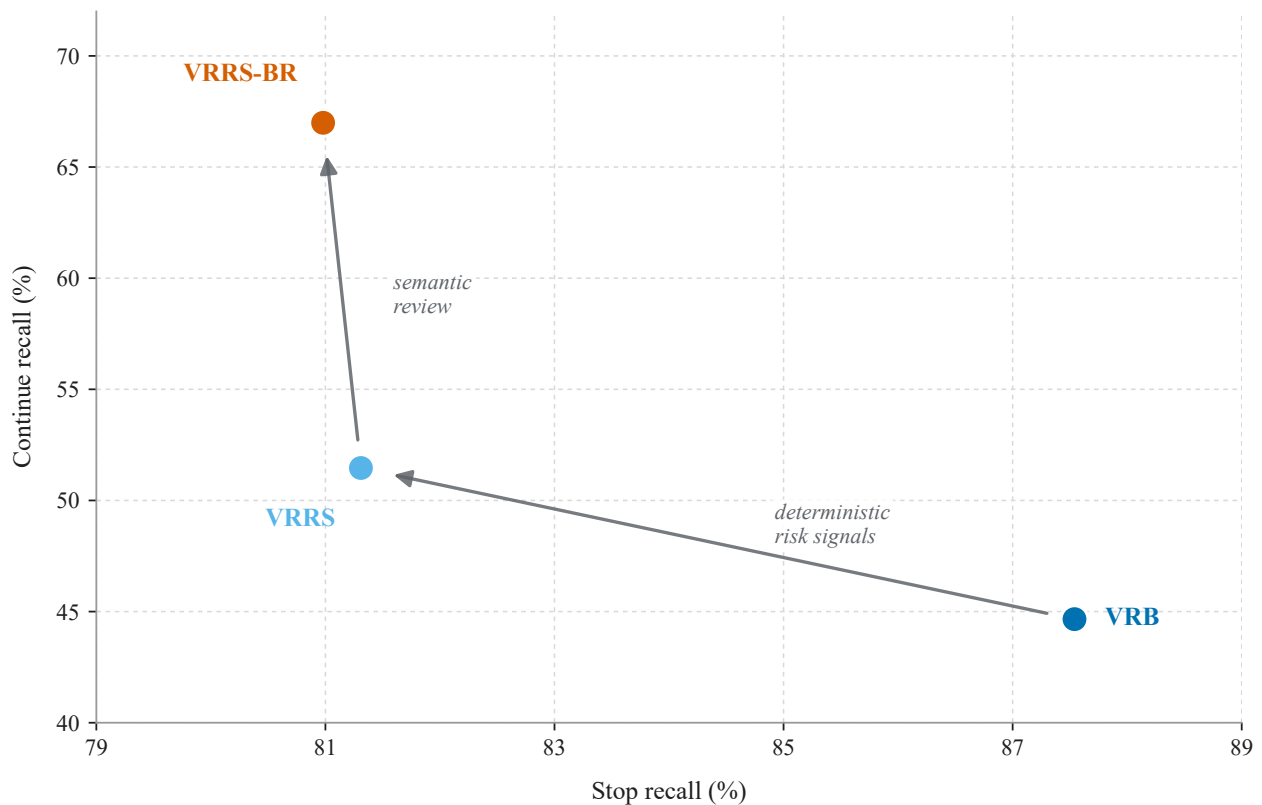


Figure 5.4 – Controller trade off profile across the combined replay suite.

Note: Stop recall measures the rate of **safe stop** among correct answers; continue recall measures the rate of **justified continuation** among wrong answers.

Table 5.6 gives the numerical movements behind these positions.

Policy transition	Premature stop	Unnecessary continue	Continue recall	Balanced accuracy	MCC
VRB → VRRS	57 → 50	38 → 57	44.66% → 51.46%	66.10% → 66.38%	0.346 → 0.321
VRRS → VRRS-BR	50 → 34	57 → 58	51.46% → 66.99%	66.38% → 73.99%	0.321 → 0.450
VRB → VRRS-BR	57 → 34	38 → 58	44.66% → 66.99%	66.10% → 73.99%	0.346 → 0.450

Table 5.6 – Incremental effect of each added policy layer.

Figure 5.4 traces two transitions across the trade off space, and the geometry of each is different. The VRB → VRRS transition moves the policy upwards and leftwards: continue recall rises while stop recall falls. The leftwards movement is the cost of the upwards movement, and the class balanced metrics confirm the reading. Balanced accuracy moved by only

0.28 percentage points and MCC declined from 0.346 to 0.321. VRRS on its own remains a redistribution rather than an improvement.

The VRRS $\rightarrow$ VRRS-BR transition has a different geometry. Continue recall rises sharply, while stop recall barely changes. The point moves almost vertically in the trade off space, indicating that the added layer improved wrong answer detection without expanding continuation across the rest of the suite. **premature stop** fell by 16 while **unnecessary continuation** rose by only 1. This 16:1 net effect ratio is the central pattern of the layered design, and the mechanism that produces it is examined in the next subsection.

Across the full VRB $\rightarrow$ VRRS-BR stack, the cost side of the trade off (**unnecessary continuation** increased by 20) was incurred almost entirely at the deterministic stage. The safety side (**premature stop**: fell by 23) was almost entirely the work of the borderline review stage. Thus, the two layers play different roles that are not interchangeable: the deterministic layer produces the set of uncertain cases that the borderline layer can act on, and the borderline layer converts that set into the wrong answer detection gain that defines the final policy.

5.4.3 The borderline review mechanism

The wrong answer detection gain in the VRRS $\rightarrow$ VRRS-BR transition comes from a pipeline that converts a subset of VRRS’s borderline cases into continue decisions. Figure 5.5 reports its top line operation over the 408 example replay suite.

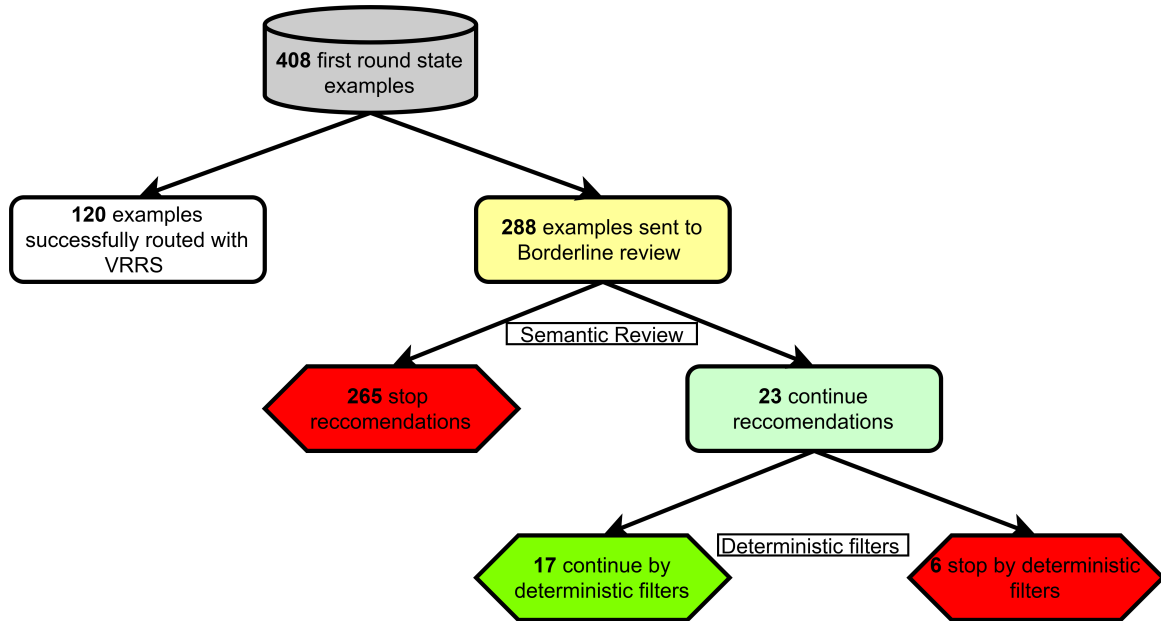


Figure 5.5 – The borderline review pipeline applied to the replay suite.

VRRS routes 120 of 408 cases by deterministic stop or continue band and forwards the remaining 288 to semantic review. The reviewer returns 23 raw continue recommendations and 265 raw stop recommendations. The final continuation filter passes 17 of the 23 continue recommendations and reverts the other 6 to stop. The complete output tables are in Appendix C.2.4.

Two features of this pipeline matter for interpreting the final policy. The first is the gating effect of the deterministic layer. The reviewer operated on 70.6% of the suite, but only on cases the deterministic layer could not resolve confidently. This is the design intent of the layered policy: deterministic risk signals are responsible for routing rather than for resolving more challenging questions, and the borderline band is the set of cases the review stage exists to act on.

The second is the conservatism of the filter. The reviewer returned 23 raw continue recommendations from the borderline cases, and the final continuation filter accepted only 17 (4.2% of the suite) while reverting the other 6 to stop. Of the 17 accepted continuations, 16 corresponded to wrong first round answers and 1 to a correct one, a ratio of 16:1 in favour of beneficial changes. The selectivity is visible inside the borderline band itself. Of the 238 correct first round answers in the band, the filter permitted continuation on only 1 (0.4%). Of the 50 wrong first round answers in the band, the filter caught 16 (32.0%). The filter is extremely conservative on correct answers while still recovering roughly a third of the wrong answers within its operational range.

These two features explain why the VRRS $\rightarrow$ VRRS-BR transition produced a 16:1 net effect rather than a uniform shift toward continuation. The deterministic layer first restricts review to the cases it cannot resolve. The filter then restricts the reviewer’s continuation

recommendations to those that satisfy concrete error conditions. Whether the same pattern holds across all four source datasets, and which of the two added layers carried the work on each dataset, is examined next.

5.5 Disaggregated policy behaviour

The combined comparison established VRRS-BR as the strongest controller across the full suite, but reported a single aggregate result. Whether the advantage holds across the four source datasets cannot be read from it. The disaggregation matters for two reasons. Firstly, the four datasets carry different class imbalances. TriviaQA contains only 10.8% wrong first round answers, Natural Questions contains 34.3%, HotpotQA 24.5% and PopQA 31.4%. As such, continue recall is a fine grained statistic on Natural Questions (each detected wrong answer moves it by 2.9 percentage points) and a coarse one on TriviaQA (each moves it by 9.1 percentage points). Secondly, the dataset disaggregation isolates the contribution of each added policy layer dataset by dataset. That contribution structure is the central object of the analysis that follows.

The headline finding is that the combined improvement was not evenly distributed across datasets, and the two added policy layers did not contribute in the same way. VRRS-BR produced its clearest gains on TriviaQA and HotpotQA, a more costly trade off on Natural Questions, and limited marginal benefit on PopQA. The layer that carried the improvement also differed dataset by dataset. The first subsection presents the dataset numbers. The second resolves the cross cutting question of which layer carried each dataset.

5.5.1 Dataset results

Figure 5.6 reports continue recall by dataset and policy. Continue recall is the safety side measure of the stopping problem: the proportion of wrong first round answers correctly sent to continuation. The full dataset level confusion counts and metric tables are in Appendix C.2.3.

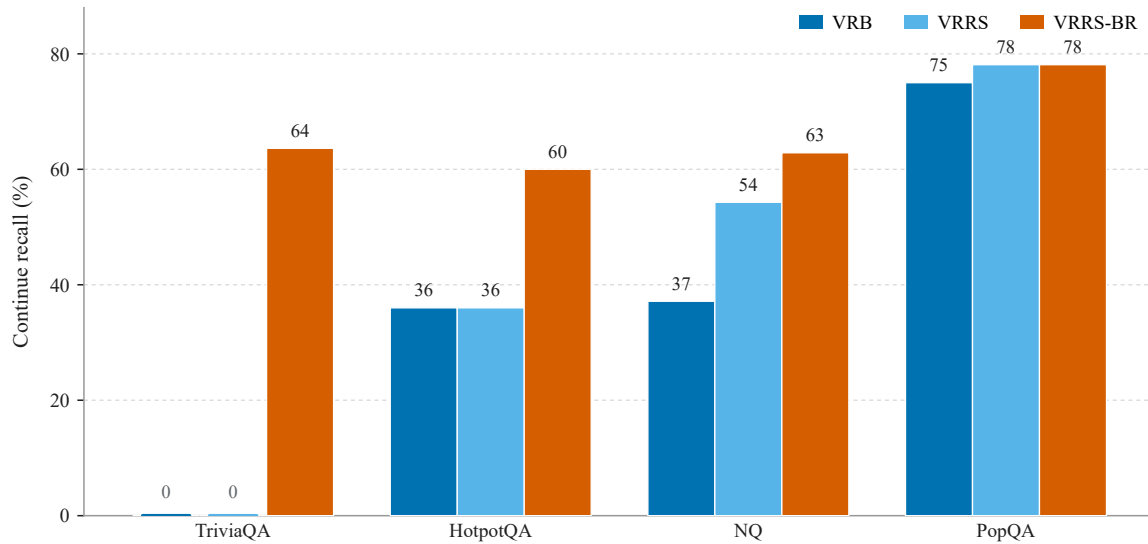


Figure 5.6 – Continue recall by dataset and policy.

Note. Continue recall measures the share of wrong first round answers correctly sent to continuation. Higher values indicate stronger detection of unsafe first round answer states.

VRRS-BR was the highest recall policy on all four datasets, but the magnitude and source of its advantage varied. The four datasets are taken in order of how cleanly the final policy improved on the earlier ones, beginning with TriviaQA where the improvement was most striking and ending with PopQA where the earlier policies were already close to the final result.

Dataset	Policy	Balanced accuracy	MCC	Continue recall
HotpotQA	VRB	60.86%	0.235	36.00%
HotpotQA	VRRS	60.86%	0.235	36.00%
HotpotQA	VRRS-BR	72.21%	0.433	60.00%
NQ	VRB	63.35%	0.319	37.14%
NQ	VRRS	62.96%	0.255	54.29%
NQ	VRRS-BR	67.25%	0.334	62.86%
PopQA	VRB	78.93%	0.562	75.00%
PopQA	VRRS	77.63%	0.523	78.12%
PopQA	VRRS-BR	77.63%	0.523	78.12%
TriviaQA	VRB	45.60%	-0.101	0.00%
TriviaQA	VRRS	43.96%	-0.121	0.00%
TriviaQA	VRRS-BR	75.77%	0.419	63.64%

Table 5.7 – Per dataset controller performance.

Balanced accuracy, MCC and continue recall by policy. Full confusion counts and the complete metric set are in Appendix C.2.3.

TriviaQA showed the most dramatic change. Under both VRB and VRRS, no wrong TriviaQA examples were sent to continuation: continue recall was 0% under both policies, so of the 11 wrong first round answers the verifier/ falsifier and deterministic machinery caught none. VRRS-BR then converted 7 of those 11 into justified continuation outcomes, raising continue recall to 63.64% and balanced accuracy from 45.60% to 75.77%. The MCC must be read with care at this scale: when a controller produces no justified continuations its MCC numerator reduces to the negative product of its two error counts and is bounded at or below zero by construction, so the negative values under VRB (-0.101) and VRRS (-0.121) record the absence of detection together with a few unnecessary continuations rather than a controller inverted against the desired action, and the marginally lower VRRS value reflects only its three extra unnecessary continuations. VRRS-BR is the first policy here to register genuine detection, with an MCC of +0.419 supported by actual true positives rather than bounded by their absence. The final layer therefore introduced wrong answer detection where the earlier policies had none.

The reason the earlier policies register no detection lies in what their components do with the saved first round state. A look at the wrong TriviaQA rows suggested the evidence needed to overturn the provisional answer was generally present in the state the judges read, so the

failure is better explained by how that state was assessed than by what it lacked. Both the verifier and falsifier of VRB and the deterministic risk signals of VRRS assess the provisional answer by inspecting the system state: the verifier evaluates the case for stopping from the answer’s apparent support and completeness, the falsifier tests that same answer for signs that it is false or that a competitor fits better, and the deterministic signals look for surface markers such as hedging or multiple enumerated candidates. Each of these is, in effect, a reading of the answer the round already produced. TriviaQA’s questions are the pre authored single entity recall form characterised in Section 3.4, and a wrong answer to such a question is typically not hedged or self doubting but fluent, internally consistent, and committed to one entity. Presented with an answer of that shape, the verifier sees support and completeness, the falsifier finds no incoherence to flag, and the deterministic signals find none of the surface uncertainty they key on. Furthermore, these LLM calls focus, not only on the correctness of an answer alone, but also an assessment of the system state. The case for stopping that the components assemble is therefore clean, and the answer is accepted, even though it is wrong. This is the dissociation between fluency and grounding established in Section 5.2.2, now acting on the judges rather than on a later round: coherence is uninformative about correctness, yet coherence is most of what an assessment of the existing answer has to read.

The borderline reviewer breaks this pattern in TriviaQA as it focuses on a different assessment of the same answer. The earlier components assess the provisional answer holistically. In contrast, the reviewer is constrained to a narrower, adversarial task: rather than focusing on judging whether the answer looks adequate, it is instructed to continue only if it can point to a concrete, identifiable error in it, such as a wrong entity, a contradiction, an answer type mismatch, or a competing answer that fits the question better, and to treat mere under grounding as insufficient. The two stages read the same answer but look for different things: the earlier components weigh whether the answer is well supported, while the reviewer hunts for a specific defect. That difference in task is what the TriviaQA result isolates. A wrong single entity answer of the kind TriviaQA produces is coherent enough to present a clean case for stopping, so it survives the holistic assessment; but its error is exactly the concrete kind, a wrong entity, that the reviewer is built to name. The failure of VRRS and the success of VRRS-BR on the same rows is therefore not a difference in how hard each looks, but in what each is looking for. With only 11 wrong examples in TriviaQA’s 102, this is read as an account of the components’ roles and provides some insight into the role of each policy.

HotpotQA showed a similar but cleaner effect from the final layer. Both VRB and VRRS left 16 of the 25 wrong examples as **premature stop** outcomes. VRRS-BR reduced **premature stop** and increased **justified continuation**. Continue recall rose to 60.00%, and balanced accuracy rose to 72.21%. The cost on correct answers was minimal: **unnecessary continuation** increased only from 11 to 12. Therefore, HotpotQA provides the clearest case where the final

policy improved wrong answer detection without a substantial offsetting cost on correct answers.

Natural Questions showed the trade off most fully developed. The movement toward continuation began earlier in the policy stack here than on any other dataset. VRRS improved continue recall from 37.14% to 54.29% relative to VRB, and VRRS-BR extended this further to 62.86%, reducing **premature stop** from 22 under VRB to 13. The improvement carried the largest continuation cost of any dataset: **unnecessary continuation** rose from 7 under VRB to 19 under VRRS-BR. The cost dominated raw gate success, which fell from 71.57% to 68.63%, even as balanced accuracy rose from 63.35% to 67.25%. Natural Questions therefore shows the cost side of the progressive design most clearly: the increased risk sensitivity detected more unsafe states, but also continued more answers that were already correct.

PopQA showed the smallest marginal change across the policy stack. VRB already detected 24 of the 32 wrong examples on this dataset, giving a baseline continue recall of 75.00% (higher than VRRS-BR’s aggregate continue recall on the full suite). VRRS and VRRS-BR both raised this slightly to 78.12%, but the two policies produced identical confusion cells on PopQA. Borderline review did not change a single decision on this dataset. As such, the small improvement on PopQA was entirely the work of the deterministic risk signal layer, which reduced **premature stop** from 8 to 7. The long tail factoid grounding errors found in the PopQA evidence are largely caught by the structured judges already, with the deterministic layer recovering one additional wrong answer.

5.5.2 Which layer carried the improvement

The per dataset walk pointed at a pattern that the per dataset numbers make explicit: the two added policy layers did not both fire on every dataset. Figure 5.7 reports the reduction in **premature stop** and the gain in continue recall attributable to each policy transition, by dataset.

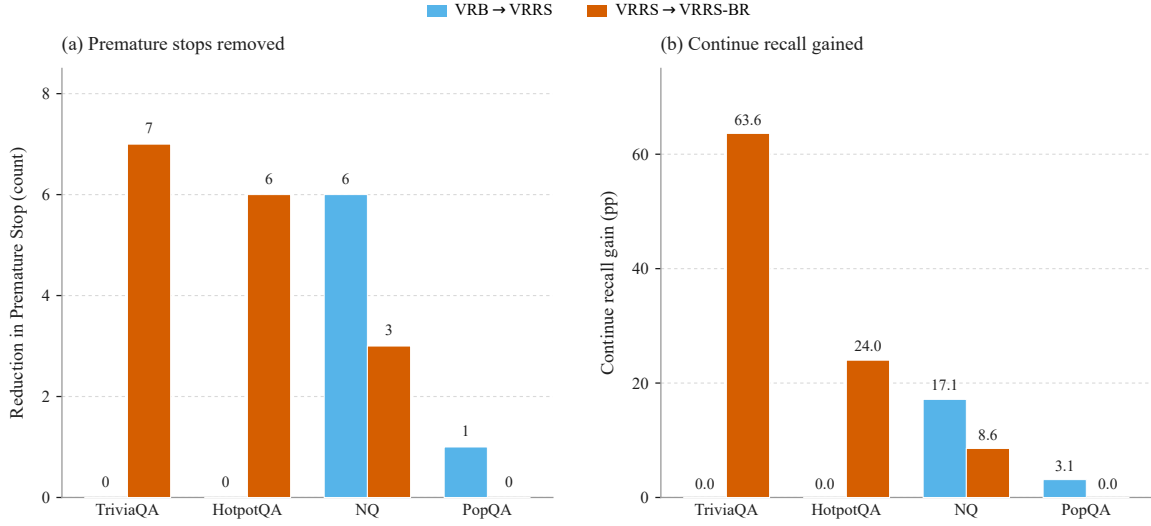


Figure 5.7 – Dataset level effect of each added policy layer.

Note: Panel (a) reports the reduction in **premature stop** for each policy transition. Panel (b) reports the corresponding gain in continue recall, measured in percentage points. The first transition isolates deterministic risk signal routing; the second isolates semantic borderline review.

The central finding of this subsection is that each added policy layer carried the improvement on a different subset of datasets, and that no single layer of the present design would have produced the combined improvement on its own. Three concrete patterns in the confusion cells are key here.

On HotpotQA, VRRS produced confusion cells identical to VRB: 66 **safe stop**, 11 **unnecessary continuation**, 16 **premature stop**, and 9 **justified continuation** under each policy. This means the deterministic risk signal layer made no change to the controller’s behaviour on this dataset. The entire HotpotQA improvement (**premature stop** down from 16 to 10, **justified continuation** up from 9 to 15) came from the borderline review layer alone. Without that layer, HotpotQA’s continue recall would have stayed at 36.00% under both VRB and VRRS.

PopQA shows the inverse pattern. VRRS-BR produced confusion cells identical to VRRS: 54 **safe stop**, 16 **unnecessary continuation**, 7 **premature stop**, and 25 **justified continuation** under each policy. Here the borderline review layer made no change to the controller’s behaviour, and PopQA’s small improvement (**premature stop** down from 8 to 7) came from the deterministic layer alone.

On TriviaQA, the two layers had asymmetric effects. VRB and VRRS each produced zero **justified continuation** outcomes, so the deterministic layer contributed nothing to wrong answer detection on this dataset. The entire TriviaQA gain (7 of 11 wrong examples sent to continuation under VRRS-BR) again came from the borderline review layer. Natural Questions is the only dataset on which both layers carried wrong answer detection: the deterministic layer raised **justified continuation** from 13 to 19, and the borderline layer raised

it further from 19 to 22.

Across the four datasets, the layered design produces a pattern in which neither layer is redundant. Removing the deterministic layer would leave PopQA’s gain unrecoverable, since borderline review made no change on that dataset. Removing the borderline review layer would leave the HotpotQA and TriviaQA gains unrecoverable, since the deterministic layer made no wrong answer detection change on either. Natural Questions confirms the complementarity: both layers contributed there, and the combined effect was larger than either alone. The combined improvement reported earlier in the chapter is therefore not the action of a single dominant layer applied uniformly across the suite. It is the combination of STARC’s layers, each operating best on different question types provided in the QA datasets.

5.6 Statistical robustness and uncertainty analysis

The preceding results were reported over the full example replay suite. However, whether that pattern remains visible when the grouped structure of the suite is taken into account is still an open question. The 408 replay examples are derived from 85 base question clusters, so examples sharing a base question may also share semantic difficulty, answer type, evidence structure, or failure mode. Treating them as fully independent would overstate the precision of the estimated policy differences. Uncertainty and paired policy differences are therefore assessed at the level of the base question cluster.²

The three analyses that follow, cluster bootstrap intervals, paired cluster comparisons, and a cluster collapsed McNemar check, are three views of the same 85 clusters rather than independent tests. A multiplicity correction across them would therefore not be meaningful, and they are reported as exploratory robustness evidence rather than confirmatory hypothesis tests. As such, in this circumstance, p -values are not useful for establishing significance at a fixed threshold. The weight of the evidence rests instead on whether the three analyses agree in direction, and on where in the policy stack any separation is concentrated. Furthermore, the modest number of clusters limits the power of any individual test, which is a further reason to weight agreement across the three analyses over any single result.

Robustness is assessed through three linked analyses. The cluster bootstrap estimates uncertainty around the principal class balanced controller metrics. The paired cluster comparisons then examine how each policy transition changed gate success within the same base question clusters. A final cluster collapsed McNemar check reduces each cluster to a single majority success outcome, providing a stricter binary comparison. Together, these analyses test

²The cluster definition is given in Section 4.1.1. The supporting cluster procedure and paired comparison objects are reported in Appendix C.4.

whether the descriptive advantage of VRRS-BR remains visible when repeated context variants are treated as grouped evidence rather than independent replay examples.

5.6.1 Cluster bootstrap intervals

Table 5.8 reports the cluster bootstrap means and 95% intervals for each policy on balanced accuracy, continue recall, and MCC.³ Under cluster aware resampling, VRRS-BR holds the highest mean on every class balanced metric, but its intervals overlap with those of VRB and VRRS.

Policy	Balanced accuracy mean	95% CI	Continue recall mean	95% CI	MCC mean	95% CI
VRB	66.14%	[58.40%, 73.93%]	44.81%	[30.12%, 60.38%]	0.345	[0.186, 0.495]
VRRS	66.24%	[58.01%, 74.46%]	51.19%	[35.71%, 66.67%]	0.318	[0.156, 0.478]
VRRS-BR	73.89%	[65.95%, 81.46%]	66.82%	[52.29%, 80.36%]	0.448	[0.295, 0.590]

Table 5.8 – Cluster bootstrap 95% intervals for the main controller metrics.

These bootstrap means differ slightly from the point estimates in Section 5.4 because they average over resamples rather than reporting the single suite value.

The intervals overlap substantially. VRB’s upper bound on balanced accuracy is 73.93%, almost exactly at VRRS-BR’s mean of 73.89%, and above VRRS-BR’s lower bound of 65.95%. The same pattern holds on continue recall (VRB upper 60.38%, VRRS-BR lower 52.29%) and on MCC (VRB upper 0.495, VRRS-BR lower 0.295). This is consistent with the cluster structure: 85 clusters is a small sample for estimating uncertainty, even with 408 replay examples drawn from them, and the cluster bootstrap is the conservative choice because it does not treat those 408 examples as independent.

Figure 5.8 shows the balanced accuracy intervals directly: VRRS-BR’s mean sits clearly above the other two, but its interval overlaps both. The cluster bootstrap therefore speaks to the stability of the ordering, not to its significance. The paired comparison in the next subsection takes a complementary view.

³The bootstrap resampled the 85 base question clusters with dataset stratification, using 4000 repetitions and random seed 42. Full procedural detail is given in Appendix C.4.

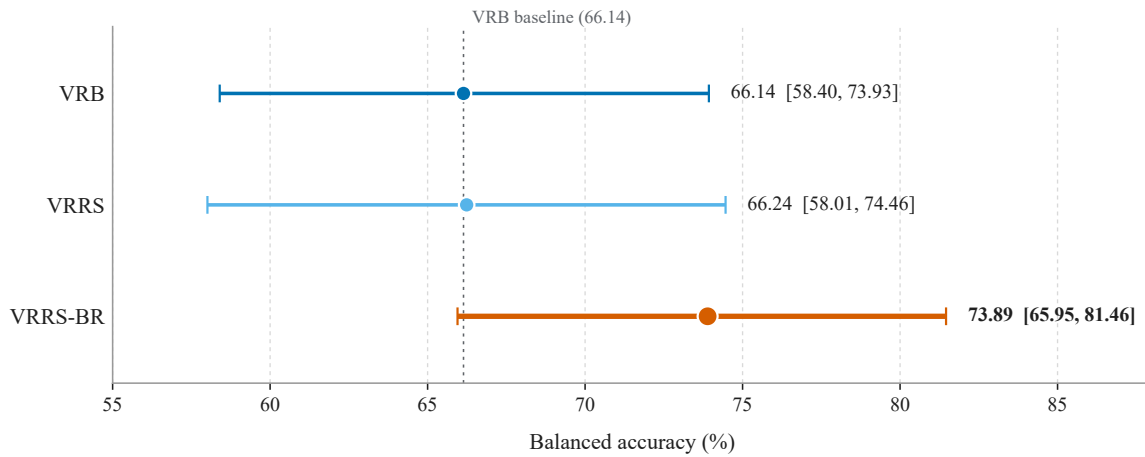


Figure 5.8 – Cluster bootstrap balanced accuracy intervals.

5.6.2 Paired cluster comparisons

The paired cluster comparison asks a key question: when one policy layer replaces the next, in how many of the 85 base question clusters does per cluster gate success rise, in how many does it fall, and how lopsided is the disagreement?⁴ Table 5.9 reports the three policy transitions.

Comparison	Δ success	Improved	Worsened	Unchanged
VRB $\rightarrow$ VRRS	-0.0235	1	7	77
VRRS $\rightarrow$ VRRS-BR	+0.0392	8	1	76
VRB $\rightarrow$ VRRS-BR	+0.0157	8	7	70

Table 5.9 – Paired cluster comparisons on gate success rate.

Δ success denotes the mean paired difference in per cluster gate success rate for the transition shown. *Improved*, *Worsened* and *Unchanged* count clusters in which the second policy raised, lowered or matched per cluster gate success relative to the first policy.

The VRRS $\rightarrow$ VRRS-BR transition is the one that isolates semantic borderline review, since the deterministic stop and continue decisions are held fixed and only borderline cases are eligible for the reviewer. It produces the clearest separation in the table: among the 9 clusters where the two policies disagree, 8 favour the reviewer and 1 does not.

The VRB $\rightarrow$ VRRS transition runs the other way: mean per cluster gate success falls by 2.35 percentage points, with 1 cluster improving and 7 worsening. The deterministic risk signal

⁴The 85 base question clusters were used as paired units, with per cluster gate success rate as the response variable.

layer raised continuation sensitivity, but, as the aggregate counts already showed, it did so by escalating roughly three already correct answers for every wrong one it recovered.

The direct VRB $\rightarrow$ VRRS-BR comparison is more balanced on paired gate success, with 8 clusters improving, 7 worsening, and 70 tied. This does not erase the aggregate advantage of the final policy; it locates it. The advantage is not a uniform rise in raw gate success across clusters but a change in the structure of the errors, with fewer wrong answers prematurely stopped and higher continue recall. Raw paired gate success therefore captures only part of the effect, which is why the class balanced metrics, not gate success, carry the comparison.

5.6.3 Cluster collapsed McNemar check

The strictest of the three views reduces each base question cluster to a single majority success outcome under each policy, and counts how many clusters change outcome between policies.⁵ Table 5.10 reports the result.

Comparison	Improved	Worsened	Tied
VRB $\rightarrow$ VRRS	1	1	7
VRRS $\rightarrow$ VRRS-BR	4	0	6
VRB $\rightarrow$ VRRS-BR	5	1	8

Table 5.10 – Cluster collapsed McNemar exact checks.

Improved and *Worsened* count clusters in which the second policy’s majority success outcome differs from the first policy’s. *Tied* counts clusters with a tied (0.5) majority outcome that were excluded from the comparison. Clusters with the same non-tied outcome under both policies are not shown, so the rows do not sum to the 85 clusters.

The collapsed view is directionally consistent with the other two but recovers less of the signal. The VRRS $\rightarrow$ VRRS-BR comparison again favours VRRS-BR, with four clusters moving from failure to success and none moving the other way; the other two transitions are close to even. The loss of signal is expected and is a property of the framing, not a contradiction of it: collapsing each cluster to a majority label discards the improvements within clusters that the bootstrap and paired comparisons both register, so a policy can improve several replay examples in a cluster without shifting that cluster’s majority outcome. The collapsed check therefore confirms the direction of the result while standing as the most conservative of the three views.

⁵This collapses variation within each cluster across context variants, so a policy that improves several replay examples within a cluster without crossing its majority success threshold registers no change. The procedure is described in Appendix C.4.

5.6.4 Robustness of band placements

The core band placements that drive the policy decisions were fixed before the replay suite was used as an evaluation set, against case study trajectories and prompt design observations only. It follows logically that an analysis must be performed to determine whether variations within the band placements meaningfully change the outcomes.

Each rule in the policy stack fires on a band label (low, moderate, or high), not on the underlying score produced by the LLM. As such, a placement that sits in the interior of its band should produce the same rule outcome as any other value in the same band. The sweep tests this directly: each of the eleven placements was perturbed across its band range in steps of 0.025, and every policy decision was re run over the 408 example suite at each step. The complete per placement sweep tables and a full grid figure showing all four confusion cells are in Appendix C.4.7. Figure 5.9 reports the MCC observed at each step, grouped by policy.

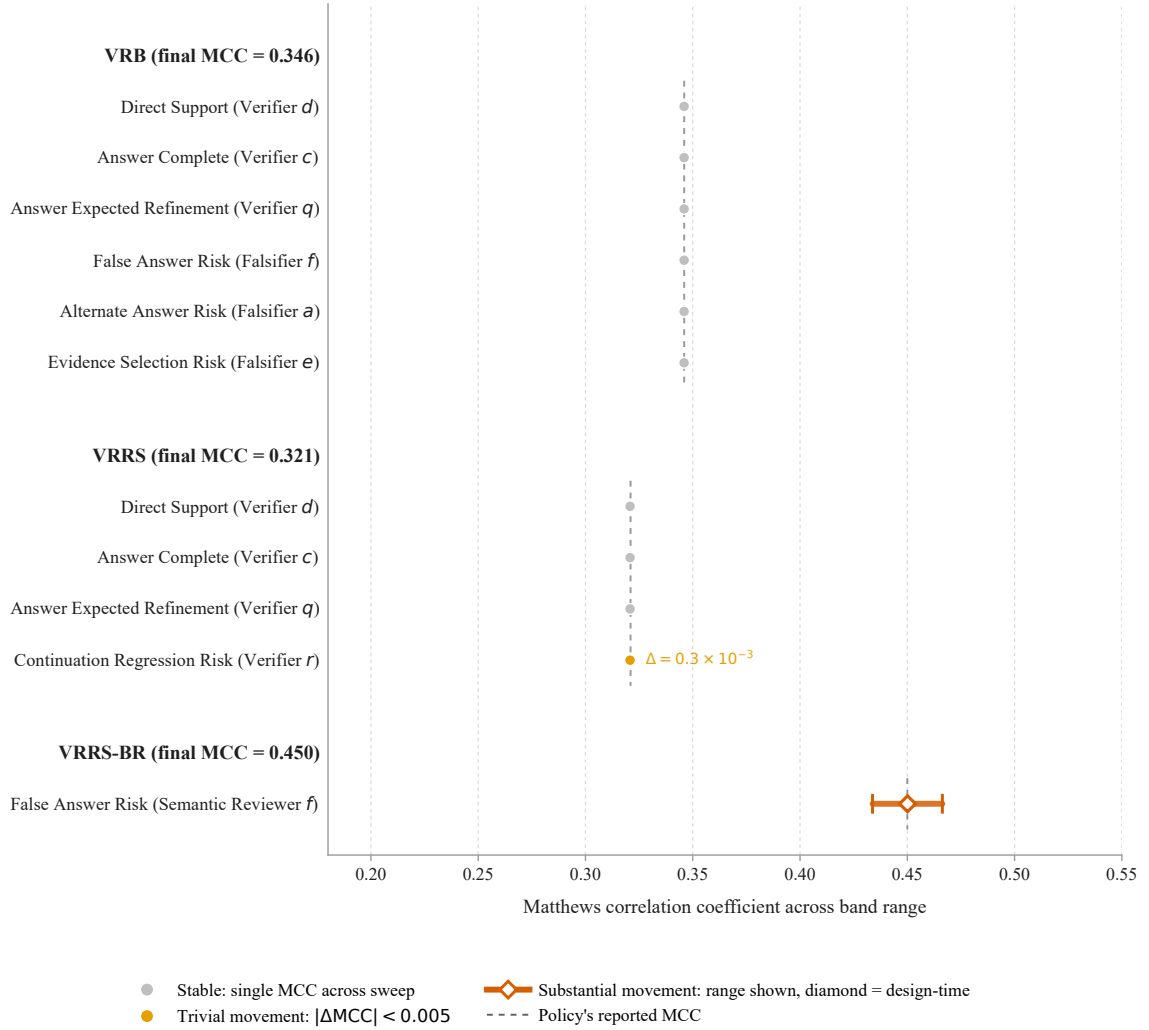


Figure 5.9 – MCC range under value perturbations of each band, grouped by policy.

Note. Each row reports one band placement.

Overall, nine of the eleven placements produced an identical confusion outcome at every step of their sweep. The remaining two placements move under perturbation. The VRRS continuation regression risk placement changes by less than 0.005 in MCC at the upper edge of its sweep, a single decision shift. The VRRS-BR reviewer’s false answer risk placement is the only substantial mover.

The VRRS-BR false answer risk placement moves symmetrically and within a narrow range. Lowering the threshold across the band gains at most two justified continuations and removes two premature stops; raising it to the top of the band loses two justified continuations and adds two premature stops. The reported result sits in the middle of this range rather than at either edge, the full sweep moves only four cases across the two safety cells, and unnecessary continuation does not change at any step. Consequently, the reported VRRS-BR result is not the product of a placement tuned to a knife edge optimum on the replay suite.

5.6.5 Interpretation of statistical robustness

The three analyses agree in direction while differing in strength, and that pattern is itself the result. The class balanced advantage of VRRS-BR survives the clustered structure of the replay suite: it holds across cluster aware resampling, paired cluster comparison, and the strict collapsed framing alike. Where they differ is in sensitivity, and the differences are informative. The advantage is not spread evenly across the policy stack but concentrated at the VRRS $\rightarrow$ VRRS-BR step, where the paired comparison shows the clearest separation of the three analyses; the deterministic step contributes routing rather than a paired gate success gain, consistent with the descriptive results. The collapsed McNemar check, the strictest view, recovers the same direction but less of the signal, because reducing each cluster to a majority label discards cluster contained improvements the other two analyses retain.

5.7 Chapter summary

The case study and the offline policy evaluation together establish two claims about the first clarification boundary in the CoC workflow.

Firstly, the trajectory case study establishes that a second clarification round is only conditionally beneficial. A further round repaired some wrong answers but degraded others that the first round had right, and this variation is what makes the boundary a genuine decision point: whether to continue cannot default to yes. This decision is best constructed from an evaluation of the agentic system state at that boundary.

Secondly, the offline evaluation designed around that decision constructed STARC as three

layered policies; VRB, VRRS and VRRS-BR. Of the three, Verification and Refinement with Risk Signals and Borderline Review (VRRS-BR) produced the strongest results, with continue recall, balanced accuracy and MCC all rising over the VRB baseline.

The per dataset attribution provides further detail: the aggregate improvement is composed of qualitatively different per stage contributions on different datasets. A cluster aware robustness analysis supports the ordering. The cluster bootstrap holds the ranking on every class balanced metric, with overlapping but ordered intervals; the paired cluster comparison gives the clearest separation, since where VRRS and VRRS-BR disagreed at the cluster level the movement was almost entirely in favour of VRRS-BR; and the cluster collapsed McNemar check agrees in direction while losing sensitivity, because reducing each cluster to a majority outcome discards the within cluster improvements the other two analyses detect.

Taken together, these results establish the core of the STARC architecture: the stopping decision at the first clarification boundary can be made effectively from the agentic system state alone. This is further developed in Chapter 6.

Chapter 6

Discussion

The findings from the analysis established two key facts. Firstly, at the first clarification boundary of AgenticLU’s Chain of Clarifications (CoC) workflow, an adaptive stopping decision is meaningfully available. Secondly, of the three policies evaluated, the policy Verification and Refinement with Risk Signals and Borderline Review (VRRS-BR) redistributes errors in the direction the cost asymmetric metrics favour. While it picks up a small increase in unnecessary continuation, it also catches substantially more wrong answers.

The architecture this thesis proposes for the stopping decision is the State Aware Risk Controller (STARC), and the three policies evaluated are its layers rather than independent controllers in competition. STARC uses structured Large Language Model (LLM) judgement to evaluate the agentic trace at a control boundary, rather than to produce the answer the trace is supposed to yield or to read agentic system confidence directly. It instantiates a more general design pattern applicable to agentic clarification systems.

The chapter proceeds as follows. It first works through what the case study’s four mechanisms imply for any controller built over the system state (Section 6.1). It then develops STARC and identifies it as the central contribution of the thesis (Section 6.2). It returns to the cost asymmetric framing and locates the regimes in which each policy is the appropriate operating point (Section 6.3). It situates the pattern against adjacent literature on adaptive RAG, agentic verification, and process level reward modelling (Section 6.4).

6.1 What the case study’s typology says about controller design

The four continuation outcomes are not statistical labels imposed on one underlying dimension. Each arises from a different reasoning pattern and leaves a different kind of evidence in the first round state, and thus, each places a different demand on the controller. STARC, developed in Section 6.2, is a direct response to that heterogeneity, composing multiple judgement modalities because no single one answers all four.

Two of the mechanisms are within reach of structural inspection. Repair trajectories tend to be explicit: the first round carries visible markers that the answer is poorly grounded, such as an underspecified entity, a hedged answer, or a clarification trace that did not narrow the answer space. These are exactly the markers the initial verifier / falsifier evaluation in VRB and deterministic risk signals in VRRS are built to read. The same is true of persistent failure: trajectories that cycle through the same evidence path, repeating a wrong answer with minor surface variation, leave a structural signature of weak evidence selection and shallow grounding that the signal families already detect.

The other two mechanisms are not explicit. Degradation cannot be caught by inspecting first round state at all, because the failure has not happened yet: the first round answer is correct, and the second round introduces the error through unsupported refinement. The only move available to a controller evaluating the first round state is the positive one: to recognise that the answer is already supported and accept it. The remaining half of persistent failure is subtle for the opposite reason: trajectories that produce a different wrong answer on a different evidence path present a fluent surface and a plausible trace, so the error is semantic rather than structural. Identifying such instances requires semantic judgement against known QA failure patterns, the task the reviewer performs. This is the self correction failure Huang et al. document, in which a model without an external signal revises a wrong answer into another wrong one [37]. It is the case the semantic reviewer exists to handle.

The asymmetry between the mechanisms also fixes where the controller’s effort is worth spending. Because the bulk of continuation’s value lies in preservation, most of that value is secured at no inference cost by accepting correct first round answers, which leaves the controller’s active work concentrated on the wrong answer repair cases. The cost asymmetric metrics encode this directly: the upside of continuation is dominated by preservation, so the marginal value of continuation per case is lower than a naive reading of the favourable count would suggest.

The typology also bounds what any first round state controller can achieve, and this is the sharper limit. A controller decides whether to continue, not whether continuation will

succeed. The moderate repair rate shown in the case study indicates that even a controller flagging every wrong first round answer is bounded above by the downstream repair rate of the second clarification round. As such, the headline policy numbers measure the quality of the stopping decision, not the success of the repair it enables. Final QA accuracy therefore carries a separate ceiling the controller cannot move. Persistent failure trajectories further illuminate this, because their second round failure is already partly fixed by first round features that constrain what additional reasoning can recover. How strongly those features correlate with second round irrecoverability is not resolved here: some persistent failure cases may be detectable from first round properties alone, and others may need the second round to run before the failure is visible.

6.2 STARC as the central contribution

STARC is a single architecture for deciding when to stop, realised here as three progressively layered policies. VRB produces structured LLM judgements for the rest of the system to consume: the verifier and falsifier records, with their typed fields for support, completeness, expected refinement gain, missing information type, regression risk, and an explicit refinement recommendation. VRRS adds no further LLM calls. It adds a deterministic stage that reads the verifier record together with four signal families built from the saved state, classifies what it sees into hard and soft risks, and partitions the suite into stop, continue, and borderline bands. VRRS-BR adds one more LLM call, the semantic reviewer, but it does not run that call on every case, only on the borderline band, and even there the reviewer’s recommendation is admitted only through a deterministic final continuation filter. Treated as three separate controllers, the three policies sit at different points on a trade off curve. The aggregate comparison in Section 5.4.1 is then evidence that the full design outperforms either of its layers. The per dataset analysis begins an investigation into which stages are doing the work in which setting, rather than which policy is best in the abstract. This is left as future work.

STARC unifies its layers by using the LLM calls as evaluators of the agentic system trace. The verifier reads the first round answer against its pinned context and reports whether the answer is supported. The falsifier reads the same state and reports whether a plausible alternative answer is available. The semantic reviewer, when invoked, examines the trace the clarification system produced in the first round, the provisional answer and its clarification chain rather than the question or the long context input, and reports whether known error conditions hold. None of these stages writes a new answer. All three write structured judgements over the trace produced by the underlying CoC pipeline. That separation is STARC’s central feature: the CoC system answers the question, and STARC evaluates whether the answer should be accepted, refined, or escalated. The two responsibilities call for different prompts,

different inputs, and, on the evidence here, different reasoning patterns.

The deterministic logic sitting between the LLM stages is not a rival to LLM judgement. Its job is connective, and across the stopping policies it does three things that the LLM stages, left to themselves, would not do reliably. First, aggregation. VRRS combines the verifier call’s structured fields with question and answer form, grounding, and clarification quality signals into hard and soft risk classifications. The deterministic logic reasons over the outputs the LLM generated, rather than displacing the LLM from the reasoning. Second, routing. The ordered band rule in Table 4.12 decides which cases are resolved by deterministic stop or continue assignment, and which are forwarded to semantic review. The deterministic layer is not trying to make the final call on every case. Instead, it identifies the cases it cannot confidently resolve and hands those cases on. Third, gating: the final continuation filter in VRRS-BR, shown in Table 4.13, requires the reviewer to identify known error categories, at high false answer risk, with its concrete error flag set. The filter stops the reviewer from acting on vague or generalised concerns: it confines the reviewer’s effective influence to the cases where the reviewer has identified an error. Together, LLM judgements enter the controller’s decision through structured outputs that deterministic logic can read, combine, and filter, and the deterministic logic composes those judgements without replacing them.

This reading is evidenced across the four datasets. HotpotQA: VRRS produced confusion cells identical to VRB, so the deterministic stage made no change to the controller’s behaviour, and the entire HotpotQA improvement came from the semantic reviewer alone. PopQA showed the inverse pattern: VRRS-BR produced confusion cells identical to VRRS, the semantic reviewer made no change, and the small PopQA improvement came from the deterministic stage alone. TriviaQA: the deterministic stage caught zero wrong answers, and once again the entire gain came from the semantic reviewer. Natural Questions was the only dataset on which both stages made complementary contributions. Failures of reasoning across multiple hops, on the HotpotQA evidence, are not exposed by structural inspection of the verifier’s fields or by the deterministic signal families. They are surfaced when an LLM evaluates the trace for a concrete identifiable error, which is what the reviewer is constrained to do. Fluent but wrong factoid answers, on the TriviaQA evidence, slip past the structural checks because they present a clean case for stopping; the reviewer catches them because a wrong entity is exactly the kind of concrete error it is built to name. Long tail factoid grounding errors, on the PopQA evidence, are caught by the deterministic stage’s structural inspection of the verifier record and the relationship between the answer and the evidence. The per dataset evidence shows that within STARC each stage is necessary for at least one dataset. Whether a single richer judgement modality could subsume the staged design is not tested here. A controller running the verifier alone would miss HotpotQA and TriviaQA. A controller running the semantic reviewer on every case would either incur the cost of an extra LLM call on every row, or lose the PopQA gain under a binding compute budget. The

layered design handles the mixture.

The small size of the VRB to VRRS improvement must be inspected. VRRS does not add any LLM calls over VRB: it adds the deterministic risk signal layer. On three of the four datasets, that layer produces either no change to confusion cells over VRB or only a marginal one. The most parsimonious explanation is that the verifier and falsifier prompts already encode, in their structured outputs, much of what the deterministic signal families were independently built to detect. An LLM prompted to fill typed fields for support, completeness, and expected refinement gain performs a version of the same structural inspection the signal families perform. The deterministic stage is therefore not principally adjudicating: it is routing. It produces the borderline band that the semantic reviewer can act on, and removing it would also remove the gating that protects the reviewer from being run indiscriminately. The implication for future controllers is clear: the value of an explicit deterministic stage lies in its ability to identify cases worth escalating, not in its ability to make stopping decisions in isolation. Further engineering of the deterministic signals should therefore be evaluated by what reaches the reviewer rather than by the aggregate metric.

The 70.6% borderline routing fraction can be misread as a sign that the design is barely more efficient than running the reviewer everywhere. The figure measures compute, not influence. The reviewer is consulted on 288 of 408 cases (70.6%), but its recommendations pass the final continuation filter on only 17 (4.2% of the suite). The compute cost of the layered design is the cost of borderline review. The decision influence of the reviewer is the much smaller set of cases the filter lets it flip from stop to continue, and the deterministic stage’s routing job concentrates that influence.

6.3 Cost asymmetric redistribution and why it is appropriate

The VRB to VRRS-BR result was characterised earlier as a risk sensitive redistribution of controller errors. The framing yields a recommendation only against a cost ratio between the two error types: how much more expensive is a prematurely accepted wrong answer than an unnecessary continuation? The metrics treat the first as the more serious error, but they do not fix the magnitude of the asymmetry.

Let C denote the cost of a premature stop relative to an unnecessary continuation. The $\text{VRB} \rightarrow \text{VRRS-BR}$ transition breaks even at

$$C = \Delta_{UC}/\Delta_{PS} = 20/23 \approx 0.87, \quad (6.1)$$

where $\Delta_{UC} = 20$ and $\Delta_{PS} = 23$ are the changes in unnecessary continuations and premature stops on the replay suite. The full layered design is the appropriate choice whenever a prematurely accepted wrong answer is judged at least as costly as the compute and latency of one additional clarification round. User facing and decision support deployments satisfy that condition by a substantial margin. The exception is the compute dominated regime where a wrong delivered answer is treated as cheap.

Comparing the two break-even points shows why the intermediate policy is rarely the right choice. VRRS-BR becomes worthwhile as soon as C exceeds 0.87, a bar most deployments clear, whereas VRRS only repays its larger continuation cost once C exceeds 2.71, which few deployments reach. VRRS is therefore preferable to VRB only in that narrow high cost regime. It is preferable to VRRS-BR in no regime at all, because VRRS-BR extends VRRS on the safety side metrics, catching sixteen more wrong answers, at the cost of a single additional unnecessary continuation. The intermediate policy is thus squeezed from both sides: dominated by VRRS-BR everywhere, and better than VRB only where the cost asymmetry is steep. The deployment choice that matters is between VRB and the full controller.

This reshapes the practical recommendation. The value of the full controller is concentrated in the semantic review stage, given that the deterministic stage has filtered correctly. If a deployment is considering dropping the semantic review stage in order to save the extra LLM call, it should also consider dropping the deterministic stage and reverting to verifier and falsifier judgement alone. The intermediate configuration trades efficiency for sensitivity at an unfavourable ratio. The deterministic stage’s principal practical job is routing, not adjudication, and without an adjudication stage to receive borderline cases, routing alone does only half the work the stage was built for.

STARC therefore exposes a small set of operating points, and the right one follows from the cost ratio a deployment imposes. The specific thresholds derive from the aggregate counts on this replay suite, and they do not account for per dataset variation or for how recoverable a wrong answer is downstream. A deployment outside the regimes considered here would derive its own cost ratio from the costs that apply in its domain and reevaluate the three policies on that basis. This trade off geometry is not unique to clarification. The cost saving cascade literature reaches a similar conclusion in single shot QA, where allowing earlier stage abstention trades a small rise in overall abstention for a substantial reduction in cost and error [66]. The present setting differs only in that the alternative to commitment is the second clarification round, rather than a flat refusal. Consequently, a small concession on the efficiency side buys a larger gain on the safety side.

6.4 Broader applicability of the staged evaluator pattern

The contribution developed so far is local: STARC applies to agentic QA at the first boundary of AgenticLU’s Chain of Clarifications (CoC) system, and the empirical work is small in absolute scale. STARC is primarily an instance of a general design pattern, not a novel algorithm. Consequently, the relevant question for generalisation is not whether STARC as a whole transfers, which would need evidence the thesis does not produce. It is whether the underlying design pattern aligns with directions the wider agentic literature is independently pursuing. Four threads converge in ways that strengthen the broader case.

The first is the framing of judges as evaluators of traces rather than outcomes. The *Agent-as-a-Judge* proposal of Zhuge et al. [67] crystallises this shift, and the AgentRewardBench evaluation [68] reports that the strongest LLM judge of web agent trajectories agrees with the rule based evaluator on roughly 80% of cases, which is useful but not enough to act on as the sole stopping signal, thus establishing why composing the judge with a deterministic filter is preferable to letting it decide alone. STARC is a worked instance of trace level judgement at a boundary this literature has not yet addressed.

The second is the emergence of staged controllers in adaptive RAG. Corrective RAG [69] routes documents through a lightweight evaluator into three downstream actions. Adaptive RAG [70] routes queries among retrieval strategies of differing cost via an upstream classifier. Stop-RAG [71] treats iterative retrieval as a finite horizon control problem and learns when to halt directly. Three independent designs have converged on the same shape: a cheap classifier handles the easy cases, escalation is held back for the truly uncertain ones, and the stopping decision is treated as a problem in its own right. This thesis is another instance operating over a different boundary that has been established in the agentic workflow.

The third is the cost asymmetric verification literature. Sherlock [72] attaches verification stages selectively to error prone nodes of an agentic workflow, on the explicit motivation that verifying every step is too expensive and verification value is concentrated at a small number of locations. This is the same logic as the borderline band gating in VRRS-BR. The thesis instantiates the same selective verification logic, applied to a clarification trace rather than a single reasoning step, and gated by an explicit cost ratio analysis.

The fourth is the process evaluation literature, which has shown that supervising reasoning at the step level outperforms supervising it at the outcome level for mathematical reasoning [73], and that the advantage extends to verbalised judges that emit explanations alongside scores [74]. Both lines of work compose multiple judgement modalities to catch different failure signatures, and both find that selective application of the more expensive modality

outperforms uniform application. The thesis carries the principle outside the mathematical reasoning setting in which it was developed.

The niche is at the intersection of these threads. The clarification driven QA literature [75, 76] addresses what to clarify and whether clarification will help; none of it addresses what to do at the boundary where the clarification has happened. The adaptive RAG literature addresses retrieval halting, not clarification halting. The process reward literature addresses mathematical reasoning steps, not clarification rounds. As such, the transferability is architectural: the staged evaluator pattern is plausibly useful at any control boundary in an agentic system where the evaluation criterion is semantically distinct from the answer generation criterion. Candidate boundaries include whether to retrieve more, whether to decompose a sub question further, whether to invoke a tool, whether to commit to a final answer, and whether to abstain. The thesis shows the pattern works in one concrete setting and gives reasons to expect it to work in adjacent ones. The weakest of the candidate boundaries is tool invocation. The thesis assumes a trace whose elements are uniformly LLM produced text, which is what makes the evaluation criterion legible to an LLM judge. A tool using agent’s trace is cross modal, mixing model outputs with external API returns and environmental side effects, so whether a staged evaluator can read trace sufficiency under that condition is not something the present evidence speaks to. Showing that the pattern generalises would take additional studies and is left for future work.

6.5 Conclusions

The empirical contribution of the thesis is STARC, a staged evaluator architecture for clarification driven question answering. Dual perspective judgements feed deterministic aggregation and routing. The borderline reviewer is invoked only on cases the deterministic stage cannot resolve, with the reviewer’s recommendation filtered before it is allowed to act. The per dataset evidence shows that different stages do most of the work on different question types, and the design shows that composing LLM judgements with deterministic gating between them produces a class balanced controller across heterogeneous question distributions. The broader contribution is the pattern itself: deterministic logic performing aggregation, routing, and selective escalation around LLM judgements of the trace. The most important question the evidence leaves unresolved is how the pattern transfers to deeper clarification boundaries and to other agentic control problems where the evaluation criterion is semantically distinct from the answer generation criterion. Future work, in Chapter 9, takes those questions up directly.

Chapter 7

Limitations

7.1 Replay design

The main comparison runs on a fixed offline replay suite, where every policy receives the same saved first round state for the same example, and the desired action is fixed from first round correctness. The design utilises the replay suite to isolate the results from LLM run variation, allowing for concrete conclusions to be made on policy performance. The same design choice means the replay cannot characterise how STARC behaves under model version drift, retrieval distribution shift, or non stationary user question distributions. The replay establishes the relative behaviour of the three policies on identical inputs. It does not establish cross distribution stability.

7.1.1 The scale of the empirical evaluation

The empirical base is modest: a 408 state replay suite across 85 clusters, and a 26 trajectory case study drawn from Natural Questions. The constraint is practical rather than methodological. The saved states are generated by running the inherited AgenticLU/HELMET system at its maximum 128K token configuration, which requires A100 class hardware and executes over many hours per run; on the compute available to this project, those runs were also subject to interruption, which is what the persistent progress mechanism described in Section 4.2.2 was built to absorb. The case study is bounded a second time by the cost of manual per round adjudication on top of generation.

The claims are calibrated to this evidence. The 26 trajectories are not used to estimate population rates of the four continuation mechanisms; they are used only to establish that all four occur and that each places qualitatively different demands on the controller, which

is a claim that hand adjudicated trajectories of this number can support. Likewise, the statistical analysis in Section 5.6 reports the overlapping cluster bootstrap intervals directly and treats the headline gain as a stable ordering of the policies rather than a significance claim.

The limitation that remains is therefore narrow. The work establishes the existence and structure of the continuation mechanisms and the relative ordering of the policies, but the small, correlated sample means the magnitude of the central quantitative gain is directional rather than statistically secured, and the frequencies of the four mechanisms are left uncharacterised. This bounds the strength of the empirical conclusions. It does not bear on the architectural and methodological contributions, which do not depend on the scale of the evaluation. It is the most consequential limitation of the thesis, and it reflects the compute economics of long context agentic research.

7.2 Judge dependence at every stage

STARC rests on three LLM judgements: the verifier, the falsifier, and the semantic reviewer. The case study uses manual round level adjudication, and the offline replay uses an automated judging protocol for first round correctness. Manual judging at the case study scale is small sample and noisy on edge cases, and automated judging inherits LLM as judge biases including position bias, verbosity bias and self preference. The case study’s manual adjudication was performed against the four mechanism typology specified in Section 4.3.5 before any trajectory was examined, which bounds inductive judge variance by removing the category discovery step from the adjudication task. A second human rater on the same trajectories would characterise residual variance on the assigned labels directly, and is a natural extension of the case study protocol.

The staged design reduces exposure to any single judge’s biases in three specific ways. The final continuation filter constrains the reviewer’s effective influence to cases where it has identified a specific named error from a predefined set. The deterministic stage between the dual perspective Stage 1 judges and the Stage 3 reviewer aggregates rather than relies on any one signal. The paired cluster comparison cancels biases that affect compared policies identically. Residual exposure remains, particularly for biases specific to the reviewer’s role inside the filter’s constraints, and replication with a different reviewer model on the same replay suite characterises this exposure directly.

7.3 Scope of evaluation

The empirical work covers one control boundary, four RAG QA benchmarks, and one fine tuned base model. The boundary is the first clarification boundary of AgenticLU’s Chain of Clarifications (CoC) workflow. The benchmarks are Natural Questions, PopQA, TriviaQA and HotpotQA. The base model is the one AgenticLU is calibrated against. The staged evaluator pattern transfers to deeper clarification boundaries, to closed book settings, to non QA agentic tasks, and to different base models at the level of design pattern. The transfer is argued, not tested. The architectural pattern is the part of the contribution that travels, while the specific prompts, signal families and threshold bands are calibrated to the present setting and would need to be re derived elsewhere.

A concrete instance of this re-derivation requirement appears in the deterministic hard signals. Several are implemented as surface heuristics tied to English orthography and question phrasing: a four digit year regex, lexical candidate counting and hedge cues, and question echo token rules. These are deliberately cheap and inspectable, but they are brittle to language, locale, and question style. A year expressed in a non-Gregorian or non-numeric form, or a question whose answer type is signalled differently, would evade them. This fragility is confined to the signal implementations themselves; the staged evaluator pattern, and the hard and soft separation it rests on, do not depend on these particular surface checks, and a different setting would substitute signals appropriate to it.

7.4 Binary correctness as the desired action signal

The desired action for each replay row is derived from a binary first round correctness label: stop when the answer is correct, continue when it is wrong. The label is the right operational definition for the controller’s job, and the class balanced metrics weight wrong answer detection and correct answer acceptance equally. The label nevertheless flattens the downstream cost of a wrong answer. A gently wrong answer, a confidently wrong answer and a dangerously wrong answer collapse to the same category, and the metrics treat the controller’s wrong answer detections as equivalent regardless of which kind of wrong answer they catch. The cost ratio analysis in Section 6.3 works at the aggregate level and does not unflatten this distinction. A cost weighted evaluation that differentiates among kinds of wrong answer by downstream cost is a sharper instrument than the one used here.

7.5 What the limitations do not threaten

The framework level contribution of an observable round state schema is independent of the bounds set out above, since the schema's value is its instrumentation of the workflow rather than the empirical scale of any particular evaluation. The four mechanisms identified in the case study describe continuation behaviour at the level of what happens to an answer between rounds, and are not specific to Natural Questions as a source dataset. The case study does not establish that the typology is exhaustive across all question types. It establishes only that these four are sufficiently present to require qualitatively different controller responses, and the Stage 3 design uses that property. The paired cluster evidence for the marginal effect of the semantic review stage holds within the replay design, and the per dataset attribution that different stages of STARC do most of the work on different question types is the strongest piece of evidence the thesis offers for the layered design. The contribution the thesis claims is architectural and methodological. The limitations qualify its empirical extent. They do not contest its shape.

Chapter 8

Conclusion

Agentic question answering systems increasingly treat iterative clarification as a refinement procedure: each round localises evidence, sharpens the question, and is assumed to improve the answer. This thesis shows that premise is incomplete. A second clarification round repairs some wrong answers, leaves others unrepaired, and degrades a small but consistent fraction that were already correct. Thus, a system that runs further rounds without deciding whether to run them is acting on autopilot at a boundary that demands a judgement. This thesis turns that boundary into a decision, and shows the decision can be made from the saved state alone.

8.1 What the thesis established

This thesis turns that observation into a controller. The work breaks into three results.

- i. The first boundary in an iterative clarification workflow can be made observable.

Instrumenting AgenticLU’s Chain of Clarifications (CoC) runtime preserves the full state at each boundary: the provisional answer, the pinned context, the clarification question, and the surrounding trace. An internal computation becomes a structured object that can be read, classified and compared, and the offline replay design and per stage attribution rest on it directly.

- ii. Continuation is conditional, and its value is asymmetric.

Continuation at the boundary breaks into four mechanisms: repair of a wrong answer, preservation of an already correct one, persistent failure that cycles through the same evidence path,

and degradation of a correct answer the first round had right. The largest favourable category is preservation, not repair. Accepting correct first round answers secures most of continuation’s aggregate value at zero additional inference cost, and the marginal value the controller has to actively protect is concentrated in the wrong answer repair cases.

iii. STARC decides that boundary better than structured judgement alone.

Composing a verifier and falsifier with a deterministic stage and a selectively gated semantic reviewer raises continue recall from 44.7% to 67.0%, balanced accuracy from 66.1% to 74.0%, and MCC from 0.35 to 0.45 over the baseline. The movement is a risk sensitive redistribution rather than a uniform efficiency gain: the controller catches 23 more wrong answers at a cost of 20 more unnecessary continuations. The per dataset attribution is the sharpest evidence for the design. The deterministic stage carries PopQA, the semantic reviewer carries HotpotQA and TriviaQA, and only the composed controller handles all four. No single stage is sufficient on its own.

8.2 What generalises beyond this setting

At its core, STARC is a division of labour: the LLM judges the trace, and deterministic logic composes those judgements into a stop or continue decision. This is the principle that carries the design, and two layers of it travel beyond the present setting. At the framework level, the observable round state design transfers to any agentic system with discrete reasoning rounds and inspectable intermediate state. At the architecture level, the staged evaluator pattern that STARC instantiates applies at any control boundary where the criterion for evaluating an answer is semantically distinct from the criterion for generating it. This is a direction adjacent literatures, adaptive RAG, agentic verification, and process level reward modelling, are independently moving toward. STARC is a worked instance of that pattern at the first boundary of an iterative clarification workflow.

8.3 The central argument

Stopping in agentic QA is a multi judgement control problem rather than a scalar threshold problem. A heterogeneous workload presents different kinds of errors on different question types, and a controller that decides from a single signal cannot address all of them, as no single signal is informative across every type. As such, the stopping decision is better treated as an evaluation of the agentic system state than as a threshold on a confidence value.

A controller that decides from a confidence signal assesses how confident the system is in its

answer. On the other hand, STARC assesses what the agentic system state reveals about that answer: whether the trace that produced it contains a specific, identifiable reason to continue. This distinction is the central argument of the thesis: in agentic question answering, the decision of when to stop is best read from the system state that produced the answer, not from the model's confidence in the answer itself.

Chapter 9

Future Work

9.1 The staged evaluator at deeper clarification boundaries

STARC and the surrounding work are scoped to the first clarification boundary: the case study, the policy family, and the cost asymmetric analysis are all calibrated to it. The most immediate open question is what happens at the second boundary and beyond. The staged evaluator pattern transfers to deeper boundaries, but each policy would need to be re-instantiated against a richer state, because the assumptions the policies make about the first round no longer hold further along the trajectory. The verifier and falsifier prompts assume a single provisional answer over a single pinned context, whereas a deeper boundary presents a full trajectory; the four signal families read structural properties of a first round, but at a deeper boundary the clarification history that preceded it changes the diagnostic value of those same cues; and the semantic reviewer, which reads a first round trace, would instead face a multi round one.

A study at the second boundary follows directly, and the replay design extends to it without modification: each round already produces a saved state, so the same offline approach applies using state preserved from the second round. Three questions would structure such a study. First, does the four mechanism typology from the case study persist at deeper boundaries, or does it split further as additional rounds open new failure modes, such as recovery from a previously degraded answer? Second, does the per dataset attribution found here, in which the semantic reviewer carries HotpotQA and TriviaQA while the deterministic stage carries PopQA, hold or shift as the trace grows? Third, does the cost ratio analysis change in kind, given that the marginal compute cost of round N differs from that of the first round? The first two questions can be answered by replaying the second boundary on the same four

datasets; the third requires a different cost model.

9.2 Refining the deterministic stage for routing rather than adjudication

The VRB to VRRS improvement is small relative to the gain from adding the semantic reviewer. This points toward a redesign of the deterministic stage: rather than making stop or continue assignments directly, the signal families would be tuned to produce a borderline band that the reviewer is well positioned to act on. Under this view, the deterministic layer is judged not by its own confusion cells against a reference, but by the band it produces and how well that band correlates with what the reviewer can recover.

Two lines of work follow. The first holds the reviewer fixed and varies the design of the deterministic stage, measuring the reviewer’s continue recall and false positive rate within the band rather than across the full suite; this identifies which combinations of signal families expose the cases the reviewer can act on. The second extends the signal families with features the present design does not use: trajectory level shape, evidence overlap between the first round and the clarification question, and answer entropy across context variants of the same base question. All three are inspectable from the saved first round state.

9.3 Transfer to other agentic control boundaries

The staged evaluator pattern applies at any control boundary in an agentic QA system where the evaluation criterion is semantically distinct from the answer generation criterion. Three transfer studies test the pattern at three concrete boundaries.

The first is retrieval halting. Stop-RAG [71] treats the question of when to halt retrieval as a control problem, learning a value based controller over a finite horizon. A staged evaluator alternative replaces the learned value function with three components: a verifier that judges retrieved evidence sufficiency, a deterministic stage that aggregates retrieval quality signals with the verifier’s output, and a reviewer that adjudicates the retrieval trace when the deterministic stage cannot resolve. A direct comparison with Stop-RAG on the same benchmark would test whether the staged judgement pattern is competitive with learned value control.

The second is sub question decomposition in multi hop QA. Multi hop pipelines split a question into smaller queries, and at each split there is a control decision: decompose further, or commit to the current decomposition. A staged evaluator at this boundary judges decomposition sufficiency rather than answer sufficiency, but the architectural structure is the same.

The transfer study tests whether the pattern produces controllers competitive with existing decomposition stopping heuristics.

The third is tool invocation in agents. A tool calling agent has a control decision at each step: invoke a tool, continue reasoning, or commit to an answer. A staged evaluator that judges trace sufficiency sits at this boundary too. This is the largest of the three transfers: these agents have a richer action space than QA pipelines, and the binary stop or continue decision the thesis evaluates becomes a multi way choice. Whether the pattern extends to multi way control is itself an open question, which a tool calling agent study would address.

9.4 Isolating the dual perspective split

The Verification and Refinement Baseline (VRB) splits the structured LLM judgement of the first round state into two separate calls: a verifier whose framing argues the case for stopping, and a falsifier whose framing argues the case for continuing. The design choice is motivated in Section 4.4.3.2 from the *LLM as judge* literature, in which a single judge asked to weigh supporting and adversarial evidence within one prompt has been shown to collapse to whichever frame the model finds more natural [44]. The empirical effect of the split on the present replay suite remains to be isolated.

The ablation that would isolate it is small in scope. A combined judge call would fill the union of the verifier and falsifier fields in a single prompt over the same shared decision context. The deterministic stage that follows is held fixed. The combined judge emits the same band labels on the same fields, and the resulting policy decisions are scored against the same desired action signal. Three quantities follow from such a comparison: the change in agreement on the per field bands between the split and combined configurations, the change in the controller’s confusion cells against the desired action signal, and the change in the operational cost of the verifier stage measured in LLM calls.

Three outcomes are possible. The split configuration may produce systematically different band labels on the same field on the same input, supporting the self consistency mechanism described in Section 4.4.3.2. The two configurations may produce equivalent band labels, in which case the split adds an LLM call per replay row without changing the policy decision and would be a candidate for removal. Or the split may shift the band labels but not the downstream policy decision, which would identify the deterministic stage, rather than the dual perspective split, as the load bearing part of the architecture. Each outcome is informative about a different element of the design, and the present evaluation does not separate them.

9.5 Cost weighted evaluation that does not flatten answer value

The binary correctness label used here treats every wrong answer the same, and the cost ratio analysis in Section 6.3 works at the aggregate level on top of that label. A cost weighted evaluation would differentiate among kinds of wrong answer by downstream cost. Three components are needed. First, a taxonomy of wrong answer types with associated downstream cost weights. Gently wrong, confidently wrong, and dangerously wrong are indicative categories; the weights are derived from the deployment context the controller serves. Second, a per example cost weight on the desired action signal, replacing the binary correct or wrong label with a valued cost of acting wrongly. Third, cost weighted versions of the class balanced metrics that aggregate over the new desired action signal.

The refinement sharpens the cost ratio analysis in two ways. The single indifference point near $C \approx 0.87$ for the full controller against VRB becomes a function over the cost weights, exposing how stable the architecture's preference structure is to plausible weight choices. The per dataset attribution gains a workload sensitivity dimension: if HotpotQA's wrong answers are systematically more costly than PopQA's, the cost weighted metric changes the relative contribution of the semantic reviewer and the deterministic stage. The methodological extension is small to implement but substantial in what it exposes. It shows how the staged design behaves under workloads with different consequences for wrong answers.

9.6 Closing

The thesis evaluates one stopping decision in one system, but it builds the apparatus needed to evaluate many. The instrumented runtime, the four mechanism case study and the staged evaluator pattern are reusable across the directions set out above. What the thesis demonstrates at the first clarification boundary, the directions above propose to test at deeper boundaries, at adjacent agentic boundaries, and under cost regimes that real deployments impose. The boundary studied here is the first one such an evaluation can run on; it is not the last.

List of References

- [1] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shubham Acharya, Dmitry Rekish, Fei Jia, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=kIoBbc76Sy>.
- [2] Yufan Zhuang, Xiaodong Yu, Jialian Wu, Ximeng Sun, Ze Wang, Jiang Liu, Yusheng Su, Jingbo Shang, Zicheng Liu, and Emad Barsoum. Self-taught agentic long context understanding, 2025. URL <https://arxiv.org/abs/2502.15920>.
- [3] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2306.13063>.
- [4] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Christopher Olah, and Jared Kaplan. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
- [5] Howard Yen, Tianyu Gao, Max Hou, Keyang Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context models effectively and thoroughly. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=293V3bJbmE>.
- [6] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a.00276. URL <https://aclanthology.org/Q19-1026/>.
- [7] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259. URL <https://aclanthology.org/D18-1259/>.
- [8] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the*

- Association for Computational Linguistics*, 10:539–554, 2022. doi: 10.1162/tacl_a.00475. URL <https://aclanthology.org/2022.tacl-1.31/>.
- [9] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. KILT: a benchmark for knowledge intensive language tasks. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2523–2544, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.200. URL <https://aclanthology.org/2021.naacl-main.200/>.
- [10] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, 2021. doi: 10.18653/v1/2021.naacl-main.365. URL <https://aclanthology.org/2021.naacl-main.365/>.
- [11] Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Jason Phang, Angelica Chen, Vishakh Padmakumar, Johnny Ma, Jane Thompson, He He, and Samuel R. Bowman. QuALITY: Question answering with long input texts, yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*, 2022. doi: 10.18653/v1/2022.naacl-main.391. URL <https://aclanthology.org/2022.naacl-main.391/>.
- [12] Sara Rosenthal, Avi Sil, Radu Florian, and Salim Roukos. CLAPnq: Cohesive long-form answers from passages in natural questions for RAG systems. *Transactions of the Association for Computational Linguistics*, 13:53–72, 2025. doi: 10.1162/tacl_a.00729. URL <https://aclanthology.org/2025.tacl-1.3/>.
- [13] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.
- [14] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a.00638. URL <https://aclanthology.org/2024.tacl-1.9/>.
- [15] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. LooGLE: Can long-context language models understand long contexts? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. doi: 10.18653/v1/2024.acl-long.859. URL <https://aclanthology.org/2024.acl-long.859/>.
- [16] Cheng-Yu Hsieh, Yung-Sung Chuang, Chun-Liang Li, Zifeng Wang, Long T. Le, Abhishek Kumar, James Glass, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Found in the middle: Calibrating positional attention bias improves

- long context utilization. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL <https://arxiv.org/abs/2406.16008>.
- [17] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. NoLiMa: Long-context evaluation beyond literal matching. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 44554–44570, 2025. URL <https://arxiv.org/abs/2502.05167>.
- [18] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2309.17453>.
- [19] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2309.00071>.
- [20] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227, 2023.
- [21] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. Making retrieval-augmented language models robust to irrelevant context. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/8011b23e1dc3f57e1b6211ccad498919-Abstract-Conference.html.
- [22] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.11511>.
- [23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [24] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023. doi: 10.18653/v1/2023.acl-long.557. URL <https://aclanthology.org/2023.acl-long.557/>.
- [25] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.

- [26] Tomer Wolfson, Harsh Trivedi, Mor Geva, Yoav Goldberg, Dan Roth, Tushar Khot, Ashish Sabharwal, and Reut Tsarfaty. MoNaCo: More natural and complex questions for reasoning across dozens of documents. *Transactions of the Association for Computational Linguistics*, 14:23–46, 2026. URL <https://aclanthology.org/2026.tacl-1.2/>.
- [27] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [28] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let’s sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL <https://arxiv.org/abs/2305.11860>.
- [29] Eunsol Choi, He He, Mohit Iyyer, Mark Yatskar, Wen-tau Yih, Yejin Choi, Percy Liang, and Luke Zettlemoyer. QuAC: Question answering in context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018. doi: 10.18653/v1/D18-1241. URL <https://aclanthology.org/D18-1241/>.
- [30] Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. AmbigQA: Answering ambiguous open-domain questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020. doi: 10.18653/v1/2020.emnlp-main.466. URL <https://aclanthology.org/2020.emnlp-main.466/>.
- [31] Gangwoo Kim, Sungdong Kim, Byungsoo Jeon, Joonsuk Park, and Jaewoo Kang. Tree of clarifications: Answering ambiguous questions with retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.63/>.
- [32] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: A large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics*, 2018. doi: 10.18653/v1/N18-1074. URL <https://aclanthology.org/N18-1074/>.
- [33] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena, 2023.
- [34] Tong Yue, Rui Mao, Xiaoming Shi, Shulin Zhan, Zhen Yang, and Dongyan Zhao. QAEval: Mixture of evaluators for question-answering task evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. doi: 10.18653/v1/2025.acl-long.716. URL <https://aclanthology.org/2025.acl-long.716/>.
- [35] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir

- Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2303.17651>.
- [36] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Sx038qxjek>.
- [37] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet, 2023.
- [38] Qianye Chen, Libo Qin, Jingyang Liu, Dongming Peng, Jiale Guan, Pengfei Wang, Mengjie Hu, Yang Zhou, Tianyu Gao, and Wanxiang Che. When more thinking hurts: Overthinking in LLM test-time compute scaling, 2025. URL <https://arxiv.org/abs/2504.10739>.
- [39] Michael Hassid, Gabriel Synnaeve, Yossi Adi, and Roy Schwartz. Don’t overthink it. preferring shorter thinking chains for improved LLM reasoning, 2025. URL <https://arxiv.org/abs/2505.17813>.
- [40] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5433–5442, 2023. doi: 10.18653/v1/2023.emnlp-main.330.
- [41] Daniel Yang, Yao-Hung Hubert Tsai, and Makoto Yamada. On verbalized confidence scores for LLMs, 2024. URL <https://arxiv.org/abs/2412.14737>.
- [42] Zhangyue Yin, Qiushi Sun, Qipeng Guo, Jiawen Wu, Xipeng Qiu, and Xuanjing Huang. Do large language models know what they don’t know? In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8653–8665, 2023. doi: 10.18653/v1/2023.findings-acl.551.
- [43] Shangbin Feng, Weijia Shi, Yike Wang, Wenxuan Ding, Vidhisha Balachandran, and Yulia Tsvetkov. Don’t hallucinate, abstain: Identifying LLM knowledge gaps via multi-LLM collaboration. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 14664–14690, 2024.
- [44] Lin Shi, Chiyu Ma, Weicheng Liang, Xinyuan Diao, Wentao Ma, and Soroush Vosoughi. Judging the judges: A systematic study of position bias in LLM-as-a-judge. In *Proceedings of the International Joint Conference on Natural Language Processing and the Annual Meeting of the Asia-Pacific Chapter of the Association for Computational Linguistics (IJCNLP-AAACL)*, pages 292–314, 2025. URL <https://arxiv.org/abs/2406.07791>.
- [45] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-Yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on

- performance of large language models. In *Proceedings of EMNLP 2024: Industry Track*, 2024. URL <https://arxiv.org/abs/2408.02442>.
- [46] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.
- [47] Amita Kamath, Robin Jia, and Percy Liang. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. doi: 10.18653/v1/2020.acl-main.503. URL <https://aclanthology.org/2020.acl-main.503/>.
- [48] Jeremy Cole, Michael Zhang, Daniel Gillick, Julian Eisenschlos, Bhuwan Dhingra, and Jacob Eisenstein. Selectively answering ambiguous questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. doi: 10.18653/v1/2023.emnlp-main.35. URL <https://aclanthology.org/2023.emnlp-main.35/>.
- [49] Ji Xin, Raphael Tang, Jaesun Lee, Yaoliang Yu, and Jimmy Lin. DeeBERT: Dynamic early exiting for accelerating BERT inference, 2020.
- [50] Weijie Liu, Peng Zhou, Zhe Wang, Zhiruo Zhao, Haotang Deng, and Qi Ju. FastBERT: A self-distilling BERT with adaptive inference time. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. doi: 10.18653/v1/2020.acl-main.537. URL <https://aclanthology.org/2020.acl-main.537/>.
- [51] Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A. Smith. The right tool for the job: Matching model and instance complexities. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. doi: 10.18653/v1/2020.acl-main.593. URL <https://aclanthology.org/2020.acl-main.593/>.
- [52] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2203.11171>.
- [53] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- [54] Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms. In *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining*, 2011. doi: 10.1145/1935826.1935878.
- [55] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, pages 1097–1104, 2011. URL <https://arxiv.org/abs/1103.4601>.
- [56] Adith Swaminathan and Thorsten Joachims. Counterfactual risk minimization: Learning from logged bandit feedback. *Journal of Machine Learning Research*, 16(52): 1731–1755, 2015.

- [57] Kay H. Brodersen, Cheng Soon Ong, Klaas E. Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *Proceedings of the 20th International Conference on Pattern Recognition*, 2010. doi: 10.1109/ICPR.2010.764.
- [58] Davide Chicco and Giuseppe Jurman. The advantages of the matthews correlation coefficient over f1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(6), 2020. doi: 10.1186/s12864-019-6413-7.
- [59] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi: 10.1371/journal.pone.0118432.
- [60] Christopher A. Field and Alan H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x.
- [61] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978, 2001.
- [62] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.546. URL <https://aclanthology.org/2023.acl-long.546/>.
- [63] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147/>.
- [64] Yichen Jiang and Mohit Bansal. Avoiding reasoning shortcuts: Adversarial evaluation, training, and model development for multi-hop QA. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2726–2736, 2019. doi: 10.18653/v1/P19-1262. URL <https://aclanthology.org/P19-1262/>.
- [65] Nina Poerner, Ulli Waltinger, and Hinrich Schütze. E-BERT: Efficient-yet-effective entity embeddings for BERT. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 803–818, 2020. doi: 10.18653/v1/2020.findings-emnlp.71. URL <https://aclanthology.org/2020.findings-emnlp.71/>.
- [66] Michael J. Zellinger, Rex Liu, and Matt Thomson. Cost-saving LLM cascades with early abstention, 2025. URL <https://arxiv.org/abs/2502.09054>.
- [67] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong

- Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents, 2024. URL <https://arxiv.org/abs/2410.10934>.
- [68] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories, 2025. URL <https://arxiv.org/abs/2504.08942>.
- [69] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation, 2024. URL <https://arxiv.org/abs/2401.15884>.
- [70] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.389. URL <https://aclanthology.org/2024.naacl-long.389/>.
- [71] Jaewan Park, Solbee Cho, and Jay-Yoon Lee. Stop-RAG: Value-based retrieval control for iterative RAG, 2025. URL <https://arxiv.org/abs/2510.14337>. NeurIPS 2025 MTI-LLM Workshop.
- [72] Yeonju Ro, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, Aditya Akella, Zhangyang Wang, Mattan Erez, and Esha Choukse. Sherlock: Reliable and efficient agentic workflow execution, 2025. URL <https://arxiv.org/abs/2511.00330>.
- [73] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6EOi>.
- [74] Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. Process reward models that think, 2025. URL <https://arxiv.org/abs/2504.16828>.
- [75] Yizhou Chi, Jessy Lin, Kevin Lin, and Dan Klein. CLARINET: Augmenting language models to ask clarification questions for retrieval, 2024. URL <https://arxiv.org/abs/2405.15784>.
- [76] Chinmaya Andukuri, Jan-Philipp Fränken, Tobias Gerstenberg, and Noah D. Goodman. STaR-GATE: Teaching language models to ask clarifying questions. In *Conference on Language Modeling*, 2024. doi: 10.48550/arXiv.2403.19154. URL <https://openreview.net/forum?id=CrzAj0kZjR>.
- [77] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.

Appendix A

Literature Review Supporting Material

A.1 Summary of reviewed literature

Table A.1 consolidates every work cited Chapter 2, grouped by research area and noting each work’s main finding and the role it plays in motivating the stop or continue decision the thesis addresses. The table is provided as a reference; it does not introduce material not already discussed in the chapter body.

Authors / Year	Research area	Main finding and relevance to this thesis
Kwiatkowski et al. (2019) ; Yang et al. (2018) ; Trivedi et al. (2022) ; Petroni et al. (2021)	Long context and multi hop question answering	Question answering increasingly requires systems to locate, combine and reason over evidence rather than generate fluent answers. KILT consolidates the broader knowledge intensive evaluation paradigm under which several of the benchmarks used in this thesis sit.
Dasigi et al. (2021) ; Pang et al. (2022) ; Rosenthal et al. (2025)	Long document and passage grounded QA	Long form and document grounded QA depend on using distributed evidence coherently. Motivates treating the clarification trace and pinned context as decision relevant state.
Liu et al. (2024) ; Hsieh et al. (2024) ; Li et al. (2024) ; Yen et al. (2025) ; Hsieh et al. (2024) ; Modarressi et al. (2025)	Long context model evaluation and positional effects	Effective context use lags nominal context size; models remain sensitive to evidence position even at large window sizes; the U curve persists into 2025 class models and degradation under paraphrase only retrieval is severe. Rules out trusting first round answers on the basis of fluency alone.

Authors / Year	Research area	Main finding and relevance to this thesis
Xiao et al. (2024) ; Peng et al. (2024)	Mechanisms of positional degradation	Attention sinks and incomplete rotary position rotations explain why nominal context capacity outruns effective context use, grounding pointback style evidence selection mechanistically rather than metaphorically.
Ji et al. (2023)	Hallucination in natural language generation	Fluent outputs can be factually unsupported. Justifies treating apparent answer coherence as insufficient evidence of correctness at the first clarification boundary.
Shi et al. (2023) ; Yoran et al. (2024) ; Asai et al. (2024)	RAG failure modes	Good retrieval is not predictive of correct answering, and poor retrieval is sometimes recovered from. Justifies treating pinned context and answer jointly rather than relying on either alone.
Yao et al. (2023) ; Trivedi et al. (2023) ; Liu et al. (2024) ; Wolfson et al. (2026)	Agentic and iterative inference time reasoning	Agentic structure decomposes inference into observable intermediate operations. Produces the artefacts (clarification question, pinned context, intermediate clarification answer) that the thesis’s controllers consume.
Choi et al. (2018) ; Min et al. (2020) ; Kim et al. (2023)	Clarification and ambiguity handling	Clarification is the established mechanism for resolving ambiguity. Studies clarification <i>generation</i> and <i>answering</i> , but not when a clarification process should stop after it has begun.
Zhuang et al. (2025)	Clarification driven long context QA (AgenticLU)	Provides the inherited Chain of Clarifications workflow used as the technical foundation of the thesis. The thesis builds on its structure and isolates the stop/continue decision after the first round.
Thorne et al. (2018) ; Zheng et al. (2023) ; Yue et al. (2025)	Verification, fact checking, and LLM as judge	Generated answers can be assessed against evidence with useful but imperfect reliability. Justifies verifier structured judgements in the stopping gate, and motivates caution in treating any judge as ground truth.
Shi et al. (2025) ; Tam et al. (2024)	LLM as judge biases and structured prompting	Capable judges show non random position bias; format restrictions degrade generative reasoning but help decisional classification. Justifies the structured judgement design and motivates borderline review as a bias mitigation layer.

Authors / Year	Research area	Main finding and relevance to this thesis
Madaan et al. (2023); Shinn et al. (2023); Gou et al. (2024); Huang et al. (2023)	Refinement, reflection, and the limits of self correction	Iterative refinement can help when feedback is informative, but intrinsic self correction is unreliable and can degrade initially correct outputs. Supports treating continuation as conditional.
Chen et al. (2025); Hassid et al. (2025); Yoran et al. (2024)	Overthinking and test time compute scaling	Extended reasoning shows diminishing or inverse returns: longer chains can abandon previously correct answers, and additional retrieval or reasoning steps can introduce drift in QA. Strongest empirical case against an always continue policy.
Kadavath et al. (2022); Tian et al. (2023); Xiong et al. (2024); Yang et al. (2024)	Calibration and verbalised confidence	Internal probability is partially informative under privileged access; verbalised confidence works only for RLHF models under careful elicitation; vanilla prompts overconfide. Rules out model confidence as a safe primary stopping signal.
Yin et al. (2023); Feng et al. (2024)	Abstention and knowledge gap detection	LLMs systematically attempt to answer questions they should abstain from; multi LLM collaboration improves but does not solve this. Frames wrong stopping as the empirically dominant safety failure mode.
Geifman and El-Yaniv (2017); Kamath et al. (2020); Cole et al. (2023)	Selective prediction and selective QA	Frames prediction as a control problem: answer, abstain, or defer. Closest conceptual analogue to the stop/continue decision at the first clarification boundary.
Xin et al. (2020); Liu et al. (2020); Schwartz et al. (2020)	Adaptive computation and early exiting	Different examples warrant different amounts of inference compute. Supports per example clarification depth rather than fixed depth policies.
Wang et al. (2023); Aggarwal et al. (2023); Snell et al. (2024)	Sampling stopping rules and test time compute	Stopping policies in prior work decide over vote distributions or process reward signals; none decides over an explicit textual round state, leaving the present design space open.
Li et al. (2011); Dudík et al. (2011); Swaminathan and Joachims (2015)	Offline policy evaluation and counterfactual risk	Logged states allow policies to be compared without rerunning each one live; doubly robust estimators and counterfactual risk minimisation provide the statistical basis. Justifies the thesis’s fixed replay design over saved first round states.

Authors / Year	Research area	Main finding and relevance to this thesis
Elkan (2001)	Cost sensitive learning	Under asymmetric costs the optimal decision threshold is not 0.5 even for calibrated classifiers. Motivates evaluating the controller with cost asymmetric metrics rather than raw accuracy alone.
Brodersen et al. (2010) ; Chicco and Jurman (2020) ; Saito and Rehmsmeier (2015)	Evaluation under class imbalance	Raw accuracy is misleading when one class dominates or error types have different costs. Justifies balanced accuracy, continue recall, MCC and explicit four cell outcome analysis as the controller’s evaluation metrics.
Field and Welsh (2007)	Cluster bootstrap for clustered data	Resampling whole clusters rather than individual observations preserves within cluster dependence and yields confidence intervals appropriate for the base question cluster structure of the replay suite.

Table A.1 – Summary of literature relevant to adaptive stopping in clarification driven, long context question answering.

Appendix B

Methodology and Reproducibility Objects

B.1 AgenticLU round state saving method specification

B.1.1 Saved AgenticLU runtime round state schema

This section records the saved round-state object produced by the modified AgenticLU runtime. The object captures the clarification state generated during a live AgenticLU run, and is distinct from the offline controller fields added later: verifier records, refinement decisions, local risk flags, semantic review outputs and final policy decisions.

Field	Description	Origin	Methodological role
<code>example_id</code>	Unique identifier for the example	Dataset / runtime	Keys the example across saved state, replay row and metrics
<code>dataset</code>	Dataset name	Dataset / runtime	Supports per dataset attribution
<code>source_idx</code>	Source or context variant index	Runtime	Distinguishes context variants of one base question
<code>question</code>	Original benchmark question	Source benchmark	Input to clarification, verification and replay
<code>answers</code>	Saved benchmark answer set	Source benchmark	Used for case study judging and correctness evaluation
<code>clarification_rounds</code>	Saved clarification round records	Modified AgenticLU runtime	Preserves round trajectory
<code>round1_answer</code>	Provisional answer after the first clarification round	Modified AgenticLU runtime	Main input to the stopping gate
<code>provisional_answers_by_round</code>	Provisional answer states indexed by round	Modified AgenticLU runtime	Enables round comparison
<code>clarification_state</code>	Structured clarification state at the relevant boundary	Modified AgenticLU runtime	Provides clarification question, pinned context and intermediate answer
<code>context_token_count</code>	Token count of the constructed long context	HELMET / AgenticLU runtime	Reproducibility metadata
<code>context_pieces</code>	Context segments used in the long context input	HELMET / AgenticLU runtime	Evidence traceability
<code>final_answer</code>	Final answer returned by the run	AgenticLU runtime	Run output record

Table B.1 – Saved AgenticLU round state fields

Subfield	Description	Used by
<code>intermediate_question</code>	Clarification question generated at the relevant round	Case study, verifier prompt, deterministic gate
<code>pinned_context</code>	Evidence selected for the clarification step	Verifier prompt, grounding risk signals
<code>intermediate_answer</code>	Answer to the clarification question	Verifier prompt, local risk signals
<code>provisional_answer</code>	Provisional answer to the original question at the round boundary	Case study, offline gate

Table B.2 – Clarification state subfields

The exact JSON representation of the saved AgenticLU output is available in the public GitHub repository for this thesis.

B.1.2 Long context runtime execution and resumability

The AgenticLU live clarification runs were executed under a long context configuration. The runtime used a Llama 3.1-8B-Instruct base model, which supports a 128K token context window [77]. Running the inherited AgenticLU HELMET system at this context length required GPU hardware with sufficient memory, so the experiments could not be run reliably on a local machine. As such, they were conducted in Google Colab using an A100 GPU.

This execution environment introduced a practical reliability constraint. Colab executes notebook code inside an active runtime session, and files stored only within that runtime are not a reliable durable record because they may be lost when the runtime is disconnected, reset or deleted. This was relevant for the present experiments because individual multi round Chain of Clarifications examples could take several minutes to complete, and full dataset runs involved many examples. In practice, the number and duration of examples meant that runtime interruptions were expected rather than exceptional.

To address this constraint, the runtime was extended with persistent progress writing and resume logic. Completed examples and saved intermediate round states were written during execution to a mounted Google Drive directory, which functioned as durable storage across Colab runtime sessions. If a session ended or a run was interrupted, the experiment could therefore be resumed from the last saved state rather than discarding already completed outputs or repeating expensive long context inference.

This modification affected the reliability of the experimental pipeline, not the clarification

method itself. The resumability mechanism did not change the AgenticLU clarification workflow, the generated clarification questions, the pinned context, the intermediate clarification answers, or the provisional answers used in later evaluation. Its role was operational: to ensure that long running experiments could be completed and that saved round state objects were not lost across Colab runtime interruptions.

B.1.3 AgenticLU runtime reproducibility outline

This subsection documents the AgenticLU runtime used to produce the saved clarification states analysed in the thesis. The purpose is not to restate the full AgenticLU implementation, but to identify the runtime assumptions and thesis instrumentation needed to reproduce the saved state construction used by [Stage 1](#), [Stage 2](#) and [Stage 3](#). The base runtime configuration and the resumability mechanism are described in [Appendix B.1.2](#); the saved field definitions are given in [Table B.1](#).

The replay projection separated controller visible information from evaluation only information. The stopping policies received the original question, first round provisional answer, clarification question, pinned context and intermediate clarification answer, together with controller side verification or risk fields where relevant. Gold answers, correctness labels and future round outcomes were not available to the stopping policies; they were attached only after policy decisions had been produced, for metric computation. The rationale for comparing policies over fixed saved traces is set out in [Appendix B.7.1](#).

B.2 Answer judging protocol

This section records the judging rules used to assign round level correctness to provisional answers.

B.2.1 Case study: manual judging protocol

Round correctness in the case study was judged manually against the saved benchmark answer set. The judgement asked whether the answer at a given clarification boundary was sufficiently resolved to be accepted for stopping purposes. This turned on more than whether the answer contained an accepted gold string: it required that the answer commit to an accepted answer, leave no unresolved alternatives, and introduce no contradictory non gold claims.

The manual procedure applied the same core principle at each boundary: an answer was judged correct only where it gave a resolved answer compatible with the saved benchmark answer set. An answer was judged wrong where it failed to commit, left multiple candidates live, selected a non gold answer, or added unsupported specificity that changed the answer away from an accepted target.

Table B.3 gives representative examples of how the protocol was applied in the case study.

Table B.3 – Representative manual case study judging examples

Judgement pattern	Question family	Answer state being judged	Judgement	Reason for judgement
Direct commitment to accepted answer	Evolution proposed in 1859	The answer identified Charles Darwin as proposing evolution in 1859 through natural selection.	Correct	The answer committed to an accepted gold answer. Additional explanatory wording did not undermine the answer because it remained compatible with the saved benchmark answer set.
Acceptable broad temporal answer	Pull tabs on beer cans	The answer described pull tabs as being phased out or replaced during the 1970s.	Correct	The saved benchmark answer set accepted a 1970s framing. The answer did not need to provide a single exact date where the accepted target allowed the broader period.
Gold answer present but unresolved	Person who jumped from space	The answer left Felix Baumgartner and Joe Kittinger as live alternatives.	Wrong	Although the accepted answer appeared in the response, the answer did not commit to it. For stopping purposes, an unresolved candidate set was not treated as a correct answer.
Resolution of a previously unresolved answer	Person who jumped from space	The answer committed to Felix Baumgartner only.	Correct	The answer selected the accepted answer and no longer preserved the competing Joe Kittinger alternative as a live candidate.
Unsupported over specification	Pull tabs on beer cans	The answer shifted from a usable 1970s phase out answer to a specific 1982 claim tied to an irrelevant drinking age law.	Wrong	The added specificity moved the answer away from the accepted target. The answer was therefore judged wrong even though it appeared more specific.
Persistent wrong evidence path	Chris Cornell lyric	The answer proposed incorrect song titles such as “She’s Got the Sun” or “Change” rather than the accepted song title.	Wrong	The answer did not commit to the accepted gold answer and remained on an incorrect evidence path after clarification.
Unresolved candidate set	Singer of <i>A Thousand Years</i>	The answer left multiple possible singers live rather than committing to Christina Perri.	Wrong	The answer remained insufficiently resolved for stopping purposes. A likely candidate was not enough where the response preserved unresolved alternatives.

Judgement pattern	Question family	Answer state being judged	Judgement	Reason for judgement
Resolved entity answer	Singer of <i>A Thousand Years</i>	The answer selected Christina Perri as the singer.	Correct	The answer committed to the accepted answer and did not preserve competing candidates as live alternatives.

The examples in Table B.3 illustrate three aspects of the protocol. First, correctness turned on answer commitment rather than mere string presence: a response containing a gold answer was still judged wrong if it left an incompatible candidate unresolved. Second, specificity was not automatically rewarded; a more specific answer was judged wrong where the added specificity introduced an unsupported or contradictory claim. Third, the judgement was made at the saved clarification boundary, over the answer available at that round.

Manual adjudication was used for the case study because exact string matching and automated judgement were insufficient for several transition types: acceptable paraphrases, unresolved candidate sets, partial references to the gold answer, and gold strings embedded in non committal responses.

This procedure was used only for the diagnostic case study.

B.2.2 AgenticLU base judging protocol

This subsection documents the automated round correctness judging protocol used for the larger replay evaluation. The protocol assigned a correctness label to each generated answer at a given round, so that controller decisions could be scored against it. It should be distinguished from the VRRS-BR semantic borderline reviewer: the automated judge produced evaluation labels, whereas the reviewer was part of one stopping policy and was invoked only on deterministic borderline cases.

For each judged answer, the automated judge received the original question, the saved benchmark answer set, the generated answer for the relevant round and the round label. The saved judging rows contain the following fields:

- `question`;
- `gold_answers`;
- `predicted_answer`; and
- `round_label`.

The saved answer logs do not contain pinned context, clarification questions, intermediate clarification answers, verifier records, falsifier records or the full clarification trace. The au-

tomated judge therefore evaluated answer correctness against the question and saved benchmark answer set. It was not a grounding judge and did not inspect the full AgenticLU reasoning state.

The stored output schema consisted of three fields:

```
{
  "explanation": "string",
  "confidence": 0.0,
  "correct_answer": true
}
```

The replay label extraction used `gpt4_verification_round1.correct_answer` to construct the replay label `round1.correct`. This label was written into the offline stopping gate label artefacts and used to compute controller metrics including gate success, stop recall, continue recall, balanced accuracy, Matthews correlation coefficient and wrong stopped count.

The prompt form of the round level judging was:

```
Question: {question}
Correct answers: {gold_answers}
Answer: {predicted_answer}

Judge whether the answer is correct.
Return JSON with:
{
  "explanation": "...",
  "confidence": 0.0 to 1.0,
  "correct_answer": true or false
}
```

This prompt is reconstructed from the saved input and output artefacts rather than recovered verbatim from a prompt file.

This protocol supplied the `round1.correct` labels across the 408 replay examples, and therefore underlies the main controller metrics and the cluster statistical analysis.

B.2.3 Offline round judging protocol

The offline round judging procedure applied the AgenticLU judging protocol to saved provisional answers at explicit clarification boundaries. The thesis did not redesign the underlying correctness evaluation protocol. Instead, it used the saved round states produced by the

thesis instrumentation to apply the evaluator to answers that were previously only available as intermediate outputs.

For each judged item, the evaluator received the original benchmark question, the saved benchmark answer set and the provisional answer produced at the relevant round. The judgement was therefore made over the answer available at that clarification boundary. In the replay evaluation, the most important label was the correctness of the saved first round provisional answer, because that label determined the desired controller action: stop if the first round answer was correct, and continue if it was wrong.

Where second round or final answer states were available in the saved artefacts, the same procedure was applied to them separately. Each answer state was judged independently against the same benchmark answer set: a first round answer was not judged using the second round answer, and no future round outcome informed an earlier round label. Each label therefore reflects only the answer available at that point in the clarification trajectory.

The resulting labels were evaluation fields only. They were not available to the stopping policies when producing stop/continue decisions. In the offline replay pipeline, the saved first round state was first projected into a controller visible replay row, the policy decision was produced, and the correctness label was then attached for metric computation.

The automated round judging protocol is described in Appendix B.2.2. The limitations of the automated judging procedure, including observed inconsistency across some variants and the use of a fixed operational label set for policy comparison, are discussed in Appendix B.2.3.1.

B.2.3.1 Limitations of automated judging and consistency controls

The automated judging protocol was an operational correctness labelling procedure, not a source of stable ground truth. The replay evaluation depends on the `round1_correct` labels it produced, and those labels were suitable for scalable controller comparison while remaining judged labels rather than independently verified annotations.

The recovered artefacts showed several limitations. Some examples received inconsistent labels across context variants even where the question and generated answer were effectively the same. Some gold containing answers were accepted despite leaving multiple candidates unresolved. In other cases, answers matching the saved benchmark answer were rejected where the judge appeared to rely on external temporal knowledge. Table B.4 summarises representative examples.

Observed limitation	Representative example	Implication
Inconsistent labels across variants	The question “what is the name of the under butler in downtown abbey” with answer “Thomas Barrow” received both true and false labels across saved variants.	The automated judge was not perfectly stable across repeated variants.
Inconsistent labels on disputed factual cases	The question about the first baseball player featured on a postage stamp with answer “Babe Ruth” received inconsistent labels across variants.	Some factual or historically ambiguous examples were unstable under automated judgement.
Gold containing but unresolved answer judged correct	For “In what country is Willow River?”, the answer “Canada and the United States” was accepted because it included the United States, even though it left Canada live as another candidate.	The judge was not always strict about answer commitment or unresolved candidate sets.
External temporal knowledge overriding saved gold	For “who is the present rajya sabha speaker of india”, the answer “Venkaiah Naidu” matched the saved gold answer but was judged false as outdated.	The judge sometimes evaluated against external temporal knowledge rather than only the saved benchmark answer set.

Table B.4 – Known limitations of the automated round correctness judge

These limitations connect to the broader consistency issue in Appendix B.7.1: just as the policy comparison used fixed saved traces, the replay evaluation used a fixed stored label set, so that every policy was compared against the same operational target.

The judging outputs were inspected across two passes, and inconsistent or methodologically notable cases were reviewed manually to identify the main failure modes. This review interpreted the labels; it did not convert the replay suite into a manually adjudicated benchmark. The replay results should therefore be read as a relative comparison of stopping policies under a shared judged label protocol, with the manually adjudicated case study supplying separate qualitative evidence on continuation mechanisms.

B.3 Case study method specification

B.3.1 AgenticLU case study runtime reproducibility outline

The Stage 2 case study used saved continuation trajectories produced by the AgenticLU clarification runtime to examine how an answer changed when the system continued from the first to a second clarification round. It therefore required examples with both a first and a second round answer state.

The runtime followed the inherited CoC structure unchanged, and was continued for a second clarification round so that both round states were preserved for direct comparison. Each

saved case study object holds the original question, the first and second round clarification states, the first and second round provisional answers, and the benchmark answer set used for judging; each clarification state holds the clarification question, pinned context and intermediate clarification answer for its round.

The resulting case study subset should be interpreted as a diagnostic continuation subset rather than as a full Natural Questions evaluation set. It was constructed to support close analysis of continuation behaviour, including whether the second clarification round repaired an initially wrong answer, preserved an already correct answer, failed to repair an error, or degraded a previously correct answer.

Round answer correctness was assigned using the manual judging protocol of Appendix B.2.1, whose criteria fit the case study’s need to weigh answer commitment, unresolved alternatives, paraphrase and contradictory non gold claims. The full transition table for the 26 analysed examples is reported in Appendix C.1.3.

B.4 VRB method specification

This appendix specifies the Verification and Refinement Baseline (VRB) used in the offline first round stopping gate evaluation. VRB converts a saved first round clarification state into a single stop or continue decision by combining two structured LLM assessments, the verifier and the falsifier, with a deterministic ordered decision rule operating over their banded outputs. The banding scheme itself is introduced in Section 4.4.2 of the main text and is used by all three stopping policies.

The method has three stages. First, the saved first round state is projected into the shared decision context fields supplied to both the verifier and the falsifier. Second, the two structured assessments are produced as graded scores in $[0, 1]$ on each band typed field, projected into the Low, Moderate and High bands of Table 4.5. Third, the parsed records feed an ordered decision rule with two stages: a clean stop test that requires strong evidence on every relevant axis simultaneously, followed by an ordered cascade of continuation triggers that fires on the first risk condition that holds.

B.4.1 Effective first round inputs

VRB operated over the saved first round clarification state. The verifier and falsifier received the same core decision context: the original benchmark question, the clarification question generated at the first round, the pinned evidence selected for that clarification step, the intermediate clarification answer and the provisional answer to the original question.

Input	Source field
Original question	<code>question</code>
First round clarification question	<code>clarification_state.intermediate_question</code>
First round pinned evidence	<code>clarification_state.pinned_context</code>
First round clarification answer	<code>clarification_state.intermediate_answer</code>
First round provisional answer	<code>round1_answer</code>

Table B.5 – Effective VRB first round inputs

Gold answers, correctness labels and future round outcomes were not part of the VRB decision context. They were attached only after policy decisions had been produced, for evaluation and metric computation.

B.4.2 Verifier submethod

The verifier assessed whether the provisional first round answer was directly supported by the pinned evidence, complete for the original question, and safe to retain. It returned a structured JSON record rather than an open ended judgement.

B.4.2.1 Model specification

The verifier used GPT-4o at temperature 0.0, fixing the sampling temperature to improve consistency across replay examples.

B.4.2.2 Verifier prompt inputs

Prompt input	Source field
Original question	<code>question</code>
Current clarification question	<code>clarification_state.intermediate_question</code>
Current pinned evidence	<code>clarification_state.pinned_context</code>
Current clarification answer	<code>clarification_state.intermediate_answer</code>
Current provisional final answer	<code>round1_answer</code>

Table B.6 – Verifier prompt inputs and sources

B.4.2.3 Verifier prompt

The following prompt template was used to construct the structured verifier record from each saved first round clarification state.

```
You are verifying a provisional answer in a long context QA workflow.

Original question:
{ ... }

Current clarification question:
{ ... }

Current pinned evidence:
{ ... }

Current clarification answer:
{ ... }

Current provisional final answer:
{ ... }

Judge whether the current answer is directly supported, complete and safe to
keep as the best answer so far.
Only recommend another refinement if there is a concrete missing piece or a
real need to verify against alternatives.
Do not invent ambiguity such as unspecified time periods, positions, or roles
unless the evidence truly requires it.

Return exactly one JSON object:
{
  "direct_support": number between 0.0 and 1.0,
  "answer_complete": number between 0.0 and 1.0,
  "regression_risk": number between 0.0 and 1.0,
  "missing_info_type": "none" or a short label,
  "missing_info_text": "brief description of the concrete missing information",
  "refinement_expected_gain": number between 0.0 and 1.0,
  "keep_current_best": true or false,
  "recommend_refine": true or false,
  "rationale": "short explanation"
}
```

B.4.2.4 Verifier output schema

The verifier was required to return a JSON object with the following fields.

```
{
  "direct_support": 0.0,
  "answer_complete": 0.0,
  "regression_risk": 0.0,
  "missing_info_type": "none",
  "missing_info_text": "",
  "refinement_expected_gain": 0.0,
  "keep_current_best": true,
  "recommend_refine": false,
  "rationale": ""
}
```

An example verifier output is shown below.

```
{
  "direct_support": 1.0,
  "answer_complete": 0.9,
  "regression_risk": 0.1,
  "missing_info_type": "none",
  "missing_info_text": "",
  "refinement_expected_gain": 0.0,
  "keep_current_best": true,
  "recommend_refine": false,
  "rationale": "The current answer is directly supported by the evidence and is
    complete, with only a small risk of regression. No further refinement is
    expected to gain significant information.",
  "parse_error": null
}
```

B.4.2.5 Verifier output field meanings

Field	Meaning
<code>direct_support</code>	Estimated probability that the answer is directly supported by the pinned evidence
<code>answer_complete</code>	Estimated probability that the answer is complete for the original question
<code>regression_risk</code>	Estimated risk that further refinement would degrade the current answer
<code>missing_info_type</code>	Type of missing information, if any
<code>missing_info_text</code>	Short description of the missing information
<code>refinement_expected_gain</code>	Estimated value of another refinement step
<code>keep_current_best</code>	Whether the current answer should remain the best answer
<code>recommend_refine</code>	Whether another refinement step is recommended
<code>rationale</code>	Short explanation of the verifier judgement

Table B.7 – Verifier JSON field meanings

B.4.3 Falsifier submethod

The falsifier stress tested the provisional answer for concrete failure modes that support and completeness alone may miss: not to confirm the answer, but to test whether an answer that read as fluent or locally coherent was nonetheless unsafe to accept.

At the first clarification boundary, the falsifier used the same saved first round decision context as the verifier. Because the thesis evaluates the first round stopping boundary, there was no earlier best answer to compare against at this point.

B.4.3.1 Falsifier prompt inputs

Prompt input	Source field
Original question	question
first round clarification question	clarification_state.intermediate_question
first round pinned evidence	clarification_state.pinned_context
first round clarification answer	clarification_state.intermediate_answer
first round provisional answer	round1_answer

Table B.8 – Falsifier prompt inputs and sources

B.4.3.2 Falsifier prompt

The following prompt template was used to construct the falsifier assessment.

```

You are stress testing a provisional answer in a long context QA workflow.
Your job is not to confirm the answer. Your job is to identify whether the
current answer may still be wrong even if it looks coherent.

Original question:
{ ... }

Current clarification question:
{ ... }

Current pinned evidence:
{ ... }

Current clarification answer:
{ ... }

Current provisional final answer:
{ ... }

Focus on three risks:
1. The answer may be unsupported by the right evidence even if the current
chain sounds coherent.
2. A plausible competing answer may fit the question better.
3. The current evidence selection may have locked onto the wrong entity, date,
role, or relation.

```

Return exactly one JSON object:

```
{
  "false_answer_risk": number between 0.0 and 1.0,
  "alternative_answer_risk": number between 0.0 and 1.0,
  "evidence_selection_risk": number between 0.0 and 1.0,
  "issue_type": "none" or a short label,
  "issue_text": "brief description of the most likely issue",
  "continue_for_falsification": true or false,
  "rationale": "short explanation"
}
```

B.4.3.3 Falsifier output schema

The falsifier was required to return a JSON object with the following fields.

```
{
  "false_answer_risk": 0.0,
  "alternative_answer_risk": 0.0,
  "evidence_selection_risk": 0.0,
  "issue_type": "none",
  "issue_text": "brief description of the most likely issue",
  "continue_for_falsification": true,
  "rationale": "short explanation"
}
```

B.4.3.4 Falsifier output field meanings

Field	Meaning
false_answer_risk	Estimated risk that the answer is false
alternative_answer_risk	Estimated risk that a competing answer fits the question or evidence better
evidence_selection_risk	Estimated risk that the answer relies on incomplete or incorrect evidence selection
issue_type	Short label for the suspected issue
issue_text	Short description of the suspected issue
continue_for_falsification	Whether continuation is recommended to test or repair the answer
rationale	Short explanation of the falsifier judgement

Table B.9 – Falsifier JSON field meanings

B.4.4 VRB ordered decision rule

The parsed verifier and falsifier records are consumed by a deterministic ordered rule that emits the policy’s stop or continue decision. The rule operates over the band projections of the verifier and falsifier fields and over the two binary verifier flags (v , the refinement recommendation, and m , the missing information type flag). The two stage structure of clean stop test followed by ordered continuation cascade is motivated and discussed in Section 4.4.3.5; this subsection states the rule as it is applied at offline replay.

B.4.4.1 Clean stop test

The clean stop test fires first and evaluates the joint verifier and falsifier evidence. It accepts the provisional answer only when every condition in Table B.10 holds at once. If any one condition fails, control passes to the ordered continuation cascade in Section B.4.4.2.

Condition	Field	Required band
Direct support is high	Verifier d	High
Answer completeness is high	Verifier c	High
Expected refinement gain is low	Verifier q	Low
False answer risk is low	Falsifier f	Low
Alternative answer risk is low	Falsifier a	Low
Evidence selection risk is low	Falsifier e	Low

Table B.10 – Clean stop conditions over the parsed verifier and falsifier records.

When all six conditions hold, the rule emits **Stop** with the decision reason **already supported and complete**. The asymmetry between this stage and the continuation cascade is deliberate; the clean stop test demands strong evidence on multiple axes simultaneously because the cost of an unsafe stop is accepting a wrong answer, whereas the cost of an unnecessary continuation is a single additional clarification round (Section 4.4.3.5).

B.4.4.2 Ordered continuation cascade

If the clean stop test does not fire, the rule evaluates the continuation triggers of Table B.11 in order. The first trigger to fire produces the policy decision and its decision reason; if no trigger fires the rule defaults to **Stop** with reason **insufficient gain for refinement**.

#	Trigger	Condition
1	High false answer risk	Falsifier f is High
2	Plausible alternative answer	Falsifier a is High
3	Evidence selection risk	Falsifier e is High
4	Verifier recommends refinement	Verifier v is true and verifier q is High
5	Concrete missing information	Verifier m is true and verifier q is High

Table B.11 – Ordered continuation triggers used by VRB.

Triggers 1–3 fire on a single falsifier risk projecting into the High band. Triggers 4 and 5 require both a verifier indication that another round is warranted (v true, or m concrete) and an at least High expected refinement gain (q), so the verifier signals can produce continuation only when corroborated by the verifier’s own gain estimate.

B.4.4.3 Default stop

If neither the clean stop test nor any continuation trigger fires, the rule emits **Stop** with the decision reason **insufficient gain for refinement**. The default to stop behaviour is consistent with the cost framing of the controller task: continuation is emitted only where a specific rule branch has identified a reason for it, so that the policy does not inflate the predicted continue count by treating residual uncertainty as evidence of risk (Section 4.4.3.5).

The complete rule, including the clean stop test, the ordered cascade and the default branch, is stated procedurally in Algorithm 4.1 of the main text.

B.5 VRRS method specification

This appendix specifies the Verification and Refinement with Risk Signals (VRRS) policy used in the offline stopping gate evaluation. VRRS extended the Verification and Refinement Baseline with a deterministic risk signal layer over the saved replay state and verifier fields, converting observable features of the round state into explicit stop, continue, or borderline bands. It called no additional LLM and used no semantic reviewer.

VRRS is the deterministic middle policy in the family: VRB supplies the structured verification and refinement record, VRRS builds its own decision from local rule signals over that record and the saved state, and VRRS-BR reuses the same banding but escalates only borderline cases to semantic review.

B.5.1 Effective inputs

VRRS operated over the saved first round replay state and the structured verifier record produced during the Verification and Refinement construction. It did not use gold answers, correctness labels, future round outcomes, or a semantic reviewer.

Input	Source
question	Saved replay row
round1.answer	Saved replay row
clarification.state.intermediate.question	Saved clarification state
clarification.state.pinned.context	Saved clarification state
clarification.state.intermediate.answer	Saved clarification state
saved.verifier.record.parsed.output .direct.support	Verification component
saved.verifier.record.parsed.output .answer.complete	Verification component
saved.verifier.record.parsed.output .refinement.expected.gain	Verification component
saved.verifier.record.parsed.output .regression.risk	Verification component
saved.verifier.record.parsed.output .recommend.refine	Verification component
saved.verifier.record.parsed.output .missing.info.type	Verification component

Table B.12 – Deterministic gate inputs

B.6 VRRS-BR method specification

This appendix specifies the Verification and Refinement with Risk Signals and Borderline Review (VRRS-BR) policy used in the offline first round stopping gate evaluation. VRRS-BR extended the deterministic VRRS policy with a constrained semantic review stage for cases assigned to the deterministic **borderline** band. The reviewer was a strict control layer judge, not an answer generator: it decided only whether an existing first round answer contained a concrete likely error that made stopping unsafe.

As the final policy in the family, VRRS-BR preserved VRRS’s automatic stop and continue decisions unchanged and acted only on the deterministic borderline band, routing those cases to semantic review.

B.6.1 Semantic review trigger rule

Semantic review was triggered only where the deterministic VRRS controller returned a borderline decision band:

```
{
  "decision_band": "borderline"
}
```

If the deterministic gate returned **stop** or **continue**, GPT-4o was not called. Those deci-

sions were preserved unchanged. This routing rule ensured that semantic review was used selectively, rather than as a general second pass judge over the full replay suite.

Borderline condition	Rule
Accumulated soft risk under verifier concern	Two or more soft signals active and the VRB high risk signal active

Table B.13 – Semantic review trigger condition

B.6.2 Prompt inputs

The semantic review prompt used the first round decision context and local risk information already constructed by the controller. The reviewer received the original question, the first round clarification trace, the first round provisional answer, the saved verifier summary and the local deterministic risk flags.

Prompt input	Source
Original question	Replay row
First round clarification question	Saved clarification state
First round pinned context	Saved clarification state
First round clarification answer	Saved clarification state
First round provisional answer	Replay row
Verifier summary	Verification and refinement component
Local deterministic risk flags	Deterministic Risk Signal Gate

Table B.14 – Semantic review prompt inputs

B.6.3 Reviewer role and full prompt

The prompt placed four constraints on the reviewer: it could not answer the question from scratch; it could recommend continuation only on a concrete likely error; weak grounding or general uncertainty was not sufficient on its own; and it had to return a structured JSON object.

The full prompt template was:

```
You are deciding whether a QA system should stop after round 1 or continue to
round 2.
```

Do not answer the question yourself from scratch.

Only judge whether the current round 1 answer is too risky to stop on.

Original question: {question}

Round 1 clarification question:

{intermediate_question}

Round 1 pinned context:

{pinned_context}

Round 1 clarification answer:

{intermediate_answer}

Round 1 provisional final answer:

{round1_answer}

Saved verifier summary:

```
{
  "direct_support": ...,
  "answer_complete": ...,
  "refinement_expected_gain": ...,
  "recommend_refine": ...,
  "missing_info_type": ...,
  "regression_risk": ...
}
```

Local deterministic risk flags for this example:

```
{
  "hard_continue_reasons": ...,
  "soft_risk_reasons": ...,
  "support": ...,
  "complete": ...,
  "gain": ...
}
```

Decision policy:

- Continue only if you can point to a concrete reason the answer is likely wrong .
- 'Not directly grounded enough' by itself is not enough to continue.
- Concrete reasons include: explicit contradiction, wrong entity selected,

- competing answer, answer type mismatch, or clue echo error.
- If the answer may be under grounded but you cannot identify a concrete likely error, choose stop.

Return exactly one JSON object:

```
{
  "continue_to_round2": true or false,
  "false_answer_risk": number between 0.0 and 1.0,
  "issue_type": "contradiction" | "wrong_entity" | "competing_answer" | "
    answer_type_mismatch" | "clue_echo" | "under_grounding_only" | "none",
  "concrete_error_detected": true or false,
  "reason": "short explanation"
}
```

B.6.4 Required output JSON fields

The reviewer was required to return a JSON object with the following fields.

```
{
  "continue_to_round2": false,
  "false_answer_risk": 0.0,
  "issue_type": "none",
  "concrete_error_detected": false,
  "reason": ""
}
```

B.6.5 Output field meanings

Field	Meaning
continue_to_round2	Raw reviewer recommendation on whether to continue
false_answer_risk	Estimated risk that the first round answer is false
issue_type	Type of issue identified by the reviewer
concrete_error_detected	Whether a concrete likely error was detected
reason	Short explanation of the reviewer decision

Table B.15 – Semantic review JSON fields

B.6.6 Allowed concrete issue types

A semantic review continuation recommendation was eligible to pass the final filter only if the identified issue type belonged to the predefined concrete error set. Table B.16 lists the canonical issue types used by the controller.

Canonical issue type	Meaning
<code>contradiction</code>	The first round answer conflicts with the available evidence
<code>wrong_entity</code>	The answer refers to the wrong person, place, organisation, or object
<code>competing_answer</code>	A different answer remains more plausible or unresolved
<code>answer_type_mismatch</code>	The answer form does not match the question requirement
<code>clue_echo</code>	The answer appears to echo wording from the question rather than resolve it

Table B.16 – Semantic review issue types

Note. Implementation variants using spaced or underscored forms were normalised to these canonical issue types. The prompt also allowed `under_grounded_only` and `none`, but those issue types were not eligible to pass the continuation filter.

B.6.7 Final review filter

A raw recommendation to continue became a final continue decision only if all of the conditions in Table B.17 were satisfied.

Condition	Required value
Continue recommendation	<code>continue_to_round2 == true</code>
Concrete error	<code>concrete_error_detected == true</code>
Issue type	In the allowed concrete error set
False answer risk	<code>false_answer_risk</code> is High

Table B.17 – Semantic review continuation filter

If any condition failed, the raw continuation recommendation was rejected and the case

returned to a stop decision. The filter therefore prevented the semantic stage from acting as a broad uncertainty detector: the reviewer could overturn the deterministic outcome only on a concrete likely error meeting the predefined conditions.

B.6.8 Worked semantic review examples

This subsection works through two borderline cases, showing how each was reviewed, parsed and passed through the final continuation filter.

B.6.8.1 Semantic review stop example

In this example, the deterministic gate assigned the case to the borderline band, but the semantic reviewer did not identify a concrete likely error. The final decision therefore remained stop.

Field	Value
Example ID	kilt_hotpotqa:0:0734e514c53a:e517fb675f84
Question	After Patrick Mercer's suspension in April 2014, who won the seat for Newark?
First round provisional answer	Robert Jenrick.
Deterministic status before review	Borderline, with soft risks including answer missing from pinned context, meta clarification, meta intermediate answer and high confidence without grounding.
Reviewer outcome	Stop; no concrete error detected.
Final VRRS-BR decision	Stop.

Table B.18 – Worked VRRS-BR semantic review stop example

The semantic reviewer returned:

```
{
  "continue_to_round2": false,
  "false_answer_risk": 0.1,
  "issue_type": "none",
  "concrete_error_detected": false,
  "reason": "The answer is consistent with the context provided and there is no
    indication of error or contradiction. The information is complete and
    directly addresses the question."
}
```

```
}

```

This output failed the continuation filter because the reviewer did not recommend continuation, did not detect a concrete error and placed false answer risk in the Low band. The borderline case therefore remained a stop decision.

B.6.8.2 Semantic review continue example

In this example, the deterministic gate assigned the case to the borderline band, and the semantic reviewer identified a concrete contradiction between the provisional answer and the pinned context. The final filter accepted the continuation recommendation.

Field	Value
Example ID	kilt_nq:54:a0ae6cbd6a3c:9fc9250344f5
Question	During which season does Cape Town receive rainfall?
First round provisional answer	Cape Town receives rainfall during the rainy season, which is from September to April, with the most significant rainfall occurring in October and November.
Deterministic status before review	Borderline, with soft risks including answer missing from pinned context, meta clarification, meta intermediate answer, broad pinned context and high confidence without grounding.
Reviewer outcome	Continue; contradiction detected with high false answer risk.
Final VRRS-BR decision	Continue.

Table B.19 – Worked VRRS-BR semantic review continue example

The semantic reviewer returned:

```
{
  "continue_to_round2": true,
  "false_answer_risk": 0.8,
  "issue_type": "contradiction",
  "concrete_error_detected": true,
  "reason": "The provisional final answer contradicts the pinned context, which
    states that Cape Town receives most of its rainfall during the winter months
    (June to August), not from September to April."
}
```

This output passed the continuation filter because `continue_to_round2` was true, `concrete_error_detected` was true, the issue type was an allowed concrete error type, and `false_answer_risk` fell in the High band. The borderline case was therefore converted into a final continue decision.

B.7 Policy implementation

B.7.1 Justification for consistent datasets and CoC traces

The policy comparison used a fixed replay suite rather than generating a fresh AgenticLU clarification trajectory for each stopping policy. The AgenticLU clarification path contains LLM generated intermediate objects, so even with the benchmark question and long context input held constant, separate runs can produce different clarification questions, pinned context, intermediate answers and provisional answers. Had each policy been run on its own freshly generated trace, a better result could reflect the easier trace a policy happened to receive rather than better stopping logic. The same concern applies to the verifier and falsifier records, on which all three policies depend: recomputing them per policy would add further nondeterminism.

The replay suite removes this confound. Every policy is evaluated on the same reconstructed first round decision object, and on the same VRB derived records where applicable, so differences in outcome are attributable to controller logic rather than to different generation traces. The replay example, a realised first round clarification state, is therefore the unit of analysis. This does not make the suite a universal estimate: it reflects the particular saved AgenticLU runs, and the results should be read as controller behaviour over that constructed suite. For comparing stopping policies, that trade-off is the point, as it is what preserves the internal validity of the comparison.

Appendix C

Results and Supporting Objects

C.1 Case study supporting results

This section collects the supporting material for the continuation case study: the question family reference table, the family level transition counts, the full transition table for all 26 trajectories, and representative trajectories for each transition category.

C.1.1 Example Family Id Mapping

Family ID	Short label	Quoted question family
F1	Chris Cornell lyric	“chris cornell she’s going to change the world”
F2	Carthage leader	“leader of carthage in the first punic war”
F3	Pull tabs on beer cans	“when did they stop making pull tabs on beer cans”
F4	Person who jumped from space	“who is the guy that jumped from space”
F5	Evolution proposed in 1859	“who proposed evolution in 1859 as the basis of biological development”
F6	Singer of “A Thousand Years”	“who sings ive loved you for a thousand years”

Table C.1 – Case study question family reference table.

C.1.2 Family case study counts

Family	N	Transition counts				Short interpretation
		$W \rightarrow C$	$C \rightarrow W$	$W \rightarrow W$	$C \rightarrow C$	
F1	3	0	0	3	0	Clarification did not rescue this family. The model stayed on the wrong song or shifted to another wrong song.
F2	6	0	0	0	6	Clarification was neutral. The answer was already correct and stayed correct.
F3	5	0	2	1	2	Mixed family. Some round 2 answers preserved the accepted 1970s answer, while others drifted into unsupported 1980s claims.
F4	6	6	0	0	0	Clarification consistently helped by collapsing ambiguity onto Felix Baumgartner.
F5	1	0	0	0	1	Round 2 was redundant; the answer was already correct after round 1.
F6	5	1	0	3	1	Mixed family. One example improved, several stayed wrong and one stayed correct.
Total	26	7	2	7	10	

Table C.2 – Case study family transition counts.

C denotes a correct answer and W denotes a wrong or unresolved answer. Thus, repaired means $W \rightarrow C$, harmed means $C \rightarrow W$, unrepaired means $W \rightarrow W$, and preserved means $C \rightarrow C$.

C.1.3 Full case study transition table

Example ID	Fam.	R1 → R2	Answer movement	Judgement rationale
nq_8ee775aa499d	F1	Wrong → Wrong	R1: The answer referred to “She’s Got the Sun” or an unspecified Chris Cornell lyric. R2: The answer shifted to “Change” by Chris Cornell.	Clarification did not help: the answer stayed wrong after the second round.
nq_3ed3b2aabec6	F1	Wrong → Wrong	R1: The answer repeatedly circled around “She’s Got the Sun” and other incorrect lyric possibilities. R2: The answer shifted to “Change” by Chris Cornell.	Clarification did not help: the answer stayed wrong after the second round.
nq_1399a92f81df	F1	Wrong → Wrong	R1: The answer proposed “She’s Got the Sun” or “She’s Got the Power”. R2: The answer shifted to “I’d Love to Change the World”.	Clarification did not help: the answer stayed wrong after the second round.
nq_87a4a58e865d	F2	Correct → Correct	R1: Hanno the Great was selected as the single leader after noting other Carthaginian figures. R2: Hanno the Great remained the answer.	Clarification was neutral: the answer was already correct and stayed correct.
nq_0d082dba83ab	F2	Correct → Correct	R1: Hanno the Great was selected as the single leader after noting other Carthaginian figures. R2: Hanno the Great remained the answer.	Clarification was neutral: the answer was already correct and stayed correct.
nq_c62e3cbae75e	F2	Correct → Correct	R1: Hanno the Great was selected as the single leader after noting other Carthaginian figures. R2: Hanno the Great remained the answer.	Clarification was neutral: the answer was already correct and stayed correct.
nq_1a3f475f14e9	F2	Correct → Correct	R1: Hanno the Great was selected as the single leader after noting other Carthaginian figures. R2: Hanno the Great remained the answer.	Clarification was neutral: the answer was already correct and stayed correct.
nq_a32cac0c6465	F2	Correct → Correct	R1: Hanno the Great was selected as the single leader after noting other Carthaginian figures. R2: Hanno the Great remained the answer.	Clarification was neutral: the answer was already correct and stayed correct.
nq_3a336b624b28	F2	Correct → Correct	R1: Hanno the Great was selected as the single leader after noting other Carthaginian figures. R2: Hanno the Great remained the answer.	Clarification was neutral: the answer was already correct and stayed correct.

Table C.3 – Full case study transition table, grouped by family: F1-F2.

Example ID	Fam.	R1 → R2	Answer movement	Judgement rationale
nq_3892f38146f2	F3	Correct → Wrong	R1: Pull tabs were described as largely replaced in the 1970s, with no exact date provided. R2: The answer shifted to the 1980s, specifically 1982.	Clarification hindered: the second round damaged an initially correct answer.
nq_fc3f2f4081e5	F3	Correct → Correct	R1: Pull tabs were described as declining in the 1970s due to litter and safety concerns. R2: The answer preserved the 1970s transition framing.	Clarification was neutral: the answer was already correct and stayed correct.
nq_85338856409a	F3	Correct → Wrong	R1: Pull tabs were described as declining in the 1970s and being replaced by other mechanisms. R2: The answer shifted to “By 1980”.	Clarification hindered: the second round damaged an initially correct answer.
nq_194e63c0aee7	F3	Wrong → Wrong	R1: The answer gave a vague 1970s or 1980s range. R2: The answer shifted toward the early 1980s.	Clarification did not help: the answer stayed wrong after the second round.
nq_6e69246bee5c	F3	Correct → Correct	R1: Pull tabs were described as phased out in the mid 1970s. R2: The answer preserved the mid 1970s answer.	Clarification was neutral: the answer was already correct and stayed correct.

Table C.4 – Full case study transition table, grouped by family: F3.

Example ID	Fam.	R1 → R2	Answer movement	Judgement rationale
nq_fda7ae764def	F4	Wrong → Correct	R1: Felix Baumgartner and Joe Kittinger remained live candidates. R2: Felix Baumgartner was selected as the person who jumped from space.	Clarification helped: the second round repaired an initially wrong or unresolved answer.
nq_de57eab056e1	F4	Wrong → Correct	R1: Felix Baumgartner and Joe Kittinger remained live candidates. R2: Felix Baumgartner was selected as the person who jumped from space.	Clarification helped: the second round repaired an initially wrong or unresolved answer.
nq_b5d5a7ab972b	F4	Wrong → Correct	R1: Felix Baumgartner and Joe Kittinger remained live candidates. R2: Felix Baumgartner was selected as the person who jumped from space.	Clarification helped: the second round repaired an initially wrong or unresolved answer.
nq_dd9400778aad	F4	Wrong → Correct	R1: Felix Baumgartner and Joe Kittinger remained live candidates. R2: Felix Baumgartner was selected as the person who jumped from space.	Clarification helped: the second round repaired an initially wrong or unresolved answer.
nq_bdc5518aacc0	F4	Wrong → Correct	R1: Felix Baumgartner and Joe Kittinger remained live candidates. R2: Felix Baumgartner was selected as the person who jumped from space.	Clarification helped: the second round repaired an initially wrong or unresolved answer.
nq_cdbb73368e25	F4	Wrong → Correct	R1: Felix Baumgartner and Joe Kittinger remained live candidates. R2: Felix Baumgartner was selected as the person who jumped from space.	Clarification helped: the second round repaired an initially wrong or unresolved answer.

Table C.5 – Full case study transition table, grouped by family: F4.

Example ID	Fam.	R1 → R2	Answer movement	Judgement rationale
nq_1339b94c35d4	F5	Correct → Correct	R1: Charles Darwin proposed evolution in 1859 through his theory of natural selection. R2: Charles Darwin remained the answer, with the same natural selection explanation.	Clarification was neutral: the answer was already correct and stayed correct.
nq_48fba1d46f1e	F6	Wrong → Correct	R1: Christina Perri and Ronan Parke were both mentioned, with Christina Perri treated as the likely candidate. R2: Christina Perri was selected as the most likely singer of the relevant song.	Clarification helped: the second round repaired an initially wrong or unresolved answer.
nq_aff041fc6148	F6	Wrong → Wrong	R1: Celine Dion was proposed as a possible singer. R2: Celine Dion remained the proposed singer.	Clarification did not help: the answer stayed wrong after the second round.
nq_0f322d8a1130	F6	Wrong → Wrong	R1: Celine Dion was given as the singer. R2: Celine Dion remained the answer.	Clarification did not help: the answer stayed wrong after the second round.
nq_0d365cede437	F6	Wrong → Wrong	R1: Celine Dion or Sarah McLachlan were left as possible answers. R2: Celine Dion or Sarah McLachlan remained unresolved possible answers.	Clarification did not help: the answer stayed wrong after the second round.
nq_b75bcaa3ad2c	F6	Correct → Correct	R1: Christina Perri was identified as the singer. R2: Christina Perri remained the answer.	Clarification was neutral: the answer was already correct and stayed correct.

Table C.6 – Full case study transition table, grouped by family: F5-F6.

Note. The family IDs are defined in Table C.1. The four transition columns describe if the second clarification round repaired an initially wrong answer, harmed an initially correct answer, failed to repair a wrong answer, or preserved an already correct answer.

C.1.4 Representative case study transition examples

This subsection gives one representative trajectory for each of the four transition categories used in Chapter 5, making the aggregate counts concrete. The full 26 example transition table is reported in Appendix C.1.3.

Transition example	first round answer state	Second round answer state	Interpretation
Wrong → Correct Person who jumped from space	Felix Baumgartner and Joe Kittinger remained live candidates.	The second round committed to Felix Baumgartner.	Continuation repaired an unresolved answer by collapsing ambiguity onto the accepted answer.
Correct → Wrong Pull tabs on beer cans	The first round answer gave a usable 1970s phase out answer.	The second round shifted to a specific 1982 claim tied to an irrelevant drinking age law.	Continuation harmed the answer through unsupported over specification.
Wrong → Wrong Chris Cornell lyric	The first round answer proposed an incorrect or unresolved song candidate.	The second round remained on an incorrect song path.	Continuation failed to repair the evidence path.
Correct → Correct Evolution proposed in 1859	The first round answer identified Charles Darwin.	The second round preserved Charles Darwin as the answer.	Continuation was safe but redundant.

Table C.7 – Representative case study transition examples.

C.2 Controller evaluation result tables

C.2.1 Complete combined controller comparison

Policy	N	Gate success	Gate error	Balanced accuracy	MCC	Predicted continue
VRB	408	313 (76.72%)	95 (23.28%)	66.10%	0.346	84
VRRS	408	301 (73.77%)	107 (26.23%)	66.38%	0.321	110
VRRS-BR	408	316 (77.45%)	92 (22.55%)	73.99%	0.450	127

Table C.8 – Combined controller comparison: summary metrics.

C.2.2 Complete confusion breakdown by policy

Policy	Correct stopped	Correct continued	Wrong stopped	Wrong continued
VRB	267	38	57	46
VRRS	248	57	50	53
VRRS-BR	247	58	34	69

Table C.9 – Combined controller comparison: confusion cells.

Correct stopped and wrong continued are successful gate decisions. Correct continued and wrong stopped are gate errors.

C.2.3 Per dataset controller result tables

Dataset	Policy	Correct stopped	Correct continued	Wrong stopped	Wrong continued
HotpotQA	VRB	66	11	16	9
HotpotQA	VRRS	66	11	16	9
HotpotQA	VRRS-BR	65	12	10	15
NQ	VRB	60	7	22	13
NQ	VRRS	48	19	16	19
NQ	VRRS-BR	48	19	13	22
PopQA	VRB	58	12	8	24
PopQA	VRRS	54	16	7	25
PopQA	VRRS-BR	54	16	7	25
TriviaQA	VRB	83	8	11	0
TriviaQA	VRRS	80	11	11	0
TriviaQA	VRRS-BR	80	11	4	7

Table C.10 – Per dataset controller confusion counts.

Correct stopped and wrong continued are successful gate decisions. Correct continued and wrong stopped are gate errors.

Dataset	Policy	Gate success	Balanced accuracy	MCC	Continue recall	Predicted continue
HotpotQA	VRB	75 (73.53%)	60.86%	0.235	36.00%	20
HotpotQA	VRRS	75 (73.53%)	60.86%	0.235	36.00%	20
HotpotQA	VRRS-BR	80 (78.43%)	72.21%	0.433	60.00%	27
NQ	VRB	73 (71.57%)	63.35%	0.319	37.14%	20
NQ	VRRS	67 (65.69%)	62.96%	0.255	54.29%	38
NQ	VRRS-BR	70 (68.63%)	67.25%	0.334	62.86%	41
PopQA	VRB	82 (80.39%)	78.93%	0.562	75.00%	36
PopQA	VRRS	79 (77.45%)	77.63%	0.523	78.12%	41
PopQA	VRRS-BR	79 (77.45%)	77.63%	0.523	78.12%	41
TriviaQA	VRB	83 (81.37%)	45.60%	-0.101	0.00%	8
TriviaQA	VRRS	80 (78.43%)	43.96%	-0.121	0.00%	11
TriviaQA	VRRS-BR	87 (85.29%)	75.77%	0.419	63.64%	18

Table C.11 – Per dataset controller performance metrics.

C.2.4 Borderline review result tables

This subsection reports the saved borderline review outputs used to interpret the contribution of VRRS-BR. These tables support the main result that semantic borderline review changed only a small subset of decisions, but that the accepted changes were concentrated on wrong first round answers.

Item	Count
Borderline cases identified by VRRS	288
Cases sent to semantic review	288
Raw semantic review continue recommendations	23
Raw semantic review stop recommendations	265
Final stop to continue changes	17
Filtered back to stop by final continuation filter	6
Final preserved stops	271

Table C.12 – Borderline review decision outcomes.

Final outcome among borderline cases	Count
Correct stopped	237
Correct continued	1
Wrong stopped	34
Wrong continued	16

Table C.13 – Final outcomes among borderline cases.

Parsed issue type	Count
none	228
under_grounded_only	37
wrong_entity	19
competing_answer	3
contradiction	1

Table C.14 – Semantic review issue distribution.

Note. *Filtered back to stop* refers to raw semantic review continuation recommendations that failed the final continuation filter. *Final stop to continue changes* refers to raw continuation recommendations that passed the filter. The issue types `none` and `under_grounded_only` were not eligible to pass the final continuation filter.

C.3 Policy example bank: gate decision and failure mode evidence

This section gives qualitative examples drawn from the saved policy decision outputs, showing how representative first round states were classified by VRB, VRRS and VRRS-BR across correct stops, useful continuations, wrong stops, continuation costs, semantic review decisions and final filter behaviour.

C.3.1 Stratified gate decision example bank

Table C.15 reports representative examples from the main controller outcome cells. The examples were selected to cover correct stopping, wrong stopping, wrong continuation and correct continuation cost.

Ex.	Dataset / policy	Question and first round answer	Decision / outcome	Decision evidence
1	HotpotQA VRB	Q: After Patrick Mercer’s suspension in April 2014, who won the seat for Newark? R1: Robert Jenrick.	Stop Correct stopped	The verifier assigned support High , completeness High and expected gain Low , producing <code>already_supported_and_complete</code> .
2	HotpotQA VRB	Q: Cagliostro-Walzer op.370 was composed by a man born in what year? R1: Johannes Brahms was given as the composer and 1833 as the birth year.	Stop Wrong stopped	The verifier assigned support High , completeness High and gain Low , causing the wrong first round answer to be accepted.
3	HotpotQA VRB	Q: James Leal Greenleaf worked on a monument located where? R1: The Lincoln Memorial and several American battlefield cemeteries in France and Belgium.	Continue Wrong continued	The verifier recommended refinement with support High , completeness High and expected gain High .
4	HotpotQA VRRS	Q: James Leal Greenleaf worked on a monument located where? R1: The Lincoln Memorial and several American battlefield cemeteries in France and Belgium.	Continue Wrong continued	VRRS fired <code>verifier_internal_risk</code> ; soft risks included answer missing from pinned context, meta clarification and meta intermediate answer.
5	HotpotQA VRRS	Q: After Patrick Mercer’s suspension in April 2014, who won the seat for Newark? R1: Robert Jenrick won the by election for Newark in 2014.	Continue Correct continued	VRRS fired <code>verifier_internal_risk</code> ; this illustrates the continuation cost side of the deterministic signal gate.

Table C.15 – Stratified gate decision example bank.

Example identifiers for Examples 1-5 are listed in Table C.16.

Ex.	Example identifier
1	<code>kilt_hotpotqa:0:0734e514c53a:e517fb675f84</code>
2	<code>kilt_hotpotqa:15:de3386b14d69:9a23150b4cf1</code>
3	<code>kilt_hotpotqa:54:a052ce7fe0c4:ec7f3acb79ba</code>
4	<code>kilt_hotpotqa:54:a052ce7fe0c4:ec7f3acb79ba</code>
5	<code>kilt_hotpotqa:2:0734e514c53a:86e367ccb954</code>

Table C.16 – Example identifiers for the stratified gate decision example bank.

C.3.2 Borderline review example bank

Table C.17 reports representative VRRS-BR examples. These examples show semantic review preserving a stop, converting a borderline case into continuation and filtering raw continuation recommendations back to stop.

Dataset	Question and first round answer	Review result	Interpretation
HotpotQA	Q: After Patrick Mercer’s suspension in April 2014, who won the seat for Newark? R1: Robert Jenrick.	Issue: none Risk: Low Decision: Stop	The reviewer found no concrete error. The deterministic borderline case remained a correct stop.
NQ	Q: During which season does Cape Town receive rainfall? R1: September to April, with most rainfall in October and November.	Issue: contradiction Risk: High Decision: Continue	The reviewer identified a contradiction with pinned context stating that rainfall is mostly in winter. The case passed the final filter.
PopQA	Q: In what country is Long Lake? R1: Long Lake is located in Canada.	Issue: competing_answer Risk: High Decision: Stop	The reviewer recommended continuation, but false answer risk was below the High threshold, so the recommendation was filtered back to stop.
NQ	Q: Who is the current director of the US Mint? R1: David J. Ryder, as of April 2018.	Issue: competing_answer Risk: Moderate Decision: Stop	The reviewer raised possible outdatedness, but the risk was too low to pass the final filter. The stop decision was preserved.
TriviaQA	Q: In which Dickens’ novel does lawyer Mr Jaggers appear? R1: Great Expectations and Bleak House.	Issue: wrong_entity Risk: High Decision: Continue	The reviewer identified that Mr Jaggers appears in <i>Great Expectations</i> , not <i>Bleak House</i> ; the case passed the final filter.

Table C.17 – Borderline review example bank.

C.3.3 Failure mode and trade off examples

Table C.18 summarises the main qualitative failure modes and trade offs observed in the stopping gate evaluation. These examples are used to support the interpretation that VRRS-BR improved wrong answer detection while accepting some additional continuation cost.

Pattern and example	Controller behaviour	Interpretation
Premature stopping on a wrong answer <i>Cagliostro-Walzer / composer birth year</i>	VRB stopped on an incorrect first round answer after high verifier support and completeness.	Demonstrates the risk of accepting a fluent but wrong answer when verifier confidence is too high.
Useful continuation on a wrong answer <i>James Leal Greenleaf monument location</i>	VRB and VRRS continued the wrong first round answer because expected refinement gain or verifier risk was high.	Demonstrates a successful controller action even though downstream repair is a separate question.
Continuation cost on a correct answer <i>Newark by election / Robert Jenrick</i>	VRRS continued a correct first round answer because verifier risk fired.	Demonstrates the efficiency cost introduced by stronger risk sensitivity.
Semantic review correction <i>Cape Town rainfall season</i>	VRRS-BR converted a borderline case into continuation after detecting contradiction.	Demonstrates the benefit of constrained semantic review in a high risk borderline case.
Filtered raw continuation <i>Long Lake country</i>	The semantic reviewer recommended continuation, but the final filter rejected it because false answer risk was below threshold.	Demonstrates that the semantic reviewer did not have unrestricted authority over the final decision.
Dataset specific semantic gain <i>Mr Jagers / Dickens novel</i>	VRRS-BR continued a wrong TriviaQA answer after detecting a wrong entity issue.	Demonstrates why TriviaQA improved strongly under semantic borderline review.

Table C.18 – Failure mode and trade off examples.

C.4 Statistical robustness and supplementary figures

This appendix provides the statistical robustness material supporting the controller evaluation in Chapter 5. It tests whether the descriptive policy pattern remains plausible once the replay suite is treated according to its clustered structure. The 408 replay rows are not wholly independent, because multiple rows are context variants of the same underlying base question, so the analyses below use the base question cluster as the resampling and comparison unit.

The statistical robustness results should be interpreted as internal robustness evidence for the fixed replay suite, not as a population level benchmark significance claim. The procedures test whether the observed policy differences remain visible when repeated context variants

are grouped by base question.

C.4.1 Base question cluster definition

The base question cluster was the unit used for cluster resampling and paired comparison. In the implemented statistical analysis, a cluster was defined by the pair:

$$\text{cluster_key} = (\text{dataset}, \text{question}). \quad (\text{C.1})$$

Replay rows were grouped where they came from the same dataset and shared the same exact question string, so each cluster represented one underlying semantic question across its saved context variants. This produced 85 base question clusters across the 408 replay rows.

Dataset	Base question clusters	Variants per cluster	Replay rows
HotpotQA	34	3	102
Natural Questions	17	6	102
PopQA	17	6	102
TriviaQA	17	6	102
Total	85	Mixed by dataset	408

Table C.19 – Base question cluster structure of the replay suite

The replay suite was complete under this design. Each HotpotQA cluster contained three replay rows, and each Natural Questions, PopQA and TriviaQA cluster contained six replay rows. No clusters were partially represented in the 408 row replay suite.

C.4.2 Cluster bootstrap procedure

The primary uncertainty procedure was a cluster bootstrap. The resampling unit was the base question cluster rather than the individual replay row. Resampling was stratified by dataset so that each bootstrap draw preserved the dataset composition of the replay suite.

The bootstrap used 4000 repetitions and random seed 42. In each repetition, clusters were sampled with replacement within each dataset. When a cluster was sampled, all replay rows belonging to that cluster were included in the reconstructed bootstrap sample. Controller metrics were then recomputed for each policy on the reconstructed sample.

The following pseudocode summarises the procedure:

- (1) Group replay rows by base question cluster, where each cluster is defined as (`dataset`, `question`).
- (2) Split the cluster keys by dataset.
- (3) For each bootstrap repetition:
 - (a) sample the same number of clusters with replacement within each dataset;
 - (b) include all replay rows belonging to each sampled cluster;
 - (c) recompute controller confusion cells and metrics for each policy;
 - (d) store the resulting metric values.
- (4) For each policy and metric, report the mean over bootstrap draws and the 2.5th and 97.5th percentiles.

The bootstrap script stored five metrics for each draw: gate success, balanced accuracy, continue recall, Matthews correlation coefficient and predicted continue rate. The main Results chapter reports the compact set of balanced accuracy, continue recall and Matthews correlation coefficient because these are the metrics most directly related to class balanced controller behaviour.

C.4.3 Cluster aware bootstrap intervals

Table C.20 reports the cluster aware bootstrap intervals used in Chapter 5. The reported means are the means over bootstrap draws, and the intervals are percentile intervals from the 2.5th and 97.5th percentiles.

Policy	Balanced accuracy mean	95% interval	Continue recall mean	95% interval	MCC mean	95% interval
VRB	66.14%	[58.40%, 73.93%]	44.81%	[30.12%, 60.38%]	0.345	[0.186, 0.495]
VRRS	66.24%	[58.01%, 74.46%]	51.19%	[35.71%, 66.67%]	0.318	[0.156, 0.478]
VRRS-BR	73.89%	[65.95%, 81.46%]	66.82%	[52.29%, 80.36%]	0.448	[0.295, 0.590]

Table C.20 – Cluster bootstrap intervals

Table C.21 reports the observed point estimates on the original 408 replay rows. These values should be read alongside the bootstrap intervals above. The observed results show the descriptive controller profile, while the bootstrap intervals show how the main metrics vary under clustered resampling.

Policy	Gate success	Stop recall	Continue recall	MCC	Correct stopped	Correct continued	Wrong stopped	Wrong continued	Predicted continue
VRB	76.72%	87.54%	44.66%	0.346	267	38	57	46	84 (20.59%)
VRRS	73.77%	81.31%	51.46%	0.321	248	57	50	53	110 (26.96%)
VRRS-BR	77.45%	80.98%	66.99%	0.450	247	58	34	69	127 (31.13%)

Table C.21 – Observed controller point estimates on the 408 example replay suite

The bootstrap intervals support the descriptive pattern reported in the main chapter. VRRS-BR had the highest bootstrap mean balanced accuracy, continue recall and Matthews correlation coefficient. The intervals are relatively wide and should not be interpreted as precise population estimates. Their role is to show that the fixed suite descriptive pattern remains visible when the question cluster, rather than the individual replay row, is treated as the resampling unit.

C.4.4 Paired cluster comparisons

The paired cluster comparisons used the base question cluster as the paired unit. For each policy and cluster, the cluster success rate was computed as the mean gate success rate across replay rows in that cluster. A row was counted as successful where the policy stopped on a correct first round answer or continued on a wrong first round answer.

For a comparison from policy A to policy B:

- a *win* meant that policy B had a higher cluster success rate than policy A;
- a *loss* meant that policy B had a lower cluster success rate than policy A;
- a *tie* meant that both policies had the same cluster success rate.

All 85 clusters were included in the mean difference calculation. Two paired procedures were applied. The first was an exact two sided binomial sign test over the non tied clusters. The second was a Monte Carlo sign flip permutation test with 50,000 repetitions and random seed 42. The permutation statistic was the absolute mean signed per cluster difference, with random sign flips applied to the non zero cluster differences.

Comparison	Mean success rate difference	Wins	Losses	Ties	Exact sign test p	Sign flip permutation p
VRB → VRRS	-0.0235	1	7	77	0.0703	0.3030
VRRS → VRRS-BR	+0.0392	8	1	76	0.0391	0.0505
VRB → VRRS-BR	+0.0157	8	7	70	1.0000	0.6322

Table C.22 – Paired cluster comparisons on mean gate success rate

The paired cluster level comparisons are useful for interpreting the structure of the policy differences. The strongest paired support was for the movement from VRRS to VRRS-BR. The direct VRB to VRRS-BR comparison was more mixed on paired gate success rate, because VRRS-BR changed the error distribution rather than merely increasing raw success in every cluster. Its improvement was clearer in class balanced metrics and wrong stopped reduction than in paired raw success alone.

C.4.5 Cluster collapsed McNemar checks

A secondary cluster collapsed McNemar style check was used as an additional robustness analysis. This check collapsed each cluster to a single majority success or failure state for each policy.

For each policy and cluster:

- if the cluster gate success rate was greater than 0.5, the cluster was collapsed to *success*;
- if the cluster gate success rate was less than 0.5, the cluster was collapsed to *failure*;
- if the cluster gate success rate was exactly 0.5, the cluster was treated as tied and excluded from that policy comparison.

For a comparison from policy A to policy B, the discordant cells were:

- *A fail / B success*, where the cluster moved from majority failure under A to majority success under B;
- *A success / B fail*, where the cluster moved from majority success under A to majority failure under B.

The exact test was a two sided exact binomial McNemar style test over the discordant clusters. The procedure began with all 85 clusters, then excluded clusters where either policy had a majority tied cluster state.

Comparison	A fail / B success	A success / B fail	Discordant clusters	Excluded tied clusters	Exact p
VRB → VRRS	1	1	2	7	1.0000
VRRS → VRRS-BR	4	0	4	6	0.1250
VRB → VRRS-BR	5	1	6	8	0.2188

Table C.23 – Cluster collapsed McNemar exact checks

This check was intentionally secondary. It restores a paired binary structure, but it discards within cluster variation by collapsing each cluster to a majority outcome. For this reason, it is less sensitive than the cluster bootstrap and paired cluster success rate comparisons. The VRRS to VRRS-BR comparison remained directionally favourable, with four clusters moving from majority failure to majority success and no clusters moving in the opposite direction, but the exact p-value was not small after the cluster collapsed transformation.

C.4.6 Supplementary controller and statistical figures

This subsection provides supplementary interval figures for the cluster bootstrap results. The balanced accuracy interval figure is reported in the main Results chapter. The two supplementary figures below show the corresponding continue recall and Matthews correlation coefficient intervals.

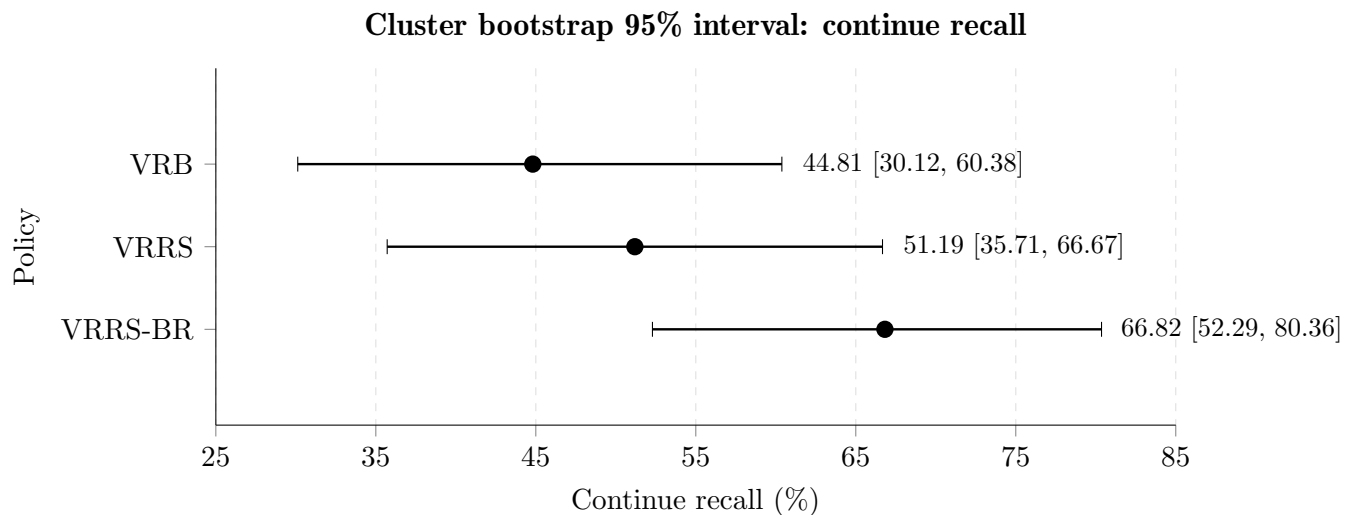


Figure C.1 – Cluster bootstrap continue recall intervals

Note. Points show bootstrap means and horizontal bars show cluster bootstrap 95% intervals.

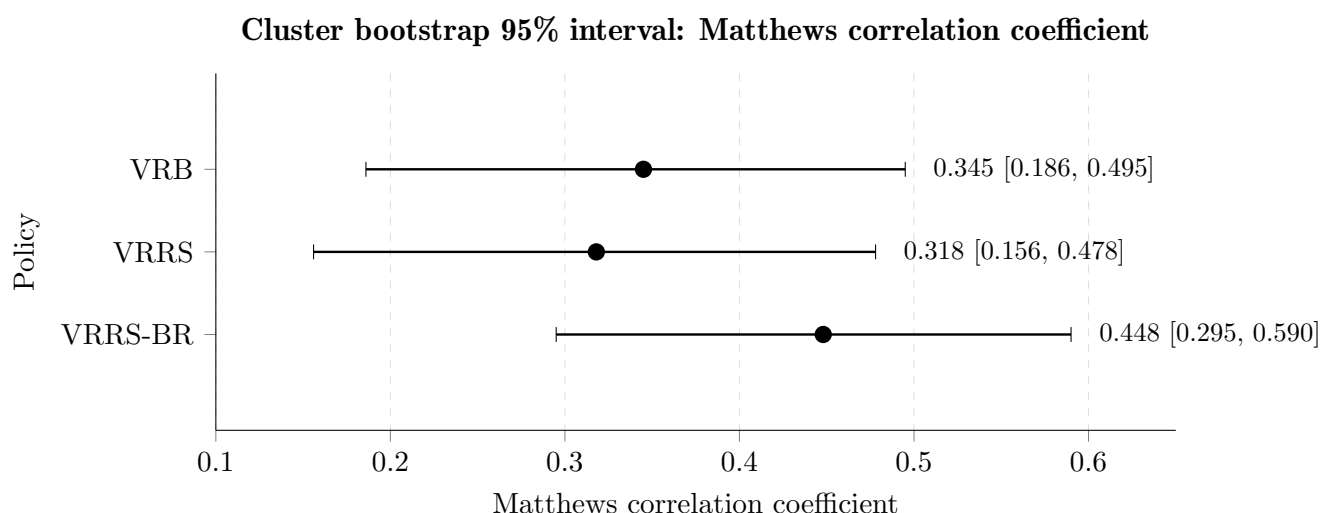


Figure C.2 – Cluster bootstrap MCC intervals

Note. Points show bootstrap means and horizontal bars show cluster bootstrap 95% intervals.

These supplementary figures reinforce the same interpretation as the main balanced accuracy interval figure. VRRS-BR has the highest bootstrap mean for continue recall and Matthews correlation coefficient, but the intervals remain broad because the analysis treats base question clusters, rather than replay rows, as the resampling unit.

C.4.7 Band placement sweep: full data

This appendix reports the full sweep data underlying Section 5.6.4. Each of the 11 band placements was perturbed across its defined range in steps of 0.025, and every policy decision was re run over the 408 example replay suite at each step.

The presentation proceeds in three parts. Table C.24 summarises the outcome of every sweep on a single page, recording the design time setting, the sweep range, the number of unique confusion outcomes observed across the range, and a short note. Figure C.3 then plots all four confusion cell counts (safe stop, justified continuation, premature stop, unnecessary continuation) for every placement across its full sweep range, making the regions of stability and the two regions of movement visually obvious at a glance. Finally, Figures C.4 and C.5 zoom in on the two placements whose outcomes varied across their sweep range, allowing the transitions to be read off precisely.

Summary table

The table below is the principal reference for this appendix. Of the 11 band placements, nine are completely stable across their sweep range, one (VRRS *r*) shows a one decision change

at the upper edge of its range, and one (VRRS-BR reviewer f) shows two transitions in the interior of its range.

Placement	Design time	Sweep range	Unique outcomes	Note
VRB clean stop d	High	[0.6, 1.0]	1	Stable across sweep
VRB clean stop c	High	[0.6, 1.0]	1	Stable across sweep
VRB gate q	Low	[0.0, 0.4]	1	Stable across sweep
VRB falsifier f	Low	[0.0, 0.4]	1	Stable across sweep
VRB falsifier a	Low	[0.0, 0.4]	1	Stable across sweep
VRB falsifier e	Low	[0.0, 0.4]	1	Stable across sweep
VRRS clean stop d	High	[0.6, 1.0]	1	Stable across sweep
VRRS clean stop c	High	[0.6, 1.0]	1	Stable across sweep
VRRS clean stop q	Low	[0.0, 0.4]	1	Stable across sweep
VRRS r	Low	[0.0, 0.4]	2	One decision change at the upper edge; not enough to affect the reported results
VRRS-BR reviewer f	High	[0.6, 1.0]	3	Transitions at 0.725, 0.8, and 0.825; expected because this is an important reviewer threshold, and all changes moved in the intended direction

Table C.24 – Summary of the band placement sweep over the offline replay suite.

Per placement sweeps

Figure C.3 reports the four confusion cell counts as a function of band value, one panel per placement. The same colour scheme is used in every panel: teal circles are safe stops, orange triangles are justified continuations, red squares are premature stops, and blue diamonds are unnecessary continuations. High side placements (design time = High) are swept over [0.6, 1.0]; low side placements (design time = Low) are swept over [0.0, 0.4]. The y-axis is fixed at [0, 280] across all panels so that magnitudes are visually comparable. Panels with completely flat lines indicate placements where every band value in the range produces the same 408 example confusion matrix as the design time setting.

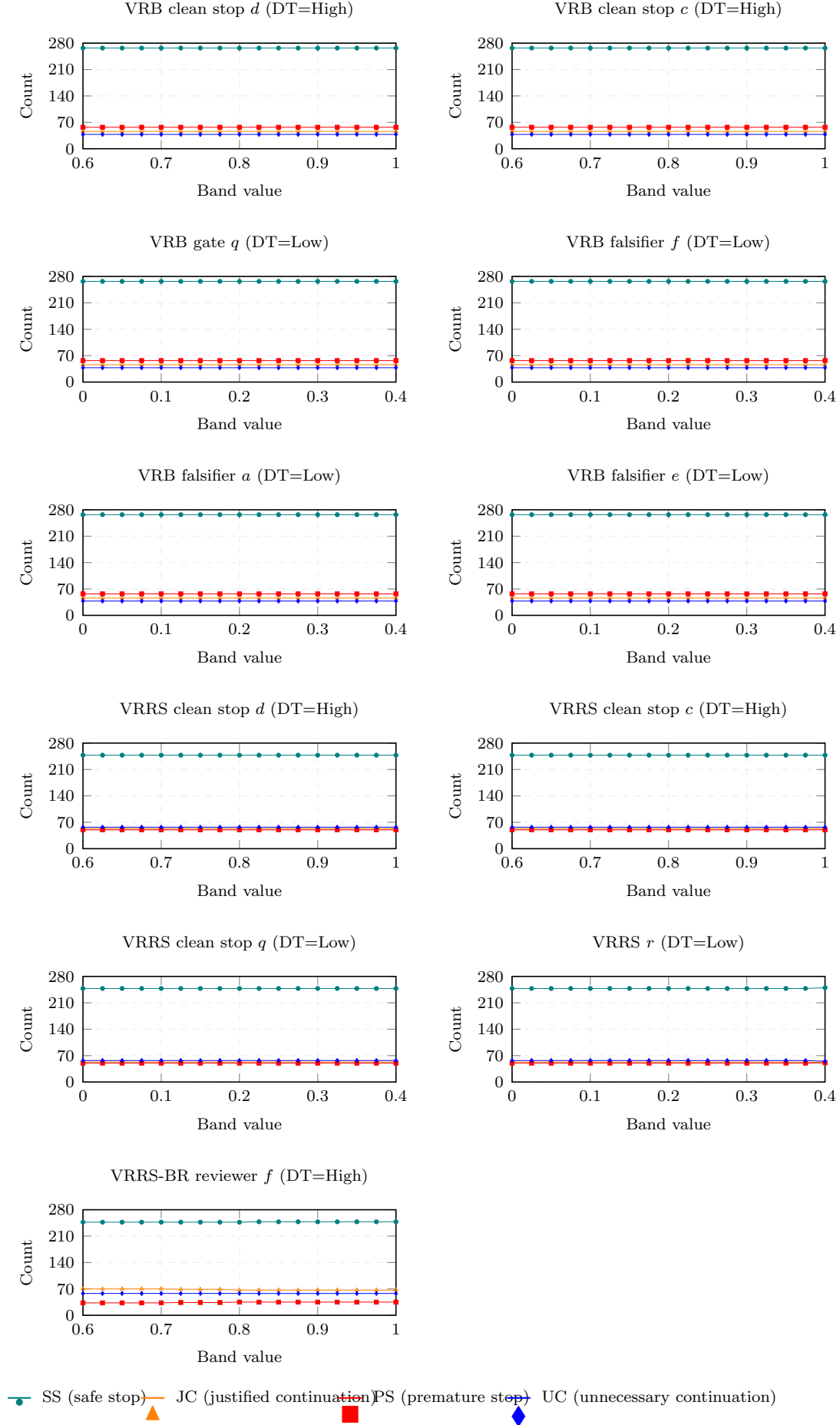


Figure C.3 – Confusion cell counts under sweep of each band placement.

Note. Each panel shows safe stop (teal circle), justified continuation (orange triangle), premature stop (red square) and unnecessary continuation (blue diamond) counts across the full sweep range for one band placement. Note that the band placement is different for each panel, indicating that the band placement is not constant across all panels.

Detail plots for placements that moved

The two figures below enlarge the panels for the two placements whose confusion cell counts changed across the sweep. Each detail plot uses the same colour scheme as Figure C.3 but at a larger scale, so that the exact transition points and the magnitude of each step can be read directly from the axes.

VRRS regression risk placement. The first detail figure (Figure C.4) shows the sweep for the VRRS placement r . This placement is stable across almost all of its sweep range, with a single decision flipping at the upper edge of the range. The flip is in the correct direction (one fewer unnecessary continuation, one additional safe stop), so the change is conservative and would not alter any of the headline results reported in Section 5.6.4.

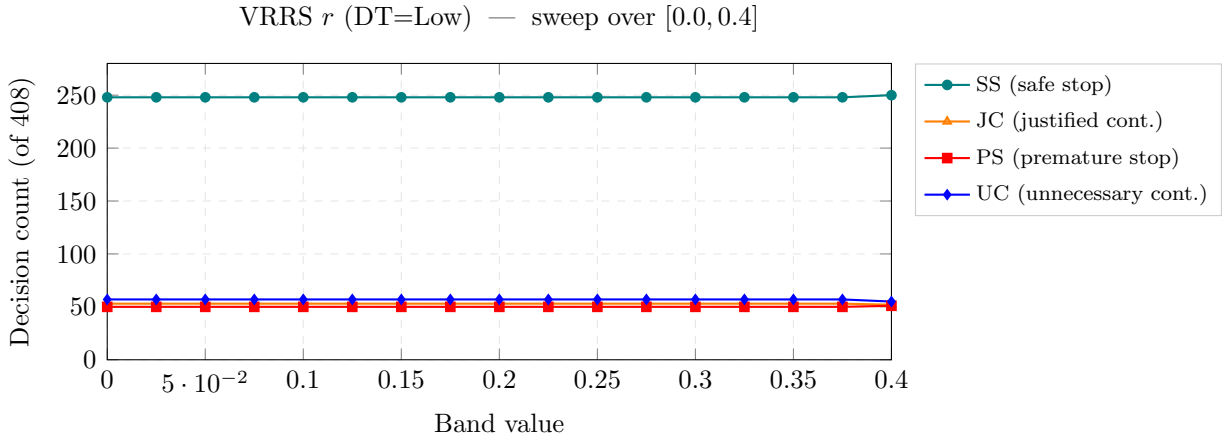


Figure C.4 – Detail sweep for the VRRS regression risk placement.

Note. A single decision change occurs at the upper edge of the sweep (0.400), where one previously unnecessary continuation becomes a safe stop. The four cells are otherwise flat across the sweep range.

VRRS-BR reviewer false answer risk placement. The second detail figure (Figure C.5) shows the sweep for the VRRS-BR reviewer placement f . This is the most active placement in the suite, with three small interior transitions at 0.725, 0.8, and 0.825. Despite the additional movement, every transition moves the controller in the intended direction with respect to the trust hierarchy design: lowering the filter admits an additional justified continuation, and raising it trades justified continuations for premature stops and one additional safe stop, leaving the controller more conservative on the safety side.

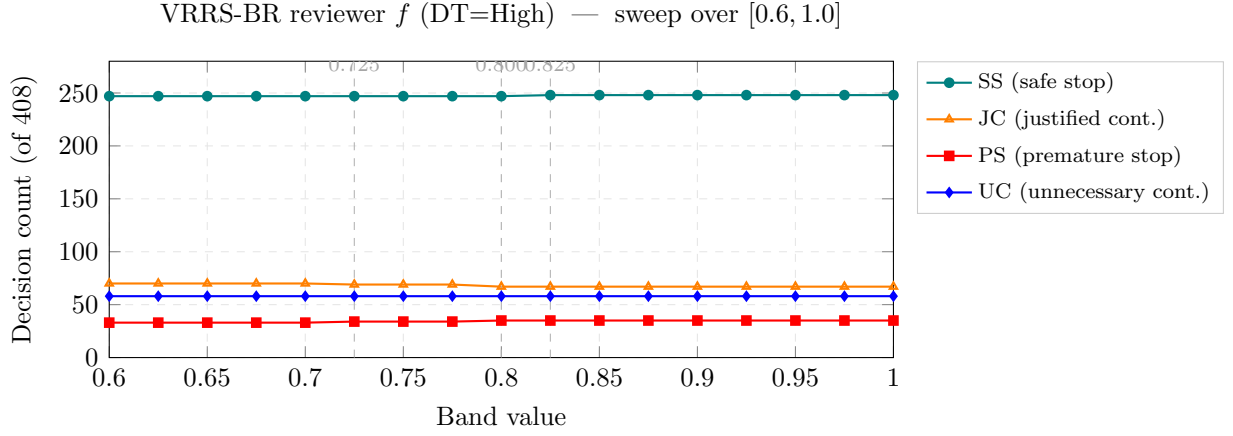


Figure C.5 – Detail sweep for the VRRS-BR reviewer’s false answer risk placement.

Note. Lowering the filter below 0.725 admits one additional justified continuation (70 instead of 69) at the cost of one fewer premature stop (33 instead of 34). Raising the filter above 0.800 trades two further justified continuations for two additional premature stops, and crossing 0.825 converts one previously premature stop into a safe stop. The unnecessary continuation count is unchanged across the entire sweep. All movements push the controller in the conservative direction expected from the trust hierarchy design of the filter. Dashed vertical guides mark the three transition points.